

Designing Multi-Agent Simulations with EDSL

An experimental manual for economic games, deliberation, markets, and shared-state workflows

Expected Parrot

August 2026

Contents

1	Status and purpose	4
2	Shared state from first principles	5
2.1	Creating a shared state	5
2.2	The primitive lifecycle	7
2.3	How a question reads state	9
2.4	How an answer becomes an operation	9
2.5	Scopes: one world or many	10
2.6	Reading, auditing, and closing	11
2.7	Choosing a primitive	11
2.8	What is saved for remote execution	12
3	From primitives to a complete simulation	13
3.1	A minimal simulation	13
4	Authoring personas and assignments	16
4.1	Behavioral traits	16
4.2	Pairing, grouping, and roles	16
5	Checked question text and state reads	18
6	Configuring mechanisms	19
7	Protocols and role-specific questions	21
8	Scheduling and visibility	22
8.1	Simultaneous action	22
8.2	Sequential roles within concurrent groups	22
8.3	Repeated interaction	22
9	Running, resuming, and inspecting	24
10	Economic games	25
10.1	Matrix games: prisoner's dilemma and stag hunt	25
10.2	Ultimatum, dictator, and trust games	26
10.3	Nash demand and the 11–20 request game	26
10.4	Public goods, punishment, and common-pool extraction	27
10.5	Beauty contest and market entry	28
10.6	Auctions and continuous double auctions	28

10.7 Signaling, adverse selection, cheap talk, and moral hazard	29
10.8 Centipede and bilateral negotiation	30
10.9 Voting and strategic voting	30
10.10 Information cascades	31
11 Deliberation and organizational workflows	32
11.1 Message boards and rumor diffusion	32
11.2 Agenda setting and legislative amendments	33
11.3 Work queues and incident response	33
11.4 Resource allocation and disaster response	34
11.5 Matching markets	35
11.6 Coalition formation	35
11.7 Forecast revision and Delphi panels	36
11.8 Prediction markets and private news	36
11.9 Hiring committees	37
11.10 Strategic planning workshops	37
11.11 Customer-feedback synthesis	38
11.12 Launch readiness review	39
11.13 General synthesis and tiered review	39
12 Validation as part of the research interface	41
12.1 Mechanism validation	41
12.2 Assignment validation	41
12.3 Protocol validation	41
12.4 Schedule validation	42
12.5 Storage and resume validation	42
12.6 Error messages	42
13 Reproducibility and reporting	43
14 What this manual reveals about the proposed design	44
15 Example coverage index	46
16 Complete current implementations	48
16.1 economic_game_11_20_money_request.py	48
16.2 economic_game_adverse_selection.py	50
16.3 economic_game_centipede.py	53
16.4 economic_game_cheap_talk.py	55
16.5 economic_game_continuous_double_auction.py	58
16.6 economic_game_information_cascade.py	61
16.7 economic_game_market_entry.py	63
16.8 economic_game_moral_hazard.py	64
16.9 economic_game_nash_demand.py	67
16.10 economic_game_public_goods_punishment.py	68
16.11 economic_game_repeated_prisoners_dilemma.py	71
16.12 economic_game_signaling.py	73
16.13 economic_game_strategic_voting.py	76
16.14 economic_game_ultimatum.py	78
16.15 economic_game_voting_rules.py	83

16.16economic_games_auction_comparison.py	85
16.17economic_games_group.py	87
16.18economic_games_matrix.py	89
16.19economic_games_transfer.py	91
16.20shared_state_adversarial_hiring_committee.py	94
16.21shared_state_bilateral_negotiation.py	98
16.22shared_state_binary_prediction_market.py	101
16.23shared_state_budget_allocation.py	103
16.24shared_state_coalition_formation.py	105
16.25shared_state_congested_matching_market.py	108
16.26shared_state_customer_feedback_synthesis.py	111
16.27shared_state_delphi_forecast.py	116
16.28shared_state_disaster_response.py	121
16.29shared_state_family_message_board.py	124
16.30shared_state_forecast_revision.py	127
16.31shared_state_hiring_committee.py	130
16.32shared_state_incident_response.py	133
16.33shared_state_launch_readiness_review.py	136
16.34shared_state_legislative_amendments.py	145
16.35shared_state_live_review_queue.py	147
16.36shared_state_matching_market.py	149
16.37shared_state_meeting_agenda.py	151
16.38shared_state_peer_review_matching.py	154
16.39shared_state_prediction_market_private_news.py	156
16.40shared_state_public_goods.py	159
16.41shared_state_rumor_diffusion.py	163
16.42shared_state_strategic_planning_tiered.py	165
16.43shared_state_strategic_planning_workshop.py	172

17 Closing perspective	179
-------------------------------	------------

Chapter 1

Status and purpose

This is a design manual for researchers who want to run multi-agent simulations with EDSL. It is also a test of a proposed programming interface. The examples are based on 43 working simulations in the EDSL repository, covering economic games, markets, group deliberation, allocation, forecasting, and organizational workflows.

The syntax in this manual is intentionally **prospective**. Some elements work in the experimental shared-state branch; others are proposed conveniences. A box labeled **Design pressure** identifies places where the proposed interface is incomplete or where an apparently simple abstraction conceals an important research choice.

The manual has two goals:

1. Show an academic reader how to specify a reproducible simulation: who acts, what each actor sees, which actions are legal, when actions occur, and how outcomes are computed.
2. Force the proposed API to confront diverse examples before it becomes a stable public interface.

This is not a claim that language-model simulations reproduce human behavior. They are programmable computational experiments. Their validity depends on persona construction, prompt design, model choice, stochastic replication, measurement, and comparison with external evidence.

Chapter 2

Shared state from first principles

Ordinary EDSL surveys treat interviews as independent: an agent answers questions, and those answers become a row in **Results**. A shared-state simulation adds a durable object that can be changed by one interview and read by another. That object is how a later player sees an earlier offer, how a committee sees submitted reviews, or how several workers claim different jobs.

The current experimental implementation has three layers:

Layer	Current object	Responsibility
Container	<code>SharedState</code>	Names the simulation scope and collects primitives
Storage	<code>FileStateStore</code>	Persists an append-only event log
Primitive	<code>SharedLog</code> , <code>SharedWorkPool</code> , <code>ConfiguredSharedGame</code> , etc.	Defines legal operations and the reader-facing view

A **scope** is the string identifier for one independent shared world. In the current implementation it can be any string chosen by the researcher: `"seminar-1"`, `"pair-17"`, or `"treatment-a/replication-4"`. The string has no built-in scientific meaning; its meaning comes from the experimental design. The current constructor even accepts an empty string, which is legal by accident rather than a useful design choice; creation should require a nonempty, serializable identifier.

In an ultimatum experiment, each pair normally has its own scope. In a message-board study, the entire group may share one scope. Events in different scopes are isolated even when they are stored in the same file. Events within a scope may target several primitives—for example, a board, an agenda, and a ballot box.

2.1 Creating a shared state

Begin with a store, a scope, and one or more named primitives:

```
from edsl.sharedstate import FileStateStore, SharedLog, SharedState

state = SharedState(
    scope="seminar-1",
    store=FileStateStore("seminar-state.jsonl"),
```

```

    comments=SharedLog(),
)

```

This creates a *configuration*. It does not write an event yet. The keyword `comments` is not a special `SharedState` field. It is a researcher-chosen name for one data structure, here a `SharedLog`. The name becomes both the primitive’s persistent event target and the Python attribute `state.comments`. Primitive names must be unique and cannot be `scope`, `store`, or `primitives`.

A state can contain several named primitives, including different kinds:

```

from edsl.sharedstate import (
    FileStateStore,
    SharedAgenda,
    SharedMessageBoard,
    SharedState,
    SharedVotingGame,
)

state = SharedState(
    scope="committee-1",
    store=FileStateStore("committee-state.jsonl"),
    discussion=SharedMessageBoard(),
    agenda=SharedAgenda(),
    election=SharedVotingGame(
        candidates=["fund", "revise", "decline"],
        voter_count=5,
    ),
)

```

The names `discussion`, `agenda`, and `election` are chosen by the author. The objects assigned to them define the data shape, legal operations, visibility, and finalization behavior. Questions read their views as `shared_state.discussion`, `shared_state.agenda`, and `shared_state.election`; operations are written through `state.discussion.add(...)`, `state.agenda.propose(...)`, and the voting primitive’s methods.

Conceptually, an event is addressed by two coordinates:

Coordinate	Example	Meaning
Scope	pair-17	Which independent shared world?
Target	game	Which primitive inside that world?

Thus a single store can contain `pair-17/game`, `pair-18/game`, and `committee-1/discussion` without combining their state.

`FileStateStore` reconstructs the current state by replaying JSONL events. The log is therefore the research record; the current snapshot is a derived view. Deleting or reusing an old log changes the experiment, so production code should choose explicitly among create-new, resume, and read-only behavior. The current constructor does not yet make those modes sufficiently obvious.

The file store is the current local implementation, not an intended limitation of the model. Shared state depends on a store interface with operations such as `apply`, `read`, `close`, `scopes`, and `history`. A

future implementation could use a remote service, database, or transactional server while preserving the same primitive and survey APIs. That becomes important for remote inference: workers on different machines cannot coordinate through a file that exists only on the submitting computer.

```
# Current local development store
store = FileStateStore("committee-state.jsonl")

# Illustrative future store; not implemented in this branch
store = RemoteStateStore(connection="https://state.example.edu/studies/42")
```

A remote store would need atomic writes, authentication, authorization, idempotent retries, durable version ordering, and a way to bind a remote run to the exact primitive configuration. Merely uploading the local JSONL file after the run would not support interactions during the run.

Design pressure — opening a store. `FileStateStore(path)` silently uses an existing path or starts an empty one. A safer public interface would provide `create`, `resume`, and `replay`, with configuration fingerprints and a clear error when a new run points at an old log.

2.2 The primitive lifecycle

Every primitive implements the same conceptual contract:

1. `initial()` creates its empty state.
2. A public method such as `append()`, `offer()`, or `claim_before()` creates a survey step describing a legal operation.
3. `apply()` validates that operation and produces the next state.
4. `view()` returns what a particular reader is allowed to see.
5. `at_close()` optionally computes a final outcome when the scope closes.

Researchers normally call the public operation methods; they do not call `apply()` themselves. This distinction matters. A survey contains a serializable description of the intended write, while the store performs the write at interview time.

The smallest complete example records one statement per agent:

```
from edsl import Agent, AgentList, Model, QuestionFreeText, Survey
from edsl.sharedstate import FileStateStore, SharedLog, SharedState

agents = AgentList([
    Agent(name="Amina", traits={"position": "support"}),
    Agent(name="Boris", traits={"position": "oppose"}),
])

state = SharedState(
    scope="seminar-1",
    store=FileStateStore("seminar-state.jsonl"),
    comments=SharedLog(),
)

comment = QuestionFreeText(
    question_name="comment",
```



```

    question_text=(
        "Earlier comments: {{ shared_state.comments.entries }}\n"
        "Your initial position is {{ agent.position }}. Add one argument."
    ),
)

```

Next, place the question and its state-writing step in execution order:

```

survey = Survey([
    comment,
    state.comments.append(
        author="{{ agent.name }}",
        text=comment,
    ),
])

```

Finally, run the survey serially so each later interview sees earlier writes:

```

results = (
    survey
    .by(agents)
    .by(Model("gemini-2.5-flash"))
    .run(
        interview_schedule="serial",
        disable_remote_inference=True,
        cache=False,
    )
)

```

The argument `text=comment` deserves special attention. At this point, `comment` is a `QuestionFreeText` object—not the agent’s answer, because no interview has run yet. `SharedLog.append()` recognizes an object with a `question_name` and converts it to an answer reference equivalent to:

```
AnswerRef("comment")
```

The reference is a placeholder for the future answer. It is resolved only when the append step executes during an interview:

Time	Meaning of <code>comment</code> / <code>text</code>
Survey construction	The question object is converted to <code>AnswerRef("comment")</code>
Question execution	The model produces the actual <code>comment</code> answer
Append-step execution	The reference resolves to that answer and the value is written

By contrast, `author="{{ agent.name }}"` is a runtime reference to an agent trait. A literal such as `round=1` would be written unchanged.

This shorthand is compact, but it is also somewhat magical: passing a question object sometimes means “use its eventual answer.” The prospective syntax later in this manual uses an explicit `comment.answer`

reference, which is easier to read and permits creation-time type checking. The current implementation's explicit spelling is `AnswerRef(comment.question_name)`.

There are two distinct data paths in this example:

- `comment` remains an ordinary EDSL answer in `results`.
- `state.comments.append(...)` copies that answer into shared state as a typed event, together with the agent name.

Placing the write step after `comment` is meaningful: the answer must exist before the step can resolve it. With serial scheduling, the second prompt can read the first event. With concurrent scheduling, what each interview observes depends on its read time; that should be declared as part of the design rather than left to response timing.

2.3 How a question reads state

When a survey contains a shared-state step, EDSL attaches the corresponding `SharedState` configuration to that survey. During prompt rendering, `shared_state` is the reader-specific snapshot. A primitive's named view is available beneath it:

```
{{ shared_state.comments.entries }}
{{ shared_state.comments.count }}
{{ shared_state.comments.tail }}
```

These are *views*, not arbitrary access to the Python object. `SharedLog`, for example, can filter entries for the current viewer. A private-information study must test the view under every role; merely omitting a secret from one prompt is not an access-control policy.

A question can read ordinary context and shared state together:

```
question_text=(
    "You are {{ agent.name }} and your role is {{ agent.role }}.\n"
    "Public history: {{ shared_state.actions.entries }}\n"
    "Choose your next action."
)
```

Design pressure — declared reads. Current Jinja text is convenient, but a misspelled path may fail only when an interview renders. The preferred `Stem` syntax introduced later makes reads explicit, checks them before execution, and carries enough metadata for a remote runner to enforce visibility.

2.4 How an answer becomes an operation

Primitive methods return survey steps. They may receive constants, agent or run references, and question-answer references:

```
state.actions.append(
    actor="{{ agent.name }}",          # resolved from the current agent
    round="{{ run.round }}",          # resolved from run context
    choice=choice,                     # resolved from this question's answer
)
```

Internally, the step records the primitive target, operation name, arguments, and interview identity. The step is serializable with the survey, which is a requirement for remote execution. At runtime it becomes one `StateEvent` with a monotonically increasing version within its scope.

Some operations must occur before a question. A work pool, for example, claims an item atomically and then exposes that item to the review prompt:

```
review = QuestionFreeText(
    question_name="review",
    question_text=(
        "Your assigned item is {{ shared_state.work.claimed }}. "
        "Return your review."
    ),
)

survey = Survey([
    state.work.claim_before(review),
    review,
    state.work.complete(review),
])
```

This ordering is different from an ordinary write-after-answer step. The claim must be idempotent so a retry does not consume a second item.

2.5 Scopes: one world or many

It is useful to distinguish a **scope identifier** from a **scope selector**. An identifier is the concrete string stored on every event, such as "pair-1". `SharedState.scope` is the selector used during interviews. It may be a literal identifier or the supported agent-trait template.

A literal scope gives every interview the same shared world:

```
state = SharedState("committee-1", store, board=SharedMessageBoard())
```

A scope can instead be selected from an agent trait:

```
state = SharedState(
    "{{ agent.pair_id }}",
    store,
    game=ConfiguredSharedGame(...),
)
```

If agents have `pair_id` values `pair-1`, `pair-1`, `pair-2`, and `pair-2`, the single `SharedState` configuration addresses two realized scopes and therefore creates two isolated games in the same event file. In that sense, yes: one configured `SharedState` can manage multiple scopes. Its `scope` attribute is the selection rule, while `state.scopes()` returns the concrete scope strings actually present in the store. `resolve_scope()` fails if an interview lacks the named trait.

Code outside an interview can also select a concrete scope explicitly:

```
state.read(scope="pair-1")
state.history(scope="pair-2", target="game")
state.close(scope="pair-1")
```

The present template resolver intentionally supports only a whole-string agent trait such as "`{{ agent.pair_id }}`"; it is not a general Jinja expression. This restriction makes scope identity inspectable and serializable, although a future assignment object would be cleaner than placing operational grouping metadata in agent traits.

Scope traits are operational assignments, not persona characteristics. They should be generated by the experimental design and saved separately even though the current integration places them in `Agent.traits`.

2.6 Reading, auditing, and closing

The public read API distinguishes a current snapshot from the event history:

```
snapshot = state.read(scope="pair-1")
snapshot.state      # reader-facing primitive views
snapshot.version    # number of events replayed in this scope
snapshot.closed     # whether further writes are prohibited

events = state.history(scope="pair-1", target="game")
records = state.records(target="game")
markdown = state.render_markdown(scope="pair-1")
```

Historical reads support causal inspection:

```
before_response = state.read(scope="pair-1", at_version=1)
```

Closing a scope prevents further writes and invokes each primitive's `at_close()` hook. For a sealed auction or simultaneous game, closing is the natural point to reveal actions and calculate the outcome:

```
state.close(scope="auction-1")
final = state.read(scope="auction-1")
```

Do not call `close()` merely because a Python loop ended; close only when the mechanism's scientific completion condition holds. Group stop conditions and interview schedules automate this in several examples.

2.7 Choosing a primitive

The primitive should express the smallest reusable state transition that still preserves the research design.

Need	Primitive family	Characteristic operation
Public or restricted observations	<code>SharedLog</code> , <code>SharedMessageBoard</code>	Append a structured entry
Exclusive tasks	<code>SharedWorkPool</code>	Claim before a question, then complete
Counts and quotas	<code>SharedCounterMap</code> , <code>SharedBudgetPool</code>	Increment or allocate under a constraint
Documents and agendas	<code>SharedDocument</code> , <code>SharedAgenda</code>	Propose, amend, prioritize, or adopt

Need	Primitive family	Characteristic operation
Matching and coalitions	<code>SharedMatchPool</code> , <code>SharedDeferredAcceptance</code> , <code>SharedCoalitionPool</code>	Submit preferences or consent; finalize globally
Forecasting and markets	<code>SharedForecast</code> , <code>SharedDelphiPanel</code> , <code>SharedBinaryMarket</code>	Record beliefs, revise, trade, or settle
Economic mechanisms	<code>ConfiguredSharedGame</code> and game primitives	Take a role-authorized action; settle on completion

The long catalog of specialized game classes is experimental evidence, not necessarily the desired public API. If two mechanisms differ only in constants, field schemas, legal actions, and settlement expressions, they should probably be configurations of one generic mechanism. If they differ in timing, visibility, or allocation semantics, forcing them into one class may conceal an important treatment.

2.8 What is saved for remote execution

`SharedState.to_dict()` serializes the scope, store configuration, and primitive configurations. Survey serialization also saves anchored before-question and after-answer steps. A remote worker can therefore reconstruct the state machine without importing the local example file.

That promise has boundaries. Custom Python callables, undeclared dynamic Jinja lookups, local filesystem assumptions, and model-completion timing do not become reproducible merely because the outer survey serializes. The safest mechanism is composed from declarative primitives and expression objects whose complete configuration appears in the run artifact.

Design pressure — the durable artifact. The JSONL log alone does not say which agents, questions, model, schedule, seed, or primitive configuration produced it. A publishable run should package all of those inputs with the events and ordinary `EDSL Results`.

Chapter 3

From primitives to a complete simulation

An EDSL simulation has six separable parts.

Part	Scientific question	Proposed object
Personas	Who are the simulated actors?	<code>AgentList</code>
Assignments	Who is paired, grouped, seated, or assigned a role?	<code>Participants</code> , <code>Pairing</code>
Mechanism	What state exists and which transitions are legal?	<code>ConfiguredSharedGame</code> , shared primitives
Protocol	Which actor answers which question at each stage?	<code>Protocol</code>
Schedule	What is simultaneous, sequential, repeated, or revealed?	<code>Schedule</code>
Run	Which model, seed, cache, concurrency, and storage policy apply?	<code>Simulation</code> , <code>RunConfig</code>

Keeping these parts separate is not merely stylistic. Pair assignment should not silently become a personality trait. A sealed action should not become visible because a prompt happens to mention the full state. Repetition should not be confused with independent statistical replications.

3.1 A minimal simulation

The smallest proposed pattern is:

```
from edsl import AgentList, Model, QuestionMultipleChoice, Survey
from edsl.simulations import Simulation, RunConfig
from edsl.sharedstate import SharedState

agents = AgentList.from_records([
    {"name": "Avery", "strategy": "cooperate unless exploited"},
```

```

    {"name": "Blake", "strategy": "maximize own payoff"},
])

game = games.prisoners_dilemma(
    payoffs={
        ("cooperate", "cooperate"): (3, 3),
        ("cooperate", "defect"): (0, 5),
        ("defect", "cooperate"): (5, 0),
        ("defect", "defect"): (1, 1),
    }
)

participants = Pairing.adjacent(agents).seats("row", "column")

choice = QuestionMultipleChoice(
    question_name="action",
    question_text=Stem(
        "You are {name}. Your strategy is {strategy}. Choose an action.",
        name=current.agent.name,
        strategy=current.agent.strategy,
    ),
    question_options=["cooperate", "defect"],
)

protocol = Protocol.all(Survey([
    choice,
    game.bind("choose", choice.answer),
]))

simulation = Simulation(
    state=SharedState.create("pd.jsonl", game=game),
    participants=participants,
    protocol=protocol,
    schedule=Schedule.simultaneous(
        by=participants.group,
        reveal=AfterRound,
        finalize_when=game.complete,
    ),
)

run = simulation.run(
    model=Model("gemini-2.5-flash"),
    config=RunConfig.local(seed=42, max_concurrency=10),
)

```

The result is a run artifact rather than only a table of model responses:

```

run.results          # ordinary EDSL Results
run.state            # replayable SharedState

```

```
run.events           # typed action history
run.records(target="game") # one authoritative outcome per pair
run.agents           # original personas
run.assignments      # group, role, and seat assignments
```

Design pressure — artifact boundary. Current examples often return only `SharedState` and discard `Results`. A durable simulation needs both, plus the original agents, assignments, model configuration, schedule, and random seed. The natural serialization boundary may be a `SimulationRun` package rather than a larger `SharedState` object.

Chapter 4

Authoring personas and assignments

4.1 Behavioral traits

Traits should describe the actor, not the execution plan. For an ultimatum-game study, two continuous traits might be:

```
personas = AgentList.random(  
    50,  
    name="Person {i:02d}",  
    traits={  
        "generosity": Uniform(-1, 1),  
        "inequity_aversion": Uniform(-1, 1),  
    },  
    seed=20260828,  
    traits_presentation_template=Stem(  
        "Your generosity is {g}; -1 is strongly self-interested and +1 is "  
        "strongly generous. Your inequity aversion is {i}; -1 means tolerating "  
        "highly unequal outcomes and +1 means paying a personal cost to resist them.",  
        g=current.agent.generosity,  
        i=current.agent.inequity_aversion,  
    ),  
)
```

The seed and distribution belong in the saved run manifest. A paper should report the distribution, seed policy, number of replications, and whether trait values were shown numerically or translated into prose.

4.2 Pairing, grouping, and roles

Assignments are experimental design:

```
pairs = Pairing.random(personas, seed=20260828).roles(  
    proposer=0,  
    responder=1,  
)
```

They should be queryable:

```
pairs.groups  
pairs.members("pair-12")  
pairs.role_of(personas[7])
```

Creation should fail when an assignment violates the mechanism:

```
AssignmentError: pair-12 has two proposers and no responder;  
ultimatum requires exactly one participant in each role.
```

Design pressure — identities across phases. Some organizational examples create new agents from earlier model output, such as one theme editor per synthesized theme. Static **Participants** is insufficient. The API needs derived assignments whose provenance is saved and whose identities remain stable when a workflow resumes.

Chapter 5

Checked question text and state reads

Raw Jinja is concise but easy to misspell:

```
"You are {{ agent.name }}. Current market: {{ shared_state.market }}"
```

The proposed `Stem` compiles checked references to the same runtime template:

```
QuestionFreeText(  
    question_name="rationale",  
    question_text=Stem(  
        "You are {name}. Current market: {market}",  
        name=current.agent.name,  
        market=state.market.as_table(),  
    ),  
)
```

Checks occur at three stages:

1. `Stem` construction checks placeholders and rendering capabilities.
2. `Survey` construction checks question membership and ordering.
3. Run preflight checks agent, scenario, and run-context fields.

The compiled stem declares which state it reads. A remote runner must reconstruct this read plan from the compiled Jinja or receive equivalent sidecar metadata. Rendering still works without the metadata; precise visibility and causal audit do not.

Design pressure — dynamic reads. A template such as `shared_state[current.agent.private_key]` cannot always be resolved statically. The system needs an explicit policy: reject dynamic reads, authorize a declared set, or conservatively read all visible state. Silent fallback is unsuitable when state contains private information.

Chapter 6

Configuring mechanisms

A mechanism owns typed state, legal actions, prerequisites, terminal conditions, visibility, and settlement. The ultimatum game should be a recipe, not a custom state-transition class:

```
ultimatum = ConfiguredSharedGame(  
    constants={"stake": 100},  
    fields={  
        "offer": Field.number(minimum=0, maximum=100),  
        "decision": Field.choice("accept", "reject"),  
    },  
    actions={  
        "offer": Action(  
            actor="proposer",  
            writes="offer",  
            write_once=True,  
        ),  
        "respond": Action(  
            actor="responder",  
            writes="decision",  
            requires=("offer",),  
            write_once=True,  
        ),  
    },  
    terminal=WhenSet("decision"),  
    settlement=Settlement.when(  
        StateValue("decision") == "accept",  
        payoffs={  
            "proposer": StateValue("stake") - StateValue("offer"),  
            "responder": StateValue("offer"),  
        },  
        otherwise=0,  
    ),  
)
```

Question references and settlement expressions are deliberately different: `offer.answer` refers to an interview response, whereas `StateValue("offer")` refers to persisted mechanism state.

Creation-time validation should check:

- every action writes a declared field;
- question answer types are compatible with target fields;
- prerequisites name declared fields;
- payoff roles exist in participant assignments;
- payoff matrices cover every action profile;
- terminal predicates refer to reachable state;
- write-once fields have at most one authorized writer per scope.

Design pressure — Python versus data. Pure configuration is serializable and inspectable, but unusual mechanisms eventually need custom logic. Python callables are expressive but difficult to serialize for remote execution. The API needs a small serializable expression vocabulary and a clearly marked local-only callable escape hatch.

Chapter 7

Protocols and role-specific questions

Role-specific protocols are preferable to one survey plus skip rules:

```
protocol = Protocol({
    "proposer": Survey([
        offer,
        ultimatum.bind("offer", offer.answer),
    ]),
    "responder": Survey([
        decision,
        ultimatum.bind("respond", decision.answer),
    ]),
})
```

Protocol construction should verify that each required role has exactly one path, action bindings follow their source questions, and no question reads state that is unavailable at that stage.

For a work queue, a before-question action remains useful:

```
protocol = Protocol.all(Survey([
    state.work.claim_before(review, claimant=current.agent.name),
    review,
    state.work.complete(review.answer, claimant=current.agent.name),
]))
```

Design pressure — no available work. If concurrent workers outnumber tasks, `claim_before` may return no assignment. The protocol needs an explicit no-work path rather than asking a model to review `None`.

Chapter 8

Scheduling and visibility

Scheduling is part of the experimental treatment.

8.1 Simultaneous action

```
Schedule.rounds(  
    count=3,  
    within_round=Concurrent,  
    visibility=SnapshotAtRoundStart,  
    reveal=AfterRound,  
)
```

Every actor reads the same pre-round watermark. Writes may be persisted as they finish, but current-round actions remain hidden until the round closes.

8.2 Sequential roles within concurrent groups

```
Schedule.grouped_round_robin(  
    group_by=participants.group,  
    order_by=participants.turn,  
    concurrent_groups=10,  
    finalize_when=game.terminal,  
)
```

Each pair proceeds proposer then responder, while different pairs run in parallel.

8.3 Repeated interaction

```
Schedule.repeated_game(  
    rounds=4,  
    group_by=participants.group,  
    within_round=Concurrent,  
    reveal_between_rounds=True,  
)
```

Independent statistical replications should use `replications=`, not the same parameter as repeated strategic rounds.

Design pressure — rotating order. Several market and deliberation examples rotate first or last movers to reduce order effects. Rotation is not merely a scheduler optimization; it must be recorded as treatment assignment so results can be analyzed by position.

Chapter 9

Running, resuming, and inspecting

Use an explicit run configuration:

```
config = RunConfig.local(  
    seed=42,  
    cache=False,  
    stop_on_exceptions=True,  
    max_concurrency=10,  
)
```

Storage modes should be explicit:

```
SharedState.create("study.jsonl", game=game)    # fail if present  
SharedState.resume("study.jsonl", game=game)    # verify configuration fingerprint  
SharedState.replay("study.jsonl", game=game)    # read-only
```

Inspection should not require parsing JSONL:

```
run.state.scopes()  
run.state.history(scope="pair-1", target="game")  
run.state.snapshots()  
run.state.records(target="game")
```

Design pressure — resume semantics. Counting persisted rows is not enough when one interview writes two primitives and crashes between them. A resumable phase needs an atomic completion marker or transaction boundary that says the participant completed the phase, not merely that one expected record exists.

Chapter 10

Economic games

This chapter maps the economic-game examples to the proposed syntax. Each game should save assignments and state events so that an analyst can distinguish behavioral outcomes from completion order or scheduling effects.

10.1 Matrix games: prisoner's dilemma and stag hunt

Repository examples: `economic_games_matrix.py` and `economic_game_repeated_prisoners_dilemma.py`.

A one-shot matrix game has two sealed actions and an exhaustive payoff table:

```
game = games.matrix(
    actions=("cooperate", "defect"),
    payoffs={
        ("cooperate", "cooperate"): (3, 3),
        ("cooperate", "defect"): (0, 5),
        ("defect", "cooperate"): (5, 0),
        ("defect", "defect"): (1, 1),
    },
)

participants = Pairing.adjacent(agents).seats("row", "column")
protocol = Protocol.all(Survey([action, game.bind("choose", action.answer)]))
schedule = Schedule.simultaneous(
    by=participants.group,
    reveal=AfterRound,
    finalize_when=game.complete,
)
```

The same recipe represents stag hunt by changing actions and payoffs. This shows that mechanism structure and substantive labels should be independent.

For repeated play:

```
schedule = Schedule.repeated_game(
    rounds=3,
    group_by=participants.group,
```

```

        within_round=Concurrent,
        reveal_between_rounds=True,
    )

```

The prompt reads completed history but not current-round actions. The run should record a read watermark for every decision.

Design pressure — payoff indexing. Seat identity must be declared. Using action completion order to index asymmetric payoffs would make concurrency alter the game. Matrix construction should reject incomplete payoff tables and assignments with missing or duplicate seats.

10.2 Ultimatum, dictator, and trust games

Repository examples: `economic_game_ultimatum.py` and `economic_games_transfer.py`.

These are variations on transfers:

```

ultimatum = games.ultimatum(stake=100)
dictator = games.dictator(endowment=100)
trust = games.trust(endowment=100, multiplier=3)

```

Ultimatum uses ordered role protocols:

```

protocol = Protocol({
    "proposer": Survey([offer, ultimatum.bind("offer", offer.answer)]),
    "responder": Survey([decision, ultimatum.bind("respond", decision.answer)]),
})

```

Dictator has one action and immediate settlement. Trust has a sender transfer, deterministic multiplication, and a receiver return. The mechanism should expose the receiver's available amount as a checked state read.

These games show why roles and pair assignments should not be agent traits. The same persona may be randomized into either role across replications.

Design pressure — treatment randomization. Researchers often want each persona to play both roles or several partners without contaminating memory. The run object needs explicit replication, rematching, and interview-memory policies rather than relying on repeated `.by()` products.

10.3 Nash demand and the 11–20 request game

Repository examples: `economic_game_nash_demand.py` and `economic_game_11_20_money_request.py`.

Both are simultaneous numeric-choice mechanisms:

```

game = games.nash_demand(pie=100)
demand = QuestionNumerical(..., min_value=0, max_value=100)

simulation = Simulation(
    ...,
    protocol=Protocol.all(Survey([

```

```

        demand,
        game.bind("demand", demand.answer),
    ])),
    schedule=Schedule.simultaneous(
        by=participants.group,
        reveal=AfterRound,
    ),
)

```

The 11–20 game changes the allowed interval and settlement rule: each player receives the requested amount, and a player requesting exactly one less than the opponent receives a bonus.

Design pressure — integer versus numeric. `QuestionNumerical` permits numeric values, but these mechanisms require integers. `Field.integer()` and a compatible integer question or coercion policy are needed. Silent rounding would change the game.

10.4 Public goods, punishment, and common-pool extraction

Repository examples: `shared_state_public_goods.py`, `economic_game_public_goods_punishment.py`, and the common-pool section of `economic_games_group.py`.

A public-goods mechanism should own contributions and settlement:

```

game = games.public_goods(
    players=4,
    rounds=4,
    endowment=20,
    multiplier=1.6,
)

protocol = Protocol.all(Survey([
    contribution,
    rationale,
    game.bind(
        "contribute",
        amount=contribution.answer,
        rationale=rationale.answer,
    ),
]))

```

The current examples use a generic log and recompute payoff tables afterward. Moving settlement into the mechanism creates one authoritative outcome and allows validation that every player contributed exactly once per round.

Peer punishment is a second sealed stage:

```

workflow = GameWorkflow([
    Stage("contribution", protocol=contribution_protocol, reveal=AfterStage),
    Stage("punishment", protocol=punishment_protocol, reveal=AfterStage),
])

```

The punishment matrix should be checked against participant identities; missing or unknown targets

should fail before execution.

Common-pool extraction uses a shared stock and congestion or over-extraction settlement. Its state transition must define whether simultaneous requests are scaled, partially fulfilled, or allowed to exhaust the stock in completion order.

Design pressure — stage-generated questions. Punishment options depend on realized participant names and contribution results. Protocols must support questions built from assignment metadata and completed stage output without embedding an ad hoc formatted slate into static text.

10.5 Beauty contest and market entry

Repository examples: the beauty-contest section of `economic_games_group.py` and `economic_game_market_entry.py`.

These are anonymous group mechanisms:

```
beauty = games.beauty_contest(
    players=6,
    range=(0, 100),
    target_fraction=2 / 3,
)

entry = games.market_entry(
    firms=6,
    capacity=2,
    entry_value=10,
    congestion_cost=3,
    outside_payoff=2,
)
```

All choices are sealed and settled after group completion. The mechanism should validate expected group size against assignments before any call.

Design pressure — missing actors. If one interview fails, should the group remain incomplete, settle using actual participants, or impute an action? That is a scientific decision and must be an explicit completion policy.

10.6 Auctions and continuous double auctions

Repository examples: `economic_games_auction_comparison.py` and `economic_game_continuous_double_auction.py`.

Sealed auctions differ mainly in settlement:

```
auction = games.sealed_auction(
    item="research grant",
    mechanism="second_price",
    bidders=5,
)
```

Private valuations belong to personas; bidder identity and seat belong to assignments. Bids remain hidden until close.

A continuous double auction is different: the order book is live, trades happen during the round, and order priority matters.

```
market = games.double_auction(
    buyers=buyer_assignments,
    sellers=seller_assignments,
    price_tick=1,
    priority="price_time",
)

schedule = Schedule.rounds(
    count=3,
    within_round=Serial,
    order=RotateEachRound,
    visibility=Live,
)
```

Design pressure — nondeterministic timing. A live market cannot use model response completion time as an economically meaningful priority unless that is the intended treatment. The scheduler needs declared logical order or a reproducible event-time policy.

10.7 Signaling, adverse selection, cheap talk, and moral hazard

Repository examples: `economic_game_signaling.py`, `economic_game_adverse_selection.py`, `economic_game_cheap_talk.py`, and `economic_game_moral_hazard.py`.

These games combine roles, private information, and sequential observation.

```
protocol = Protocol({
    "worker": Survey([
        education,
        game.bind("signal", education.answer),
    ]),
    "employer": Survey([
        hiring,
        game.bind("decide", hiring.answer),
    ]),
})
```

The employer may read education but not worker productivity. In adverse selection, the seller sees the buyer's price and its own private cost. In cheap talk, the receiver sees a costless message but not the sender's private state. In moral hazard, the worker sees a contract and privately chooses effort while output remains stochastic.

Visibility should therefore be declared on fields:

```
Field.number("productivity", visible_to="worker")
Field.number("education", visible_to=PublicAfter("signal"))
Field.choice("effort", visible_to="worker")
```

Design pressure — secrets in traits. Omitting private traits from a prompt is not a sufficient access-control model if the full agent or state object is available to template

evaluation. Read authorization must apply during rendering and be tested under remote execution.

10.8 Centipede and bilateral negotiation

Repository examples: `economic_game_centipede.py` and `shared_state_bilateral_negotiation.py`.

These games have variable duration:

```
schedule = Schedule.turns(  
    group_by=participants.group,  
    order_by=participants.turn,  
    max_rounds=5,  
    stop_when=game.terminal,  
)
```

The centipede game alternates take/pass actions over a finite payoff path. Negotiation records offers, acceptances, rejections, walkaways, and messages. The action schema should make amount conditional:

```
NegotiationAction.tagged_union(  
    offer={"amount": Field.number(minimum=0)},  
    accept={"amount": Field.number(minimum=0)},  
    reject={},  
    walk_away={},  
)
```

This is preferable to instructing actors to enter 0 for actions that do not have a price.

Design pressure — conditional questions. Tagged actions want dependent forms: ask for an amount only after `offer`, not after `reject`. Protocols need typed branching that remains serializable and validates referenced answers.

10.9 Voting and strategic voting

Repository examples: `economic_game_voting_rules.py` and `economic_game_strategic_voting.py`.

Voting mechanisms separate ballot collection from tabulation:

```
election = games.election(  
    candidates=("A", "B", "C"),  
    rule="borda",  
    seats=1,  
)  
  
protocol = Protocol.all(Survey([  
    ballot,  
    election.bind("vote", ballot.answer),  
]))
```

The same ballots can be replayed under plurality, Borda, instant runoff, or Condorcet rules. A strategic-voting study instead varies information and asks whether the submitted ballot differs from the sincere ranking.

Design pressure — ballot validity. Ranking questions must enforce complete permutations when the rule requires them. Ties, exhausted ballots, and deterministic tie-breaking must be explicit and saved in the mechanism configuration.

10.10 Information cascades

Repository example: `economic_game_information_cascade.py`.

Actors receive private signals and act sequentially after observing previous public choices:

```
schedule = Schedule.sequence(
    order_by=participants.position,
    visibility=Live,
)

protocol = Protocol.all(Survey([
    choice,
    state.choices.bind(
        "append",
        actor=current.agent.name,
        signal=current.agent.private_signal,
        choice=choice.answer,
    ),
]))
```

The public view must exclude private signals even though they may be recorded in an access-controlled audit trail.

Design pressure — recording versus revealing. A field can be persisted for research audit without being prompt-visible. The state model needs separate write authorization, storage visibility, and respondent-view policies.

Chapter 11

Deliberation and organizational workflows

These simulations are less naturally described as games. They combine evidence, discussion, assignments, synthesis, and deterministic decision rules. Their central design problem is reliable multi-phase execution and resumption.

11.1 Message boards and rumor diffusion

Repository examples: `shared_state_family_message_board.py` and `shared_state_rumor_diffusion.py`.

A message board is an append-only public communication primitive:

```
board = SharedMessageBoard(
    schema=Message(
        sender=Field.participant(),
        body=Field.string(max_words=100),
        reply_to=Field.optional(Field.message_id()),
    )
)

protocol = Protocol.all(Survey([
    message,
    board.bind(
        "post",
        sender=current.agent.name,
        body=message.answer,
    ),
]))
```

The family-board example runs ordered turns across repeated rounds. Rumor diffusion varies who hears which claims and records how content changes. A researcher should save the exposure graph, not infer it from message order.

Design pressure — conversational memory. Prompt-visible board history and an interview’s internal conversational memory can both influence later text. The run manifest

must state whether each turn is a fresh interview, continues an earlier interview, or relies exclusively on public state.

11.2 Agenda setting and legislative amendments

Repository examples: `shared_state_meeting_agenda.py` and `shared_state_legislative_amendments.py`.

Agenda setting separates proposal from voting:

```
workflow = GameWorkflow([
    Stage(
        "propose",
        protocol=proposal_protocol,
        completion=OnePerParticipant,
    ),
    Stage(
        "vote",
        protocol=vote_protocol,
        after="propose",
        completion=OnePerParticipant,
    ),
])
```

Legislative amendment uses a shared document whose revisions are ordered and auditable:

```
document = SharedDocument(
    title="Draft bill",
    initial_text=INITIAL_TEXT,
    revisions=AppendOnly,
)
```

Questions should read a versioned document reference. A revision event should record its base version so concurrent edits cannot silently overwrite one another.

Design pressure — textual merge semantics. A generic `revise` operation must define whether the answer is a replacement document, patch, amendment, or suggestion. Optimistic concurrency errors need an academically meaningful resolution policy rather than last-writer-wins.

11.3 Work queues and incident response

Repository examples: `shared_state_live_review_queue.py` and `shared_state_incident_response.py`.

Workers atomically claim tasks immediately before their prompt is rendered:

```
review = QuestionDict(...)

protocol = Protocol.all(Survey([
    state.work.claim_before(review, claimant=current.agent.name),
    review,
    state.work.complete(
        review.answer,
```

```

        claimant=current.agent.name,
    ),
]))

```

Incident response adds a second phase in which a commander reads all completed investigations and writes a resolution.

```

workflow = SimulationWorkflow([
    Phase(
        "investigate",
        participants=responders,
        protocol=investigation_protocol,
        complete_when=state.work.all_completed,
    ),
    Phase(
        "command",
        participants=commander,
        protocol=resolution_protocol,
        after="investigate",
    ),
])

```

Design pressure — leases and recovery. An atomic claim prevents duplicate assignment but can strand work when an interview fails after claiming. A production work pool needs leases, heartbeat or expiry, abandonment events, retries, and an auditable requeue policy.

11.4 Resource allocation and disaster response

Repository examples: `shared_state_budget_allocation.py` and `shared_state_disaster_response.py`.

Budget allocation is a repeated claim against a finite resource:

```

budget = SharedBudgetPool(
    total=100,
    projects=PROJECTS,
    allow_partial=True,
)

schedule = Schedule.rounds(
    count=3,
    within_round=Serial,
    stop_when=budget.exhausted,
)

```

Disaster response introduces incidents, heterogeneous resources, and staged waves. The primitive should validate resource capacity and prevent two actors from deploying the same exclusive resource.

```

board = SharedResourceBoard(
    incidents=INCIDENTS,
    resources=RESOURCES,
    allocation=Exclusive,
)

```

)

Design pressure — eligibility versus atomicity. Atomic allocation prevents duplicate claims but does not decide who is qualified. Eligibility, priority, and tie-breaking should be declared independently from the atomic write.

11.5 Matching markets

Repository examples: `shared_state_matching_market.py`, `shared_state_congested_matching_market.py`, and `shared_state_peer_review_matching.py`.

Deferred acceptance combines submitted rankings, capacities, and priorities:

```
market = games.deferred_acceptance(
    options=PROGRAMS,
    capacities={"A": 2, "B": 1, "C": 1},
    priorities=PRIORITIES,
)

protocol = Protocol.all(Survey([
    ranking,
    market.bind("submit", ranking.answer),
]))
```

The congested-market example tests strategic rankings when participants know capacities and likely competition. Peer-review matching includes expertise, conflicts, and reviewer priority.

Design pressure — truthful and strategic fields. A participant may have a latent sincere ranking and submit a different strategic ranking. These must be separate variables with separate visibility. A matching primitive should not assume that an agent trait named `ranking` is the submitted ballot.

11.6 Coalition formation

Repository example: `shared_state_coalition_formation.py`.

Actors repeatedly request coalitions while observing current membership and recent requests:

```
coalitions = SharedCoalitionPool(
    participants=assignments,
    acceptance=UnanimousAmongMembers,
)

schedule = Schedule.rounds(
    count=2,
    within_round=Concurrent,
    visibility=SnapshotAtRoundStart,
)
```

The public view may include current coalition membership and recent requests, while private preference rankings remain visible only to their owner.

Design pressure — multi-party consent. Coalition requests are not ordinary writes: formation may require several independent acceptances, withdrawal, expiration, or overlapping offers. A generic action/state machine must support pending multi-actor transactions.

11.7 Forecast revision and Delphi panels

Repository examples: `shared_state_forecast_revision.py` and `shared_state_delphi_forecast.py`.

Forecast revision records probabilities and rationales over snapshot rounds:

```
forecasts = SharedForecast(
    outcome="Launch succeeds",
    probability_range=(0, 1),
    one_per_participant_per_round=True,
)

schedule = Schedule.rounds(
    count=3,
    within_round=Concurrent,
    visibility=SnapshotAtRoundStart,
)
```

A Delphi panel adds anonymous aggregation, facilitator feedback, and a convergence criterion:

```
panel = SharedDelphiPanel(
    panel_size=len(experts),
    range_threshold=10,
    median_shift_threshold=3,
    minimum_rounds=2,
)

workflow = RepeatUntil(
    expert_round,
    facilitator_summary,
    stop_when=panel.converged,
    max_rounds=5,
)
```

Design pressure — stopping bias. A data-dependent convergence rule changes the distribution of final estimates. The exact stopping criterion and maximum rounds must be stored as part of the research design, not only as runner code.

11.8 Prediction markets and private news

Repository examples: `shared_state_binary_prediction_market.py` and `shared_state_prediction_market.py`.

A binary LMSR market maintains prices and private portfolios:

```
market = games.binary_market(
    contract="Will the project ship by October 1?",
    liquidity=20,
    initial_cash=100,
```

```
)

schedule = Schedule.rounds(
    count=3,
    within_round=Serial,
    order=RotateEachRound,
    visibility=Live,
)
```

The private-news version releases one signal immediately before each trader’s action:

```
state.news.reveal_before(
    trade_question,
    recipient=current.agent.name,
    round=current.run.round,
)
```

The trader sees its own portfolio and signal history; other traders do not.

Design pressure — viewer-specific views. The same state version renders differently for each participant. Cache keys, remote payloads, and audit logs must include viewer identity and authorization context without persisting secrets into ordinary result columns.

11.9 Hiring committees

Repository examples: `shared_state_hiring_committee.py` and `shared_state_adversarial_hiring_committee.py`

The baseline workflow collects private rankings, public deliberation, and secret final ballots:

```
workflow = SimulationWorkflow([
    Phase("private_review", visibility=SealedUntilPhaseEnd),
    Phase("deliberation", visibility=Public),
    Phase("secret_ballot", visibility=SealedUntilWorkflowEnd),
])
```

The adversarial version adds conflicts and compares initial with final rankings. The deterministic analysis should use typed state queries:

```
initial = run.state.records(target="initial_rankings")
final = run.state.records(target="final_rankings")
movement = compare_rankings(initial, final, by="reviewer")
```

Design pressure — anonymity. “Anonymous” can mean hidden from other agents, hidden from the facilitator, or removed from the researcher’s event log. These are different guarantees and need distinct policies.

11.10 Strategic planning workshops

Repository examples: `shared_state_strategic_planning_workshop.py` and `shared_state_strategic_planning_adversarial.py`

These are the most demanding workflow examples. They contain proposal, challenge, revision, voting, tier assignment, portfolio construction, discussion, and final ballot phases.

```

workflow = SimulationWorkflow([
    Phase("propose", proposal_protocol, one_per=participants),
    Phase("challenge", challenge_protocol, after="propose"),
    Phase("revise", revision_protocol, after="challenge"),
    Phase("vote", voting_protocol, after="revise"),
    Phase("portfolio", portfolio_protocol, after="vote"),
])

```

The tiered version derives new options from earlier results. Each derived object should receive a stable ID and provenance record rather than being reconstructed from list order.

Design pressure — workflow scope. A general workflow engine can easily duplicate EDSL Jobs, Survey rules, or external orchestration systems. The smallest useful abstraction may be an experimental **Phase** layer that handles completion, resumption, and saved evidence while continuing to call ordinary EDSL surveys.

11.11 Customer-feedback synthesis

Repository example: `shared_state_customer_feedback_synthesis.py`.

Four analysts propose evidence-backed themes, a facilitator consolidates them, derived theme editors add detail, and reviewers prioritize the final slate.

```

workflow = SimulationWorkflow([
    Phase(
        "discover",
        participants=reviewers,
        protocol=discovery_protocol,
        rounds=2,
        completion=AtLeast(4, state.proposals),
    ),
    Phase(
        "synthesize",
        participants=facilitator,
        protocol=synthesis_protocol,
        after="discover",
        output=DerivedList("themes", length=5),
    ),
    Phase.for_each(
        "detail",
        items=outputs.themes,
        role="theme_editor",
        protocol=detail_protocol,
    ),
    Phase(
        "prioritize",
        participants=reviewers,
        protocol=priority_protocol,
        after="detail",
    ),
])

```

```
])
```

Question answers should validate cited comment IDs against the input dataset, not only validate that `evidence_ids` is a list.

Design pressure — semantic validation. EDSL can check output shape before persistence, but verifying that citations support a finding is a substantive evaluation. The API should distinguish schema validation, referential integrity, and model-based evidence evaluation.

11.12 Launch readiness review

Repository example: `shared_state_launch_readiness_review.py`.

Reviewers provide private assessments and blockers, owners propose mitigations, and reviewers issue final scores, conditions, and vetoes. A deterministic rule then chooses launch, limited launch, or delay.

```
workflow = SimulationWorkflow([
    Phase("initial", initial_protocol, one_per=reviewers),
    Phase("mitigate", mitigation_protocol, one_per=reviewers, after="initial"),
    Phase("final", final_protocol, one_per=reviewers, after="mitigate"),
])

decision = DecisionRule(
    veto=lambda review: review.veto_authority
    and (review.recommendation == "delay" or not review.approved),
    otherwise=launch_thresholds,
)
```

The decision rule should be a saved, testable object rather than an unrecorded post-processing function.

Design pressure — partial participant completion. The current workflow resumes by searching logs for reviewer names. A phase completion ledger should atomically record all writes from one participant, or record a failed partial attempt that can be repaired without duplicating earlier output.

11.13 General synthesis and tiered review

Several examples—customer feedback, hiring, planning, Delphi, and launch readiness—share a common pattern:

1. independent elicitation;
2. controlled reveal;
3. synthesis or challenge;
4. revision;
5. private ballot or deterministic settlement.

This suggests reusable phase recipes:

```
workflows.independent_then_deliberate(...)
workflows.delphi(...)
workflows.propose_challenge_revise_vote(...)
```


Recipes should expand to ordinary inspectable **Phase** objects. They must not hide visibility, completion rules, or model calls.

Chapter 12

Validation as part of the research interface

A simulation should have a `validate()` method that performs no model calls:

```
report = simulation.validate()
report.raise_for_errors()
print(report.warnings)
```

Validation should include the following layers.

12.1 Mechanism validation

- Field types, ranges, and choices are coherent.
- Every action writes declared fields.
- Prerequisite and terminal references exist.
- Settlement covers every terminal state.
- Payoff roles match required participant roles.
- Payoff matrices are exhaustive.

12.2 Assignment validation

- Required roles and seats are present exactly once per group.
- Group sizes satisfy mechanism requirements.
- Turn order is unique where required.
- Participant IDs are stable and unique.
- Private fields required by a role are available.

12.3 Protocol validation

- Every question reference points backward to a question in the same path.
- Every state rendering exists and is authorized for the role.
- Answer types are compatible with action inputs.
- Every required mechanism action is reachable.
- Skip or branch conditions refer to declared values.

12.4 Schedule validation

- Grouping and ordering references exist.
- Visibility is compatible with read declarations.
- A simultaneous stage does not expose current-stage writes.
- Stop and finalize predicates exist and return Boolean values.
- Maximum rounds prevent nonterminal infinite execution where appropriate.

12.5 Storage and resume validation

- Create mode refuses an existing path.
- Resume mode checks a configuration fingerprint.
- Replay mode refuses writes.
- Partial phase attempts are detectable.
- Closed scopes cannot be reopened accidentally.

12.6 Error messages

Errors are part of the API for both humans and coding agents:

```
ProtocolError: responder question 'decision' reads game.offer, but the
'offer' action is not guaranteed before the responder stage.
```

```
AssignmentError: group 'pair-7' has roles proposer, proposer; expected
exactly one proposer and one responder.
```

```
VisibilityError: question 'trade' reads news.signals, which is private to
role 'analyst'; current role is 'trader'. Did you mean news.your_signal?
```

```
ResumeError: log configuration fingerprint does not match this simulation;
stored stake=100, requested stake=50.
```

Chapter 13

Reproducibility and reporting

An academic report should record:

- EDSL version and commit;
- model name, service, and relevant model parameters;
- prompts after template compilation;
- personas, codebook, trait template, and generation seed;
- assignments, roles, seats, and ordering treatment;
- mechanism configuration and settlement rule;
- schedule, visibility, reveal, stopping, and failure policies;
- cache and retry policy;
- complete typed event history;
- state-read versions or watermarks for each action;
- deterministic analysis code;
- all exclusions, retries, and incomplete groups.

The run artifact should expose a manifest:

```
run.manifest()
run.audit.incomplete_interviews()
run.audit.state_reads(question="decision")
run.audit.retries()
run.replay()
```

Repeated runs should distinguish three sources of variation:

1. persona sampling;
2. assignment or ordering randomization;
3. model response stochasticity.

```
study = SimulationStudy(
    simulation,
    persona_seeds=range(10),
    assignment_seeds=range(10),
    model_replications=5,
)
```

These dimensions should not be collapsed into a single `n` parameter.

Chapter 14

What this manual reveals about the proposed design

Writing the examples in the preferred syntax exposes several unresolved core questions.

1. **Assignments need to be first-class.** Role, group, seat, and turn cannot remain ordinary traits if simulations are to be reusable and analyzable.
2. **A typed protocol is needed.** Role-specific paths and tagged conditional actions cannot be expressed cleanly with skip rules alone.
3. **Configured mechanisms need a disciplined escape hatch.** Pure Python callables conflict with remote serialization; a large expression DSL would be unpleasant. The supported boundary must be explicit.
4. **Visibility needs three dimensions.** Persisted, researcher-visible, and respondent-visible are different properties.
5. **Logical time must be distinct from completion time.** Live markets and sequential games cannot accidentally use network latency as treatment order.
6. **A phase ledger is more important than a general workflow language.** The largest examples primarily need atomic completion, resumption, and evidence.
7. **A simulation run is the durable object.** State alone cannot connect events to personas, assignments, prompts, model configuration, and results.
8. **Read metadata must survive remote execution.** Compiled templates can be re-analyzed remotely, but dynamic and private reads need explicit policy.
9. **Derived state belongs near the mechanism.** Payoffs and authoritative outcomes should not be independently reconstructed by each dashboard.
10. **Failure policy is scientific design.** Missing actors, retries, claims, partial writes, and convergence stopping cannot be hidden runner behavior.

A prudent implementation sequence is therefore:

1. checked references and `Stem`;
2. participant assignments separate from traits;
3. role-specific `Protocol`;
4. generic configured mechanisms with typed action binding;
5. visibility-aware schedules and read plans;
6. `SimulationRun` artifacts;
7. a small resumable `Phase` layer.

This sequence improves the small games before introducing machinery needed only by the largest workflows.

Chapter 15

Example coverage index

Repository example	Manual treatment
<code>economic_game_11_20_money_request.py</code>	Nash demand and 11–20 request
<code>economic_game_adverse_selection.py</code>	Signaling and private information
<code>economic_game_centipede.py</code>	Variable-duration sequential games
<code>economic_game_cheap_talk.py</code>	Signaling and private information
<code>economic_game_continuous_double_auction.py</code>	Auctions and live markets
<code>economic_game_information_cascade.py</code>	Information cascades
<code>economic_game_market_entry.py</code>	Anonymous group mechanisms
<code>economic_game_moral_hazard.py</code>	Signaling and private information
<code>economic_game_nash_demand.py</code>	Nash demand and 11–20 request
<code>economic_game_public_goods_punishment.py</code>	Public goods and punishment
<code>economic_game_repeated_prisoners_dilemma.py</code>	Repeated matrix games
<code>economic_game_signaling.py</code>	Signaling and private information
<code>economic_game_strategic_voting.py</code>	Voting
<code>economic_game_ultimatum.py</code>	Transfer games
<code>economic_game_voting_rules.py</code>	Voting
<code>economic_games_auction_comparison.py</code>	Auctions
<code>economic_games_group.py</code>	Beauty contest and common pool
<code>economic_games_matrix.py</code>	Matrix games
<code>economic_games_transfer.py</code>	Dictator and trust games
<code>shared_state_adversarial_hiring_committee.py</code>	Hiring committees
<code>shared_state_bilateral_negotiation.py</code>	Negotiation
<code>shared_state_binary_prediction_market.py</code>	Prediction markets
<code>shared_state_budget_allocation.py</code>	Resource allocation
<code>shared_state_coalition_formation.py</code>	Coalition formation
<code>shared_state_congested_matching_market.py</code>	Matching markets
<code>shared_state_customer_feedback_synthesis.py</code>	Multi-phase synthesis
<code>shared_state_delphi_forecast.py</code>	Delphi panels
<code>shared_state_disaster_response.py</code>	Resource allocation
<code>shared_state_family_message_board.py</code>	Message boards
<code>shared_state_forecast_revision.py</code>	Forecast revision
<code>shared_state_hiring_committee.py</code>	Hiring committees
<code>shared_state_incident_response.py</code>	Work queues and command synthesis

Repository example	Manual treatment
<code>shared_state_launch_readiness_review.py</code>	Launch review
<code>shared_state_legislative_amendments.py</code>	Versioned document revision
<code>shared_state_live_review_queue.py</code>	Work queues
<code>shared_state_matching_market.py</code>	Matching markets
<code>shared_state_meeting_agenda.py</code>	Agenda setting
<code>shared_state_peer_review_matching.py</code>	Matching with conflicts
<code>shared_state_prediction_market_private_news.py</code>	Private-news markets
<code>shared_state_public_goods.py</code>	Repeated public goods
<code>shared_state_rumor_diffusion.py</code>	Information diffusion
<code>shared_state_strategic_planning_tiered.py</code>	Tiered strategic planning
<code>shared_state_strategic_planning_workshop.py</code>	Propose–challenge–revise–vote

Chapter 16

Complete current implementations

The following appendix reproduces the complete source of every example discussed in this manual. It is intentionally included rather than linked: a reader needs only this document to inspect the programs. The shorter listings in the main chapters show the preferred prospective API; these full listings show the current experimental API. Reading them side by side is part of the design exercise.

Each listing is preceded by commentary identifying what to inspect. Repetition is evidence: repeated plumbing is a candidate for a library abstraction, while differences that encode a treatment or information structure must remain visible to the researcher.

16.1 `economic_game_11_20_money_request.py`

Focus: Nash demand and 11–20 request. Inspect numeric action constraints, simultaneous revelation, and deterministic settlement.

Compare this complete current program with the preferred construction in the corresponding chapter. Identify which lines express the research design and which exist only to serialize, schedule, authorize state reads, recover execution, or reshape results.

```
"""Four parallel sealed 11-20 money-request games."""

from pathlib import Path

from edsl import Agent, AgentList, InterviewSchedule, Model, QuestionNumerical, Survey
from edsl.jobs.interview_schedule import GroupStopCondition
from edsl.sharedstate import FileStateStore, SharedMoneyRequestGame, SharedState

def players():
    specs = [
        ("A1", "pair-1", "chooses the largest guaranteed personal payment"),
        ("B1", "pair-1", "expects the opponent to choose the obvious maximum"),
        ("A2", "pair-2", "uses one step of strategic reasoning"),
        ("B2", "pair-2", "uses two steps of strategic reasoning"),
        ("A3", "pair-3", "believes most people choose 19"),
        ("B3", "pair-3", "believes sophisticated players undercut repeatedly"),
        ("A4", "pair-4", "dislikes appearing greedy but values the bonus"),
```

```

        ("B4", "pair-4", "randomizes mentally between plausible strategic choices"),
    ]
    return AgentList(
        [
            Agent(name=name, traits={"pair_id": pair, "reasoning_style": style})
            for name, pair, style in specs
        ]
    )

def survey(state):
    request = QuestionNumerical(
        question_name="request",
        question_text=(
            "You are {{ agent.name }} in a one-shot two-player money-request game. "
            "You {{ agent.reasoning_style }}. Each player simultaneously requests an "
            "integer from 11 through 20 and receives that amount. Additionally, a "
            "player who requests exactly one less than the other receives a $20 bonus. "
            "Choices are sealed until both commit. Choose your request."
        ),
        min_value=11,
        max_value=20,
    )
    return Survey([request, state.game.submit(request)])

def run_simulation(
    log_path: str | Path = "economic-game-11-20.jsonl",
    model_name="gemini-2.5-flash",
):
    state = SharedState(
        "{{ agent.pair_id }}",
        FileStateStore(log_path),
        game=SharedMoneyRequestGame(),
    )
    schedule = InterviewSchedule.rounds(
        count=1,
        group_by="pair_id",
        within_round="concurrent",
        state_visibility="snapshot",
        reveal="after_round",
        finalize_when=GroupStopCondition("game", "complete"),
    )
    (
        survey(state)
        .by(players())
        .by(Model(model_name))
        .run(
            interview_schedule=schedule,

```

```

        disable_remote_inference=True,
        disable_remote_cache=True,
        cache=False,
        stop_on_exceptions=True,
    )
)
return state

if __name__ == "__main__":
    result = run_simulation()
    for pair_id in ("pair-1", "pair-2", "pair-3", "pair-4"):
        print(result.render_markdown(scope=pair_id), "\n")

```

16.2 economic_game_adverse_selection.py

Focus: Signaling and private information. Inspect private facts, public actions, and prompt-level access control.

Compare this complete current program with the preferred construction in the corresponding chapter. Identify which lines express the research design and which exist only to serialize, schedule, authorize state reads, recover execution, or reshape results.

```

"""Posted-price bilateral trade with privately informed sellers."""

from pathlib import Path

from edsl import (
    Agent,
    AgentList,
    InterviewSchedule,
    Model,
    QuestionMultipleChoice,
    QuestionNumerical,
    Survey,
)
from edsl.jobs.interview_schedule import GroupStopCondition
from edsl.sharedstate import FileStateStore, SharedBilateralTrade, SharedState

def participants():
    specs = [
        (
            "Buyer-1",
            "pair-1",
            0,
            "buyer",
            100,
            0,

```

```

        "risk neutral; believes seller cost is uniformly 20-80",
    ),
    (
        "Seller-1",
        "pair-1",
        1,
        "seller",
        0,
        30,
        "will trade whenever price covers private cost",
    ),
    (
        "Buyer-2",
        "pair-2",
        0,
        "buyer",
        100,
        0,
        "cautious; believes high-cost sellers are common",
    ),
    (
        "Seller-2",
        "pair-2",
        1,
        "seller",
        0,
        60,
        "will trade whenever price covers private cost",
    ),
    (
        "Buyer-3",
        "pair-3",
        0,
        "buyer",
        100,
        0,
        "aggressive bargainer; expects sellers to concede",
    ),
    (
        "Seller-3",
        "pair-3",
        1,
        "seller",
        0,
        75,
        "will trade whenever price covers private cost",
    ),
]
return AgentList(

```

```

    [
        Agent(
            name=n,
            traits={
                "pair_id": p,
                "turn": t,
                "role": r,
                "buyer_value": v,
                "seller_cost": c,
                "strategy": s,
            },
        )
        for n, p, t, r, v, c, s in specs
    ]
)

def run_simulation(
    path: str | Path = "economic-game-adverse-selection.jsonl",
    model_name="gemini-2.5-flash",
):
    state = SharedState(
        "{{ agent.pair_id }}", FileStateStore(path), trade=SharedBilateralTrade()
    )
    offer = QuestionNumerical(
        question_name="offer",
        question_text=(
            "You are {{ agent.name }}, a buyer valuing an asset at "
            "{{ agent.buyer_value }}. You are {{ agent.strategy }}. The seller privately "
            "knows their cost; you do not. Post one take-it-or-leave-it price from 0-100."
        ),
        min_value=0,
        max_value=100,
    )
    decision = QuestionMultipleChoice(
        question_name="decision",
        question_text=(
            "You are {{ agent.name }}, a seller with private cost {{ agent.seller_cost }}. "
            "You are {{ agent.strategy }}. The buyer posted {{ shared_state.trade.price }}. "
            "Accept or reject to maximize price minus cost."
        ),
        question_options=["accept", "reject"],
    )
    survey = Survey(
        [offer, state.trade.offer(offer), decision, state.trade.respond(decision)]
    )
    survey.add_skip_rule("offer", "'{{ agent.role }}' != 'buyer'")
    survey.add_skip_rule("decision", "'{{ agent.role }}' != 'seller'")
    terminal = GroupStopCondition("trade", "terminal")

```

```

    schedule = InterviewSchedule.grouped_round_robin(
        "pair_id", "turn", finalize_when=terminal
    )
    survey.by(participants()).by(Model(model_name)).run(
        interview_schedule=schedule,
        disable_remote_inference=True,
        disable_remote_cache=True,
        cache=False,
        stop_on_exceptions=True,
    )
    return state

if __name__ == "__main__":
    result = run_simulation()
    for pair in ("pair-1", "pair-2", "pair-3"):
        print(result.render_markdown(scope=pair), "\n")

```

16.3 economic_game_centipede.py

Focus: Variable-duration sequential games. Inspect turn ownership, terminal predicates, and protection against late actions.

Compare this complete current program with the preferred construction in the corresponding chapter. Identify which lines express the research design and which exist only to serialize, schedule, authorize state reads, recover execution, or reshape results.

```

"""Six-node centipede game with early stopping after take."""

from pathlib import Path

from edsl import (
    Agent,
    AgentList,
    InterviewSchedule,
    Model,
    QuestionMultipleChoice,
    Survey,
)
from edsl.jobs.interview_schedule import GroupStopCondition
from edsl.sharedstate import FileStateStore, SharedCentipedeGame, SharedState

TAKE_PAYOFFS = [[2, 0], [1, 3], [4, 2], [3, 5], [6, 4], [5, 7]]

def decision_nodes():
    agents = []
    for node in range(1, 7):

```

```

player = "Alice" if node % 2 else "Bob"
disposition = (
    "values reciprocity but reasons strategically"
    if player == "Alice"
    else "is cautiously cooperative but fears exploitation"
)
agents.append(
    Agent(
        name=f"{player}-node-{node}",
        traits={
            "game_id": "centipede-1",
            "node": node,
            "player": player,
            "disposition": disposition,
        },
    )
)
return AgentList(agents)

def run_simulation(
    path: str | Path = "economic-game-centipede.jsonl",
    model_name="gemini-2.5-flash",
):
    state = SharedState(
        "centipede-1",
        FileStateStore(path),
        game=SharedCentipedeGame(TAKE_PAYOFFS, [6, 6]),
    )
    action = QuestionMultipleChoice(
        question_name="action",
        question_text=(
            "You are {{ agent.player }} acting at node {{ agent.node }} of a six-node "
            "centipede game and {{ agent.disposition }}. Alice's payoff is listed first. "
            "Taking at nodes 1-6 yields respectively (2,0), (1,3), (4,2), (3,5), "
            "(6,4), and (5,7). Passing at node 6 yields (6,6). Earlier passes move to "
            "the next node. Public history: {{ shared_state.game.history }}. Choose."
        ),
        question_options=["take", "pass"],
    )
    survey = Survey([action, state.game.move(action)])
    terminal = GroupStopCondition("game", "terminal")
    schedule = InterviewSchedule.grouped_round_robin(
        "game_id", "node", stop_when=terminal, finalize_when=terminal
    )
    survey.by(decision_nodes()).by(Model(model_name)).run(
        interview_schedule=schedule,
        disable_remote_inference=True,
        disable_remote_cache=True,
    )

```

```

        cache=False,
        stop_on_exceptions=True,
    )
    return state

if __name__ == "__main__":
    print(run_simulation().render_markdown())

```

16.4 economic_game_cheap_talk.py

Focus: Signaling and private information. Inspect private facts, public actions, and prompt-level access control.

Compare this complete current program with the preferred construction in the corresponding chapter. Identify which lines express the research design and which exist only to serialize, schedule, authorize state reads, recover execution, or reshape results.

```

"""Cheap-talk communication with aligned and biased senders."""

from pathlib import Path

from edsl import (
    Agent,
    AgentList,
    InterviewSchedule,
    Model,
    QuestionMultipleChoice,
    Survey,
)
from edsl.jobs.interview_schedule import GroupStopCondition
from edsl.sharedstate import FileStateStore, SharedCheapTalkGame, SharedState

def participants():
    specs = [
        (
            "Sender-1",
            "pair-1",
            0,
            "sender",
            "L",
            "aligned",
            "Sender wants the receiver to match the true state.",
        ),
        (
            "Receiver-1",
            "pair-1",
            1,

```



```

    "receiver",
    "L",
    "aligned",
    "You know the sender is aligned.",
),
(
    "Sender-2",
    "pair-2",
    0,
    "sender",
    "R",
    "aligned",
    "Sender wants the receiver to match the true state.",
),
(
    "Receiver-2",
    "pair-2",
    1,
    "receiver",
    "L",
    "aligned",
    "You know the sender is aligned.",
),
(
    "Sender-3",
    "pair-3",
    0,
    "sender",
    "L",
    "biased",
    "Sender earns 1 whenever receiver chooses R, regardless of state.",
),
(
    "Receiver-3",
    "pair-3",
    1,
    "receiver",
    "L",
    "biased",
    "You know the sender always prefers action R.",
),
(
    "Sender-4",
    "pair-4",
    0,
    "sender",
    "R",
    "biased",
    "Sender earns 1 whenever receiver chooses R, regardless of state.",

```

```

    ),
    (
        "Receiver-4",
        "pair-4",
        1,
        "receiver",
        "L",
        "biased",
        "You know the sender always prefers action R.",
    ),
]
return AgentList(
    [
        Agent(
            name=n,
            traits={
                "pair_id": p,
                "turn": t,
                "role": r,
                "private_state": state,
                "sender_preference": pref,
                "information": info,
            },
        )
        for n, p, t, r, state, pref, info in specs
    ]
)

def run_simulation(
    path: str | Path = "economic-game-cheap-talk.jsonl", model_name="gemini-2.5-flash"
):
    state = SharedState(
        "{{ agent.pair_id }}", FileStateStore(path), game=SharedCheapTalkGame()
    )
    message = QuestionMultipleChoice(
        question_name="message",
        question_text=(
            "You are {{ agent.name }}, the sender. The equally likely true state, "
            "observed only by you, is {{ agent.private_state }}. {{ agent.information }}"
            "Send costless message L or R. The receiver knows your incentive but not the state."
        ),
        question_options=["L", "R"],
    )
    action = QuestionMultipleChoice(
        question_name="action",
        question_text=(
            "You are {{ agent.name }}, the receiver. The state is equally likely L or "
            "R and matching it pays you 1. {{ agent.information }} The sender's "

```

```

        "costless message is {{ shared_state.game.message }}. Choose action L or R."
    ),
    question_options=["L", "R"],
)
survey = Survey(
    [message, state.game.message(message), action, state.game.act(action)]
)
survey.add_skip_rule("message", "'{{ agent.role }}" != 'sender')
survey.add_skip_rule("action", "'{{ agent.role }}" != 'receiver')
terminal = GroupStopCondition("game", "terminal")
schedule = InterviewSchedule.grouped_round_robin(
    "pair_id", "turn", finalize_when=terminal
)
survey.by(participants()).by(Model(model_name)).run(
    interview_schedule=schedule,
    disable_remote_inference=True,
    disable_remote_cache=True,
    cache=False,
    stop_on_exceptions=True,
)
return state

if __name__ == "__main__":
    result = run_simulation()
    for pair in ("pair-1", "pair-2", "pair-3", "pair-4"):
        print(result.render_markdown(scope=pair), "\n")

```

16.5 economic_game_continuous_double_auction.py

Focus: Auctions and live markets. Inspect order validity, price-time priority, settlement, and concurrency.

Compare this complete current program with the preferred construction in the corresponding chapter. Identify which lines express the research design and which exist only to serialize, schedule, authorize state reads, recover execution, or reshape results.

```

"""A live unit-order double auction with private buyer values and seller costs."""

from pathlib import Path

from edsl import (
    Agent,
    AgentList,
    InterviewSchedule,
    Model,
    QuestionMultipleChoice,
    QuestionNumerical,
    Survey,

```

```

)
from edsl.sharedstate import FileStateStore, SharedDoubleAuction, SharedState

TRADERS = [
    ("Buyer 1", "buyer", 112),
    ("Buyer 2", "buyer", 98),
    ("Buyer 3", "buyer", 84),
    ("Buyer 4", "buyer", 69),
    ("Seller 1", "seller", 42),
    ("Seller 2", "seller", 58),
    ("Seller 3", "seller", 76),
    ("Seller 4", "seller", 91),
]

def traders() -> AgentList:
    return AgentList(
        [
            Agent(
                name=name,
                traits={
                    "role": role,
                    "private_limit": limit,
                    "objective": (
                        "buy one unit at or below your private value"
                        if role == "buyer"
                        else "sell one unit at or above your private cost"
                    ),
                },
            )
            for name, role, limit in TRADERS
        ]
    )

def survey(state: SharedState) -> Survey:
    action = QuestionMultipleChoice(
        question_name="order_action",
        question_text=(
            "Round {{ run.round }} of 3. You are {{ agent.name }}, a {{ agent.role }}, "
            "and should {{ agent.objective }}. Your private limit is "
            "{{ agent.private_limit }}.\n\nLive order book and your account:\n"
            "{{ shared_state.book }}\n\nChoose buy or sell only for your assigned role. "
            "If you already traded, hold. If you have an open order and want a new "
            "price, cancel it this round; you can replace it next round."
        ),
        question_options=["buy", "sell", "cancel", "hold"],
    )

```

```

price = QuestionNumerical(
    question_name="limit_price",
    question_text=(
        "You selected {{ order_action.answer }}. If buying or selling, submit a "
        "profitable integer limit price from 1 to 150. For cancel or hold, enter 0."
    ),
    min_value=0,
    max_value=150,
)
return Survey([action, price, state.book.submit(action, price)])

def realized_surplus(state: SharedState) -> tuple[float, list[dict]]:
    limits = {name: (role, limit) for name, role, limit in TRADERS}
    trades = state.read().state["book"]["trades"]
    details = []
    total = 0.0
    for trade in trades:
        buyer_value = limits[trade["buyer"]][1]
        seller_cost = limits[trade["seller"]][1]
        surplus = buyer_value - seller_cost
        total += surplus
        details.append(dict(trade) | {"surplus": surplus})
    return total, details

def run_double_auction(
    log_path: str | Path = "economic-game-double-auction.jsonl",
    model_name: str = "gemini-2.5-flash",
) -> tuple[SharedState, float, list[dict]]:
    participants = {
        name: {
            "cash": 150 if role == "buyer" else 0,
            "inventory": 0 if role == "buyer" else 1,
        }
        for name, role, _ in TRADERS
    }
    state = SharedState(
        "continuous-double-auction",
        FileStateStore(log_path),
        book=SharedDoubleAuction(participants),
    )
    schedule = InterviewSchedule.rounds(
        count=3,
        within_round="concurrent",
        state_visibility="live",
        round_order="rotate",
    )
    (

```

```

    survey(state)
    .by(traders())
    .by(Model(model_name))
    .run(
        interview_schedule=schedule,
        disable_remote_inference=True,
        disable_remote_cache=True,
        cache=False,
        stop_on_exceptions=True,
    )
)
state.close()
total, trades = realized_surplus(state)
return state, total, trades

if __name__ == "__main__":
    shared_state, surplus, executed = run_double_auction()
    print(shared_state.render_markdown())
    print(f"\nRealized surplus: {surplus:g}\nTrades: {executed}")

```

16.6 economic_game_information_cascade.py

Focus: Information cascades. Inspect private signals versus the public history visible to later actors.

Compare this complete current program with the preferred construction in the corresponding chapter. Identify which lines express the research design and which exist only to serialize, schedule, authorize state reads, recover execution, or reshape results.

```

"""Sequential social learning with public choices and private signals."""

from pathlib import Path

from edsl import Agent, AgentList, Model, QuestionMultipleChoice, Survey
from edsl.sharedstate import FileStateStore, SharedLog, SharedState

TRUE_STATE = "A"

def observers():
    signals = ["B", "B", "A", "A", "A", "A"]
    return AgentList(
        [
            Agent(
                name=f"Observer-{index}",
                traits={
                    "private_signal": signal,
                    "position": index,

```

```

        "reasoning": "Bayesian and attentive to informational redundancy",
    },
)
    for index, signal in enumerate(signals, 1)
]
)

def run_simulation(
    path: str | Path = "economic-game-information-cascade.jsonl",
    model_name="gemini-2.5-flash",
):
    state = SharedState(
        "information-cascade",
        FileStateStore(path),
        choices=SharedLog(),
    )
    choice = QuestionMultipleChoice(
        question_name="choice",
        question_text=(
            "An unknown state is equally likely to be A or B. You are "
            "{{ agent.name }}", choosing sequentially at position {{ agent.position }}. "
            "Your private signal is {{ agent.private_signal }} and independently "
            "matches the true state with probability 0.70. Prior agents' public "
            "choices are {{ shared_state.choices.entries }}. You do not observe their "
            "signals. Choose the state you believe more likely. Remember that later "
            "public choices may repeat rather than add independent information."
        ),
        question_options=["A", "B"],
    )
    survey = Survey(
        [
            choice,
            state.choices.append(
                observer="{{ agent.name }}",
                position="{{ agent.position }}",
                choice=choice,
            ),
        ]
    )
    survey.by(observers()).by(Model(model_name)).run(
        interview_schedule="serial",
        disable_remote_inference=True,
        disable_remote_cache=True,
        cache=False,
        stop_on_exceptions=True,
    )
    state.close()
    return state

```

```

def summarize(state):
    entries = state.read().state["choices"]["entries"]
    lines = [
        f"True state: {TRUE_STATE}",
        "",
        "| Position | Signal | Choice | Correct |",
        "|---:|---|---|---|",
    ]
    signal_by_name = {
        agent.name: agent.traits["private_signal"] for agent in observers()
    }
    for item in entries:
        lines.append(
            f"| {item['position']} | {signal_by_name[item['observer']] } | "
            f"{item['choice']} | {'yes' if item['choice'] == TRUE_STATE else 'no'} | "
        )
    return "\n".join(lines)

if __name__ == "__main__":
    print(summarize(run_simulation()))

```

16.7 economic_game_market_entry.py

Focus: Anonymous group mechanisms. Inspect sealed group actions and explicit missing-participant policy.

Compare this complete current program with the preferred construction in the corresponding chapter. Identify which lines express the research design and which exist only to serialize, schedule, authorize state reads, recover execution, or reshape results.

```

"""Sealed market-entry game with congestion-dependent profits."""

from pathlib import Path

from edsl import (
    Agent,
    AgentList,
    InterviewSchedule,
    Model,
    QuestionMultipleChoice,
    Survey,
)
from edsl.jobs.interview_schedule import GroupStopCondition
from edsl.sharedstate import FileStateStore, SharedMarketEntryGame, SharedState

def run_simulation(

```



```

path: str | Path = "economic-game-market-entry.jsonl", model_name="gemini-2.5-flash"
):
    beliefs = [
        ("Lena", "optimistic and willing to enter crowded markets"),
        ("Milo", "cautious and expects several competitors"),
        ("Nia", "risk neutral and calculates expected payoff"),
        ("Omar", "overconfident about being early"),
        ("Pia", "risk averse and prefers a safe outside option"),
        ("Raj", "strategic and expects others to avoid congestion"),
    ]
    agents = AgentList([Agent(name=n, traits={"belief": b}) for n, b in beliefs])
    state = SharedState(
        "market-entry", FileStateStore(path), game=SharedMarketEntryGame(6, 2, 10, 3)
    )
    action = QuestionMultipleChoice(
        question_name="entry",
        question_text=(
            "Six firms simultaneously choose enter or stay_out. You are {{ agent.name }}, "
            "{{ agent.belief }}. Staying out pays 2. If k firms enter, every entrant "
            "earns 10 - 3k. Choices are sealed. Choose your action."
        ),
        question_options=["enter", "stay_out"],
    )
    schedule = InterviewSchedule.rounds(
        count=1,
        reveal="after_round",
        finalize_when=GroupStopCondition("game", "complete"),
    )
    Survey([action, state.game.submit(action)]).by(agents).by(Model(model_name)).run(
        interview_schedule=schedule,
        disable_remote_inference=True,
        disable_remote_cache=True,
        cache=False,
        stop_on_exceptions=True,
    )
    return state

if __name__ == "__main__":
    print(run_simulation().render_markdown())

```

16.8 economic_game_moral_hazard.py

Focus: Signaling and private information. Inspect private facts, public actions, and prompt-level access control.

Compare this complete current program with the preferred construction in the corresponding chapter. Identify which lines express the research design and which exist only to serialize, schedule, authorize state reads, recover execution, or reshape results.

```
"""Principal-agent contracting with private costly effort."""
```

```
from pathlib import Path
```

```
from edsl import (
```

```
    Agent,  
    AgentList,  
    InterviewSchedule,  
    Model,  
    QuestionMultipleChoice,  
    QuestionNumerical,  
    Survey,  
)
```

```
from edsl.jobs.interview_schedule import GroupStopCondition
```

```
from edsl.sharedstate import FileStateStore, SharedPrincipalAgentGame, SharedState
```

```
def participants():
```

```
    specs = [
```

```
        (  
            "Principal-1",  
            "pair-1",  
            0,  
            "principal",  
            "calculates the minimum incentive-compatible bonus",  
        ),  
        ("Worker-1", "pair-1", 1, "worker", "maximizes expected monetary payoff"),  
        (  
            "Principal-2",  
            "pair-2",  
            0,  
            "principal",  
            "is stingy and dislikes sharing output",  
        ),  
        ("Worker-2", "pair-2", 1, "worker", "maximizes expected monetary payoff"),  
        (  
            "Principal-3",  
            "pair-3",  
            0,  
            "principal",  
            "uses a generous bonus to strongly motivate effort",  
        ),  
        ("Worker-3", "pair-3", 1, "worker", "maximizes expected monetary payoff"),  
    ]
```

```
    return AgentList(
```

```
        [  
            Agent(name=n, traits={"pair_id": p, "turn": t, "role": r, "strategy": s})  
            for n, p, t, r, s in specs  
        ]
```

```

)

def run_simulation(
    path: str | Path = "economic-game-moral-hazard.jsonl", model_name="gemini-2.5-flash"
):
    state = SharedState(
        "{{ agent.pair_id }}", FileStateStore(path), game=SharedPrincipalAgentGame()
    )
    bonus = QuestionNumerical(
        question_name="bonus",
        question_text=(
            "You are {{ agent.name }}, the principal, and {{ agent.strategy }}. Output "
            "is worth 100 on success. A worker privately chooses high effort (success "
            "probability .8, cost 20) or low effort (probability .2, cost 0). Offer a "
            "success-contingent bonus from 0-100 to maximize expected output minus bonus."
        ),
        min_value=0,
        max_value=100,
    )
    effort = QuestionMultipleChoice(
        question_name="effort",
        question_text=(
            "You are {{ agent.name }}, the worker, and {{ agent.strategy }}. The success "
            "bonus is {{ shared_state.game.bonus }}. High effort has success probability "
            ".8 and cost 20; low effort has probability .2 and cost 0. Effort is private. Choose "
        ),
        question_options=["high", "low"],
    )
    survey = Survey(
        [bonus, state.game.contract(bonus), effort, state.game.effort(effort)]
    )
    survey.add_skip_rule("bonus", "'{{ agent.role }}" != 'principal')
    survey.add_skip_rule("effort", "'{{ agent.role }}" != 'worker')
    terminal = GroupStopCondition("game", "terminal")
    schedule = InterviewSchedule.grouped_round_robin(
        "pair_id", "turn", finalize_when=terminal
    )
    survey.by(participants()).by(Model(model_name)).run(
        interview_schedule=schedule,
        disable_remote_inference=True,
        disable_remote_cache=True,
        cache=False,
        stop_on_exceptions=True,
    )
    return state

if __name__ == "__main__":

```

```

result = run_simulation()
for pair in ("pair-1", "pair-2", "pair-3"):
    print(result.render_markdown(scope=pair), "\n")

```

16.9 economic_game_nash_demand.py

Focus: Nash demand and 11–20 request. Inspect numeric action constraints, simultaneous revelation, and deterministic settlement.

Compare this complete current program with the preferred construction in the corresponding chapter. Identify which lines express the research design and which exist only to serialize, schedule, authorize state reads, recover execution, or reshape results.

```

"""Sealed simultaneous Nash demand bargaining."""

from pathlib import Path

from edsl import Agent, AgentList, InterviewSchedule, Model, QuestionNumerical, Survey
from edsl.jobs.interview_schedule import GroupStopCondition
from edsl.sharedstate import FileStateStore, SharedNashDemandGame, SharedState

def run_simulation(
    path: str | Path = "economic-game-nash-demand.jsonl", model_name="gemini-2.5-flash"
):
    specs = [
        ("A1", "pair-1", 0, "expects a fair 50/50 convention"),
        ("B1", "pair-1", 1, "expects a fair 50/50 convention"),
        ("A2", "pair-2", 0, "makes an assertive but coordination-aware demand"),
        ("B2", "pair-2", 1, "is accommodating to avoid bargaining failure"),
        ("A3", "pair-3", 0, "demands aggressively and expects the other to yield"),
        ("B3", "pair-3", 1, "demands aggressively and expects the other to yield"),
    ]
    agents = AgentList(
        [
            Agent(name=n, traits={"pair_id": p, "seat": s, "style": x})
            for n, p, s, x in specs
        ]
    )
    state = SharedState(
        "{{ agent.pair_id }}", FileStateStore(path), game=SharedNashDemandGame(100)
    )
    demand = QuestionNumerical(
        question_name="demand",
        question_text=(
            "You and one other player simultaneously demand 0-100 from a pie of 100. "
            "You are {{ agent.name }} and {{ agent.style }}. If demands sum to at most "
            "100, each receives their demand and the remainder is wasted. If they exceed "
            "100, both receive zero. Demands are sealed. Choose yours."
        )
    )

```

```

        ),
        min_value=0,
        max_value=100,
    )
    schedule = InterviewSchedule.rounds(
        count=1,
        group_by="pair_id",
        reveal="after_round",
        finalize_when=GroupStopCondition("game", "complete"),
    )
    Survey([demand, state.game.demand(demand)]).by(agents).by(Model(model_name)).run(
        interview_schedule=schedule,
        disable_remote_inference=True,
        disable_remote_cache=True,
        cache=False,
        stop_on_exceptions=True,
    )
    return state

if __name__ == "__main__":
    result = run_simulation()
    for pair in ("pair-1", "pair-2", "pair-3"):
        print(result.render_markdown(scope=pair), "\n")

```

16.10 economic_game_public_goods_punishment.py

Focus: Public goods and punishment. Inspect stage boundaries, target validation, and authoritative payoffs.

Compare this complete current program with the preferred construction in the corresponding chapter. Identify which lines express the research design and which exist only to serialize, schedule, authorize state reads, recover execution, or reshape results.

```

"""One-shot public goods followed by sealed peer punishment."""

from pathlib import Path

from edsl import (
    Agent,
    AgentList,
    InterviewSchedule,
    Model,
    QuestionMatrix,
    QuestionNumerical,
    Survey,
)
from edsl.sharedstate import FileStateStore, SharedLog, SharedState

```

```

def players():
    specs = [
        ("Avery", "conditional cooperator who punishes clear free-riding"),
        ("Blake", "self-interested optimizer who avoids costly punishment"),
        ("Casey", "strong norm enforcer focused on group welfare"),
        ("Devon", "reciprocal pragmatist who uses proportionate sanctions"),
    ]
    return AgentList([Agent(name=n, traits={"strategy": s}) for n, s in specs])

def run_simulation(
    path: str | Path = "economic-game-public-goods-punishment.jsonl",
    model_name="gemini-2.5-flash",
):
    state = SharedState(
        "public-goods-punishment",
        FileStateStore(path),
        contributions=SharedLog(),
        punishments=SharedLog(),
    )
    agents, model = players(), Model(model_name)
    contribution = QuestionNumerical(
        question_name="contribution",
        question_text=(
            "You have 20 tokens. Four simultaneous contributions are multiplied by "
            "1.6 and divided equally; unspent tokens remain yours. You are "
            "{{ agent.name }}, a {{ agent.strategy }}. Choices are sealed. Contribute 0-20."
        ),
        min_value=0,
        max_value=20,
    )
    Survey(
        [
            contribution,
            state.contributions.append(player="{{ agent.name }}", amount=contribution),
        ]
    ).by(agents).by(model).run(
        interview_schedule=InterviewSchedule.rounds(count=1, reveal="after_round"),
        disable_remote_inference=True,
        disable_remote_cache=True,
        cache=False,
        stop_on_exceptions=True,
    )

    entries = state.read().state["contributions"]["entries"]
    slate = ", ".join(f"{{item['player']}}={{item['amount']}}" for item in entries)
    names = [agent.name for agent in agents]
    punishment = QuestionMatrix(

```

```

question_name="punishment",
question_text=(
    f"Contributions were {slate}. You are {{{ agent.name }}}}, a "
    f"{{{ agent.strategy }}}. Assign 0-3 punishment points to every player. "
    "Each point costs you 1 token and reduces the target's payoff by 3. "
    "You may assign zero to yourself and everyone else."
),
question_items=names,
question_options=["0", "1", "2", "3"],
)
Survey(
    [
        punishment,
        state.punishments.append(player="{{ agent.name }}", points=punishment),
    ]
).by(agents).by(model).run(
    interview_schedule=InterviewSchedule.rounds(count=1, reveal="after_round"),
    disable_remote_inference=True,
    disable_remote_cache=True,
    cache=False,
    stop_on_exceptions=True,
)
state.close()
return state

```

```

def summarize(state):
    contributions = state.read().state["contributions"]["entries"]
    punishments = state.read().state["punishments"]["entries"]
    pot = sum(item["amount"] for item in contributions)
    share = 1.6 * pot / 4
    payoff = {item["player"]: 20 - item["amount"] + share for item in contributions}
    received = {name: 0 for name in payoff}
    spent = {name: 0 for name in payoff}
    for ballot in punishments:
        for target, points in ballot["points"].items():
            value = int(points)
            spent[ballot["player"]] += value
            received[target] += value
    lines = [
        f"Group contribution: {pot}/80",
        "",
        "| Player | Contributed | Punishment spent | Received | Final payoff |",
        "|---|---|---:|---:|---:|",
    ]
    amounts = {item["player"]: item["amount"] for item in contributions}
    for name in payoff:
        final = payoff[name] - spent[name] - 3 * received[name]
        lines.append(

```

```

        f"| {name} | {amounts[name]} | {spent[name]} | {received[name]} | {final:.1f} |"
    )
    return "\n".join(lines)

if __name__ == "__main__":
    result = run_simulation()
    print(summarize(result))

```

16.11 economic_game_repeated_prisoners_dilemma.py

Focus: Repeated matrix games. Inspect round scoping, sealed current actions, and revealed completed history.

Compare this complete current program with the preferred construction in the corresponding chapter. Identify which lines express the research design and which exist only to serialize, schedule, authorize state reads, recover execution, or reshape results.

```

"""Three-round sealed prisoner's dilemma with revealed prior-round history."""

from pathlib import Path

from edsl import (
    Agent,
    AgentList,
    InterviewSchedule,
    Model,
    QuestionMultipleChoice,
    Survey,
)
from edsl.jobs.interview_schedule import GroupStopCondition
from edsl.sharedstate import FileStateStore, SharedRepeatedMatrixGame, SharedState

PAYOFFS = {
    "cooperate|cooperate": [3, 3],
    "cooperate|defect": [0, 5],
    "defect|cooperate": [5, 0],
    "defect|defect": [1, 1],
}

def players():
    specs = [
        ("Tara", "pair-1", 0, "start cooperative, then use tit-for-tat"),
        ("Felix", "pair-1", 1, "start cooperative and forgive one defection"),
        ("Greta", "pair-2", 0, "cooperate until betrayed, then defect forever"),
        ("Dex", "pair-2", 1, "always defect to maximize immediate payoff"),
    ]

```



```

return AgentList(
    [
        Agent(name=n, traits={"pair_id": p, "seat": s, "strategy": x})
        for n, p, s, x in specs
    ]
)

def run_simulation(
    path: str | Path = "economic-game-repeated-pd.jsonl", model_name="gemini-2.5-flash"
):
    state = SharedState(
        "{{ agent.pair_id }}",
        FileStateStore(path),
        game=SharedRepeatedMatrixGame(["cooperate", "defect"], PAYOFFS, 3),
    )
    action = QuestionMultipleChoice(
        question_name="action",
        question_text=(
            "Round {{ run.round }} of 3 in a repeated prisoner's dilemma. You are "
            "{{ agent.name }} and should {{ agent.strategy }}. Payoffs are C/C=(3,3), "
            "C/D=(0,5), D/C=(5,0), D/D=(1,1). Completed public history: "
            "{{ shared_state.game.history }}. Current-round actions are sealed. Choose."
        ),
        question_options=["cooperate", "defect"],
    )
    survey = Survey([action, state.game.submit(action)])
    schedule = InterviewSchedule.rounds(
        count=3,
        group_by="pair_id",
        within_round="concurrent",
        reveal="after_round",
        finalize_when=GroupStopCondition("game", "complete"),
    )
    survey.by(players()).by(Model(model_name)).run(
        interview_schedule=schedule,
        disable_remote_inference=True,
        disable_remote_cache=True,
        cache=False,
        stop_on_exceptions=True,
    )
    return state

if __name__ == "__main__":
    result = run_simulation()
    for pair in ("pair-1", "pair-2"):
        print(result.render_markdown(scope=pair), "\n")

```

16.12 economic_game_signaling.py

Focus: Signaling and private information. Inspect private facts, public actions, and prompt-level access control.

Compare this complete current program with the preferred construction in the corresponding chapter. Identify which lines express the research design and which exist only to serialize, schedule, authorize state reads, recover execution, or reshape results.

```
"""Education signaling with private worker productivity."""

from pathlib import Path

from edsl import (
    Agent,
    AgentList,
    InterviewSchedule,
    Model,
    QuestionMultipleChoice,
    QuestionNumerical,
    Survey,
)
from edsl.jobs.interview_schedule import GroupStopCondition
from edsl.sharedstate import FileStateStore, SharedSignalingGame, SharedState


def participants():
    specs = [
        (
            "High-1",
            "pair-1",
            0,
            "worker",
            100,
            5,
            "high productivity and low education cost",
        ),
        (
            "Firm-1",
            "pair-1",
            1,
            "employer",
            0,
            0,
            "believes education of 2 or more strongly predicts high productivity",
        ),
        (
            "Low-1",
            "pair-2",
            0,
```

```

        "worker",
        40,
        20,
        "low productivity and high education cost",
    ),
    (
        "Firm-2",
        "pair-2",
        1,
        "employer",
        0,
        0,
        "believes education of 2 or more strongly predicts high productivity",
    ),
    (
        "High-2",
        "pair-3",
        0,
        "worker",
        100,
        5,
        "high productivity and low education cost",
    ),
    (
        "Firm-3",
        "pair-3",
        1,
        "employer",
        0,
        0,
        "is skeptical and requires a strong education signal",
    ),
    (
        "Low-2",
        "pair-4",
        0,
        "worker",
        40,
        20,
        "low productivity but is willing to mimic if profitable",
    ),
    (
        "Firm-4",
        "pair-4",
        1,
        "employer",
        0,
        0,
        "uses education as its only observable evidence",
    )

```

```

    ),
]
return AgentList(
    [
        Agent(
            name=n,
            traits={
                "pair_id": p,
                "turn": t,
                "role": r,
                "productivity": prod,
                "signal_cost": cost,
                "strategy": s,
            },
        )
        for n, p, t, r, prod, cost, s in specs
    ]
)

def run_simulation(
    path: str | Path = "economic-game-signaling.jsonl", model_name="gemini-2.5-flash"
):
    state = SharedState(
        "{{ agent.pair_id }}", FileStateStore(path), game=SharedSignalingGame(60)
    )
    education = QuestionNumerical(
        question_name="education",
        question_text=(
            "You are {{ agent.name }}, a worker with private productivity "
            "{{ agent.productivity }} and cost {{ agent.signal_cost }} per education "
            "unit. You are {{ agent.strategy }}. Choose education 0-3. An employer "
            "observes education but not productivity, then may hire at wage 60."
        ),
        min_value=0,
        max_value=3,
    )
    hiring = QuestionMultipleChoice(
        question_name="hiring",
        question_text=(
            "You are {{ agent.name }} and {{ agent.strategy }}. Worker productivity is "
            "either 100 or 40 with equal prior probability. You observe education "
            "{{ shared_state.game.education }} but not type. Hiring pays productivity "
            "minus wage 60; not hiring pays zero. Choose."
        ),
        question_options=["hire", "do_not_hire"],
    )
    survey = Survey(
        [education, state.game.signal(education), hiring, state.game.decide(hiring)]
    )

```

```

)
survey.add_skip_rule("education", "'{{ agent.role }}" != 'worker')
survey.add_skip_rule("hiring", "'{{ agent.role }}" != 'employer')
terminal = GroupStopCondition("game", "terminal")
schedule = InterviewSchedule.grouped_round_robin(
    "pair_id", "turn", finalize_when=terminal
)
survey.by(participants()).by(Model(model_name)).run(
    interview_schedule=schedule,
    disable_remote_inference=True,
    disable_remote_cache=True,
    cache=False,
    stop_on_exceptions=True,
)
return state

if __name__ == "__main__":
    result = run_simulation()
    for pair in ("pair-1", "pair-2", "pair-3", "pair-4"):
        print(result.render_markdown(scope=pair), "\n")

```

16.13 economic_game_strategic_voting.py

Focus: Voting. Inspect ballot validity, tabulation, tie-breaking, and sincere versus strategic votes.

Compare this complete current program with the preferred construction in the corresponding chapter. Identify which lines express the research design and which exist only to serialize, schedule, authorize state reads, recover execution, or reshape results.

```

"""Plurality voting with a public poll and incentives to desert a trailing candidate."""

from pathlib import Path

from edsl import Agent, AgentList, InterviewSchedule, Model, QuestionRank, Survey
from edsl.jobs.interview_schedule import GroupStopCondition
from edsl.sharedstate import FileStateStore, SharedState, SharedVotingGame

CANDIDATES = ["Alpha", "Beta", "Gamma"]

def voters():
    rankings = [
        "Alpha > Beta > Gamma",
        "Alpha > Beta > Gamma",
        "Alpha > Beta > Gamma",
        "Beta > Gamma > Alpha",
        "Beta > Gamma > Alpha",
    ]

```

```

        "Gamma > Beta > Alpha",
        "Gamma > Beta > Alpha",
    ]
    return AgentList(
        [
            Agent(name=f"Voter-{i}", traits={"true_ranking": r})
            for i, r in enumerate(rankings, 1)
        ]
    )

def run_simulation(
    path: str | Path = "economic-game-strategic-voting.jsonl",
    model_name="gemini-2.5-flash",
):
    state = SharedState(
        "strategic-voting",
        FileStateStore(path),
        election=SharedVotingGame(CANDIDATES, 7),
    )
    ballot = QuestionRank(
        question_name="ballot",
        question_text=(
            "You are {{ agent.name }}. Your true preference is "
            "{{ agent.true_ranking }}. This election uses plurality, so only your "
            "first-ranked candidate receives a vote. A credible poll before your "
            "sealed ballot reports Alpha 3, Beta 2, Gamma 2, and says Gamma is less "
            "likely than Beta to defeat Alpha. Submit the ranking that best advances "
            "your true preferences; strategic voting is allowed."
        ),
        question_options=CANDIDATES,
        num_selections=3,
    )
    schedule = InterviewSchedule.rounds(
        count=1,
        reveal="after_round",
        finalize_when=GroupStopCondition("election", "complete"),
    )
    Survey([ballot, state.election.vote(ballot)]).by(voters()).by(
        Model(model_name)
    ).run(
        interview_schedule=schedule,
        disable_remote_inference=True,
        disable_remote_cache=True,
        cache=False,
        stop_on_exceptions=True,
    )
    return state

```

```
if __name__ == "__main__":
    print(run_simulation().render_markdown())
```

16.14 economic_game_ultimatum.py

Focus: Transfer games. Inspect role assignment, bounded transfers, sequential revelation, and settlement.

Compare this complete current program with the preferred construction in the corresponding chapter. Identify which lines express the research design and which exist only to serialize, schedule, authorize state reads, recover execution, or reshape results.

```
"""Run concurrent ultimatum games and render their event logs as a chat dashboard."""

import argparse
import html
import random
from pathlib import Path
from datetime import datetime

from edsl import Agent, AgentList, InterviewSchedule, Model, QuestionMultipleChoice
from edsl import QuestionNumerical, Survey
from edsl.jobs.interview_schedule import GroupStopCondition
from edsl.sharedstate import (
    Action,
    ConfiguredSharedGame,
    Equals,
    FileStateStore,
    Field,
    Ref,
    Settlement,
    SharedState,
    Subtract,
)

TRAITS_TEMPLATE = """Your behavioral traits use scales from -1 to 1:
- generosity = {{ generosity }}: -1 means maximizing your own payoff without concern for the other
- inequity_aversion = {{ inequity_aversion }}: -1 means readily accepting highly unequal outcomes
Treat intermediate values proportionally and act consistently with them. As a responder, rejection"""

def ultimatum_game(stake=100):
    """Configure an ultimatum game without a game-specific state class."""
    return ConfiguredSharedGame(
        constants={"stake": float(stake)},
        fields={
            "offer": Field.number(minimum=0, maximum=stake),
            "decision": Field.choice(("accept", "reject")),
```

```

    },
    actions={
        "offer": Action(actor="proposer", writes="offer"),
        "respond": Action(
            actor="responder", writes="decision", requires=("offer",)
        ),
    },
    terminal_when_set="decision",
    settlement=Settlement(
        when=Equals(Ref("decision"), "accept"),
        payoffs={
            "proposer": Subtract(Ref("stake"), Ref("offer")),
            "responder": Ref("offer"),
        },
    ),
)

```

```

def players(persona_count: int = 50, seed: int = 20260828):
    """Create reproducible random personas and assign adjacent agents to pairs."""
    if persona_count < 2 or persona_count % 2:
        raise ValueError("persona_count must be an even integer of at least 2")
    rng = random.Random(seed)
    agents = []
    for index in range(persona_count):
        pair_number = index // 2 + 1
        turn = index % 2
        agents.append(
            Agent(
                name=f"Person {index + 1:02d}",
                traits={
                    "generosity": round(rng.uniform(-1, 1), 2),
                    "inequity_aversion": round(rng.uniform(-1, 1), 2),
                    # Operational metadata is omitted by TRAITS_TEMPLATE.
                    "pair_id": f"pair-{pair_number}",
                    "turn": turn,
                    "role": "proposer" if turn == 0 else "responder",
                },
            )
        )
    return AgentList(agents, traits_presentation_template=TRAITS_TEMPLATE)

```

```

def survey(state):
    offer = QuestionNumerical(
        question_name="offer",
        question_text=(
            "You are {{ agent.name }} , the {{ agent.role }} in a $100 ultimatum "
            "game. Current game: {{ shared_state.game }}. "

```



```

        "Choose the dollars offered to the responder."
    ),
    min_value=0,
    max_value=100,
)
decision = QuestionMultipleChoice(
    question_name="decision",
    question_text=(
        "You are {{ agent.name }}, the {{ agent.role }} in a $100 ultimatum "
        "Current game: {{ shared_state.game }}. Accept or reject the recorded "
        "offer based on your preference."
    ),
    question_options=["accept", "reject"],
)
result = Survey(
    [offer, state.game.bind("offer", offer), decision, state.game.bind("respond", decision)]
)
result.add_skip_rule("offer", "'{{ agent.role }}' != 'proposer'")
result.add_skip_rule("decision", "'{{ agent.role }}' != 'responder'")
return result

def run_simulation(
    log_path: str | Path = "economic-game-ultimatum.jsonl",
    model_name="gemini-2.5-flash",
    persona_count: int = 50,
    seed: int = 20260828,
    max_concurrency: int = 10,
):
    state = SharedState(
        "{{ agent.pair_id }}",
        FileStateStore(log_path),
        game=ultimatum_game(stake=100),
    )
    schedule = InterviewSchedule.grouped_round_robin(
        "pair_id", "turn", finalize_when=GroupStopCondition("game", "terminal")
    )
    (
        survey(state)
        .by(players(persona_count, seed))
        .by(Model(model_name))
        .run(
            interview_schedule=schedule,
            disable_remote_inference=True,
            disable_remote_cache=True,
            cache=False,
            stop_on_exceptions=True,
            max_concurrency=max_concurrency,
        )
    )

```

```

)
return state

def read_games(source: SharedState | str | Path):
    """Read analysis-ready game records through SharedState's public read API."""
    state = (
        source
        if isinstance(source, SharedState)
        else SharedState("dashboard", FileStateStore(source), game=ultimatum_game())
    )
    events_by_scope = {scope: [] for scope in state.scopes()}
    for event in state.history(target="game"):
        events_by_scope[event.scope].append(event)
    games = []
    for record in state.records(target="game"):
        proposer = record.get("proposer")
        responder = record.get("responder")
        payoffs = record.get("payoffs") or {}
        decision = record.get("decision")
        games.append(
            record
            | {
                "pair_id": record["scope"],
                "events": events_by_scope[record["scope"]],
                "proposer_payoff": payoffs.get(proposer, 0),
                "responder_payoff": payoffs.get(responder, 0),
                "status": (
                    "Accepted"
                    if decision == "accept"
                    else "Rejected"
                    if decision == "reject"
                    else "Incomplete"
                ),
            }
        )
    return sorted(games, key=lambda game: int(game["pair_id"].split("-")[-1]))

def render_dashboard(
    log_path: str | Path = "economic-game-ultimatum.jsonl",
    output_path: str | Path = "economic-game-ultimatum-dashboard.html",
    model_name: str = "gemini-2.5-flash",
    persona_count: int = 50,
    seed: int = 20260828,
):
    """Create a dependency-free HTML dashboard from a shared-state event log."""
    games = read_games(log_path)
    if not games:

```

```

        raise ValueError(f"No ultimatum-game events found in {log_path}")

def esc(value):
    return html.escape(str(value))

def trait_label(value):
    return f"{value:+.2f}" if isinstance(value, (int, float)) else "unknown"

accepted = sum(game["status"] == "Accepted" for game in games)
average_offer = sum(game.get("offer", 0) for game in games) / len(games)
player_details = {agent.name: agent.traits for agent in players(persona_count, seed)}
cards = []
for index, game in enumerate(games, 1):
    proposer = game.get("proposer", f"P{index}")
    responder = game.get("responder", f"R{index}")
    offer = game.get("offer", 0)
    proposer_share = game["proposer_payoff"]
    responder_share = game["responder_payoff"]
    proposer_traits = player_details.get(proposer, {})
    responder_traits = player_details.get(responder, {})
    proposer_pref = f"generosity {trait_label(proposer_traits.get('generosity'))} · inequity
    responder_pref = f"generosity {trait_label(responder_traits.get('generosity'))} · inequity
    event_times = [event.timestamp for event in game["events"]]
    timing = " → ".join(ts.strftime("%H:%M:%S UTC") for ts in event_times[:2])
    cards.append(f"""
<article class="game-card">
    <div class="game-head"><div><span class="eyebrow">Table {index}</span><h2>{esc(game['p
    <div class="chat">
        <div class="message proposer"><div class="avatar">{esc(proposer)}</div><div class="b
        <div class="message responder"><div class="bubble"><div class="speaker">{esc(respond
    </div>
    <div class="split" aria-label="Payoff split"><div class="p-share" style="width:{propos
    <div class="legend"><span><i class="dot p"></i>{esc(proposer)} payoff</span><span><i c
</article>""")

generated = datetime.now().astimezone().strftime("%B %d, %Y at %H:%M %Z")
document = f"""<!doctype html>
<html lang="en"><head><meta charset="utf-8"><meta name="viewport" content="width=device-width,in
<title>Ultimatum Game · Run Dashboard</title>
<style>
:root{--ink:#17211c;--muted:#66736b;--paper:#f4f0e7;--card:#fffd7;--green:#2d6a4f;--lime:#b7d3
*{{box-sizing:border-box}} body{{margin:0;color:var(--ink);background:var(--paper);font-family:I
header,main,footer{{position:relative;max-width:1100px;margin:auto}} header{{padding:68px 24px 3
.scoreboard{{display:grid;grid-template-columns:repeat(3,1fr);gap:12px;margin-top:30px}} .metric
main{{padding:10px 24px 70px;display:grid;gap:22px}} .game-card{{background:var(--card);border:1
.chat{{margin:24px 0;display:grid;gap:16px}} .message{{display:flex;gap:12px;align-items:end}} .
.split{{display:flex;height:48px;border-radius:14px;overflow:hidden;background:#ddd}} .split div
@media(max-width:620px){{.scoreboard{{grid-template-columns:1fr}} header{{padding-top:42px}} .le
</style></head><body>

```

```

<header><div class="kicker">EDSL · Shared-state field notes</div><h1>{len(games)} offers.<br>{le
<div class="scoreboard"><div class="metric"><strong>{len(games)}</strong><span>games completed</
<main>{' '.join(cards)}</main><footer>Rendered {esc(generated)} from <code>{esc(Path(log_path).na
</body></html>"""
    output = Path(output_path)
    output.write_text(document, encoding="utf-8")
    return output

def parse_args():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--model", default="gemini-2.5-flash", help="EDSL model name")
    parser.add_argument("--log", type=Path, default=Path("economic-game-ultimatum.jsonl"))
    parser.add_argument("--dashboard", type=Path, default=Path("economic-game-ultimatum-dashboard"))
    parser.add_argument("--personas", type=int, default=50, help="even number of personas (default 50)")
    parser.add_argument("--seed", type=int, default=20260828, help="random seed for reproducible results")
    parser.add_argument("--max-concurrency", type=int, default=10, help="maximum simultaneous in-process simulations")
    parser.add_argument("--render-only", action="store_true", help="render the existing log with the dashboard")
    return parser.parse_args()

if __name__ == "__main__":
    args = parse_args()
    if not args.render_only:
        if args.log.exists():
            raise SystemExit(
                f"Refusing to append to existing log {args.log}. Move it or choose a new --log path."
            )
        result = run_simulation(args.log, args.model, args.personas, args.seed, args.max_concurrency)
        for pair_id in (f"pair-{number}" for number in range(1, args.personas // 2 + 1)):
            print(result.render_markdown(scope=pair_id), "\n")
        dashboard = render_dashboard(args.log, args.dashboard, args.model, args.personas, args.seed)
        print(f"Dashboard: {dashboard.resolve()}")

```

16.15 economic_game_voting_rules.py

Focus: Voting. Inspect ballot validity, tabulation, tie-breaking, and sincere versus strategic votes.

Compare this complete current program with the preferred construction in the corresponding chapter. Identify which lines express the research design and which exist only to serialize, schedule, authorize state reads, recover execution, or reshape results.

```

"""Compare plurality, Borda, and Condorcet outcomes on sealed rankings."""

from pathlib import Path

from edsl import Agent, AgentList, InterviewSchedule, Model, QuestionRank, Survey
from edsl.jobs.interview_schedule import GroupStopCondition
from edsl.sharedstate import FileStateStore, SharedState, SharedVotingGame

```

```

CANDIDATES = ["Alpha", "Beta", "Gamma"]

def voters():
    rankings = [
        "Alpha > Beta > Gamma",
        "Alpha > Beta > Gamma",
        "Alpha > Beta > Gamma",
        "Beta > Gamma > Alpha",
        "Beta > Gamma > Alpha",
        "Gamma > Beta > Alpha",
        "Gamma > Beta > Alpha",
    ]
    return AgentList(
        [
            Agent(name=f"Voter-{i}", traits={"true_ranking": ranking})
            for i, ranking in enumerate(rankings, 1)
        ]
    )

def run_simulation(
    path: str | Path = "economic-game-voting-rules.jsonl", model_name="gemini-2.5-flash"
):
    state = SharedState(
        "voting-rules", FileStateStore(path), election=SharedVotingGame(CANDIDATES, 7)
    )
    ranking = QuestionRank(
        question_name="ranking",
        question_text=(
            "You are {{ agent.name }}. Your sincere preference is "
            "{{ agent.true_ranking }}. Submit a complete sincere ranking. Ballots are "
            "sealed and the same profile will be evaluated under plurality, Borda, "
            "and pairwise Condorcet rules."
        ),
        question_options=CANDIDATES,
        num_selections=3,
    )
    schedule = InterviewSchedule.rounds(
        count=1,
        reveal="after_round",
        finalize_when=GroupStopCondition("election", "complete"),
    )
    Survey([ranking, state.election.vote(ranking)]).by(voters()).by(
        Model(model_name)
    ).run(
        interview_schedule=schedule,

```

```

        disable_remote_inference=True,
        disable_remote_cache=True,
        cache=False,
        stop_on_exceptions=True,
    )
    return state

if __name__ == "__main__":
    print(run_simulation().render_markdown())

```

16.16 economic_games_auction_comparison.py

Focus: Auctions. Inspect replaying bids under alternative rules and deterministic tie resolution.

Compare this complete current program with the preferred construction in the corresponding chapter. Identify which lines express the research design and which exist only to serialize, schedule, authorize state reads, recover execution, or reshape results.

```

"""Compare sealed first-price, second-price, and all-pay auctions."""

from pathlib import Path

from edsl import Agent, AgentList, InterviewSchedule, Model, QuestionNumerical, Survey
from edsl.jobs.interview_schedule import GroupStopCondition
from edsl.sharedstate import FileStateStore, SharedSealedAuction, SharedState

def bidders():
    specs = [
        ("Arun", 0, 92, "risk neutral and strategically sophisticated"),
        ("Bea", 1, 76, "moderately risk averse"),
        ("Cole", 2, 61, "risk neutral but cautious about overpaying"),
        ("Dina", 3, 47, "aggressive and competitive"),
        ("Ezra", 4, 33, "highly loss averse"),
    ]
    return AgentList(
        [
            Agent(name=n, traits={"seat": s, "private_value": v, "style": x})
            for n, s, v, x in specs
        ]
    )

DESCRIPTIONS = {
    "first_price": "The highest bidder wins and pays their own bid.",
    "second_price": "The highest bidder wins and pays the second-highest bid.",
    "all_pay": "The highest bidder wins, but every bidder pays their own bid.",
}

```

```

def run_auction(mechanism, path, model):
    state = SharedState(
        mechanism, FileStateStore(path), auction=SharedSealedAuction(mechanism, 5)
    )
    bid = QuestionNumerical(
        question_name="bid",
        question_text=(
            f"You are bidding in a sealed {mechanism.replace('_', '-')} auction. "
            f"{DESCRIPTIONS[mechanism]} Your private value is "
            "{{ agent.private_value }}. You are {{ agent.style }}. Highest bid wins; "
            "ties use a predetermined seat order. Choose a bid from 0 to 100 to "
            "maximize value minus payment."
        ),
        min_value=0,
        max_value=100,
    )
    schedule = InterviewSchedule.rounds(
        count=1,
        reveal="after_round",
        finalize_when=GroupStopCondition("auction", "complete"),
    )
    Survey([bid, state.auction.bid(bid)]).by(bidders()).by(model).run(
        interview_schedule=schedule,
        disable_remote_inference=True,
        disable_remote_cache=True,
        cache=False,
        stop_on_exceptions=True,
    )
    return state

def run_simulations(root: str | Path = ".", model_name="gemini-2.5-flash"):
    root, model = Path(root), Model(model_name)
    return {
        mechanism: run_auction(
            mechanism, root / f"economic-game-auction-{mechanism}.jsonl", model
        )
        for mechanism in DESCRIPTIONS
    }

if __name__ == "__main__":
    for mechanism, state in run_simulations().items():
        print(f"# {mechanism.replace('_', ' ').title()}")
        print(state.render_markdown(), "\n")

```

16.17 economic_games_group.py

Focus: Beauty contest and common pool. Inspect sealed aggregation versus stock-changing state transitions.

Compare this complete current program with the preferred construction in the corresponding chapter. Identify which lines express the research design and which exist only to serialize, schedule, authorize state reads, recover execution, or reshape results.

```
"""Live beauty-contest and common-pool group games."""

from pathlib import Path

from edsl import Agent, AgentList, InterviewSchedule, Model, QuestionNumerical, Survey
from edsl.jobs.interview_schedule import GroupStopCondition
from edsl.sharedstate import (
    FileStateStore,
    SharedBeautyContest,
    SharedCommonPoolGame,
    SharedState,
)

def sealed_schedule(target):
    return InterviewSchedule.rounds(
        count=1,
        within_round="concurrent",
        reveal="after_round",
        finalize_when=GroupStopCondition(target, "complete"),
    )

def run_beauty(path, model):
    styles = [
        ("Ana", "chooses intuitively near the midpoint"),
        ("Bert", "uses one level of strategic reasoning"),
        ("Cleo", "uses two levels of strategic reasoning"),
        ("Dev", "iterates strategic reasoning toward equilibrium"),
        ("Eve", "expects unsophisticated opponents"),
        ("Finn", "expects highly sophisticated opponents"),
    ]
    agents = AgentList([Agent(name=n, traits={"style": s}) for n, s in styles])
    state = SharedState(
        "beauty-contest", FileStateStore(path), game=SharedBeautyContest(6)
    )
    choice = QuestionNumerical(
        question_name="choice",
        question_text=(
            "Six players simultaneously choose a number from 0 to 100. The winner "
            "is closest to two-thirds of the group mean. You are {{ agent.name }} and "
```



```

        "{{ agent.style }}. Choices are sealed. Choose one number."
    ),
    min_value=0,
    max_value=100,
)
Survey([choice, state.game.submit(choice)]).by(agents).by(model).run(
    interview_schedule=sealed_schedule("game"),
    disable_remote_inference=True,
    disable_remote_cache=True,
    cache=False,
    stop_on_exceptions=True,
)
return state

def run_common_pool(path, model):
    norms = [
        ("Gia", "strongly conservation-minded"),
        ("Hugo", "conditionally cooperative"),
        ("Ira", "moderately self-interested"),
        ("Jae", "strictly payoff maximizing"),
        ("Kira", "expects others to over-extract"),
    ]
    agents = AgentList([Agent(name=n, traits={"norm": x}) for n, x in norms])
    state = SharedState(
        "common-pool", FileStateStore(path), game=SharedCommonPoolGame(5, 60, 20)
    )
    amount = QuestionNumerical(
        question_name="extraction",
        question_text=(
            "Five players simultaneously request 0-20 units from a shared stock of "
            "60. You are {{ agent.name }} and are {{ agent.norm }}. If total requests "
            "are at most 60, you receive your request plus one-fifth of the remainder. "
            "If requests exceed 60, the stock is rationed proportionally with no "
            "remainder. Choices are sealed. Choose your request."
        ),
        min_value=0,
        max_value=20,
    )
    Survey([amount, state.game.extract(amount)]).by(agents).by(model).run(
        interview_schedule=sealed_schedule("game"),
        disable_remote_inference=True,
        disable_remote_cache=True,
        cache=False,
        stop_on_exceptions=True,
    )
    return state

```

```

def run_simulations(root: str | Path = ".", model_name="gemini-2.5-flash"):
    root, model = Path(root), Model(model_name)
    return {
        "beauty_contest": run_beauty(
            root / "economic-game-beauty-contest.jsonl", model
        ),
        "common_pool": run_common_pool(root / "economic-game-common-pool.jsonl", model),
    }

if __name__ == "__main__":
    for game_name, state in run_simulations().items():
        print(f"# {game_name.replace('_', ' ').title()}")
        print(state.render_markdown(), "\n")

```

16.18 economic_games_matrix.py

Focus: Matrix games. Inspect seat identity and exhaustive payoff-table validation.

Compare this complete current program with the preferred construction in the corresponding chapter. Identify which lines express the research design and which exist only to serialize, schedule, authorize state reads, recover execution, or reshape results.

```

"""Sealed prisoner's-dilemma and stag-hunt games across parallel pairs."""

from pathlib import Path

from edsl import (
    Agent,
    AgentList,
    InterviewSchedule,
    Model,
    QuestionMultipleChoice,
    Survey,
)
from edsl.jobs.interview_schedule import GroupStopCondition
from edsl.sharedstate import FileStateStore, SharedMatrixGame, SharedState

def players():
    specs = [
        ("A1", "pair-1", 0, "trusting and norm-following"),
        ("B1", "pair-1", 1, "trusting and norm-following"),
        ("A2", "pair-2", 0, "strictly self-interested and strategically cautious"),
        ("B2", "pair-2", 1, "strictly self-interested and strategically cautious"),
        ("A3", "pair-3", 0, "optimistic about cooperation"),
        ("B3", "pair-3", 1, "pessimistic about the other player's cooperation"),
    ]
    return AgentList(

```

```

[
    Agent(
        name=name,
        traits={"pair_id": pair, "seat": seat, "strategy": strategy},
    )
    for name, pair, seat, strategy in specs
]
)

def run_game(name, actions, payoffs, payoff_description, log_path, model_name):
    state = SharedState(
        "{{ agent.pair_id }}",
        FileStateStore(log_path),
        game=SharedMatrixGame(actions, payoffs),
    )
    choice = QuestionMultipleChoice(
        question_name="action",
        question_text=(
            f"You are playing a one-shot sealed {name}. "
            "You are {{ agent.name }} and are {{ agent.strategy }}. "
            f"Payoffs by action profile are: {payoff_description}. "
            "Neither player observes the other's choice before both commit. Choose "
            "the action that best fits your incentives and expectations."
        ),
        question_options=actions,
    )
    survey = Survey([choice, state.game.submit(choice)])
    schedule = InterviewSchedule.rounds(
        count=1,
        group_by="pair_id",
        within_round="concurrent",
        reveal="after_round",
        finalize_when=GroupStopCondition("game", "complete"),
    )
    (
        survey.by(players())
        .by(Model(model_name))
        .run(
            interview_schedule=schedule,
            disable_remote_inference=True,
            disable_remote_cache=True,
            cache=False,
            stop_on_exceptions=True,
        )
    )
    return state

```

```

def run_simulations(root: str | Path = ".", model_name="gemini-2.5-flash"):
    root = Path(root)
    prisoner = run_game(
        "prisoner's dilemma",
        ["cooperate", "defect"],
        {
            "cooperate|cooperate": [3, 3],
            "cooperate|defect": [0, 5],
            "defect|cooperate": [5, 0],
            "defect|defect": [1, 1],
        },
        "C/C=(3,3), C/D=(0,5), D/C=(5,0), D/D=(1,1)",
        root / "economic-game-prisoners-dilemma.jsonl",
        model_name,
    )
    coordination = run_game(
        "stag-hunt coordination game",
        ["stag", "hare"],
        {
            "stag|stag": [4, 4],
            "stag|hare": [0, 3],
            "hare|stag": [3, 0],
            "hare|hare": [3, 3],
        },
        "stag/stag=(4,4), stag/hare=(0,3), hare/stag=(3,0), hare/hare=(3,3)",
        root / "economic-game-stag-hunt.jsonl",
        model_name,
    )
    return {"prisoners_dilemma": prisoner, "stag_hunt": coordination}

if __name__ == "__main__":
    games = run_simulations()
    for game_name, state in games.items():
        print(f"# {game_name.replace('_', ' ').title()}")
        for pair_id in ("pair-1", "pair-2", "pair-3"):
            print(state.render_markdown(scope=pair_id), "\n")

```

16.19 economic_games_transfer.py

Focus: Dictator and trust games. Inspect reuse of transfer primitives without hiding different move sequences.

Compare this complete current program with the preferred construction in the corresponding chapter. Identify which lines express the research design and which exist only to serialize, schedule, authorize state reads, recover execution, or reshape results.

```

"""Live dictator and trust games across independent pairs."""

```

```

from pathlib import Path

from edsl import Agent, AgentList, InterviewSchedule, Model, QuestionNumerical, Survey
from edsl.jobs.interview_schedule import GroupStopCondition
from edsl.sharedstate import (
    FileStateStore,
    SharedDictatorGame,
    SharedState,
    SharedTrustGame,
)

def run_dictator(path, model):
    dictators = AgentList(
        [
            Agent(
                name="D1",
                traits={
                    "pair_id": "pair-1",
                    "recipient": "R1",
                    "norm": "strongly egalitarian",
                },
            ),
            Agent(
                name="D2",
                traits={
                    "pair_id": "pair-2",
                    "recipient": "R2",
                    "norm": "self-interested but dislikes appearing unfair",
                },
            ),
            Agent(
                name="D3",
                traits={
                    "pair_id": "pair-3",
                    "recipient": "R3",
                    "norm": "strictly payoff maximizing",
                },
            ),
        ]
    )
    state = SharedState(
        "{{ agent.pair_id }}", FileStateStore(path), game=SharedDictatorGame(100)
    )
    transfer = QuestionNumerical(
        question_name="transfer",
        question_text=(
            "You are {{ agent.name }} and are {{ agent.norm }}. You unilaterally divide "
            "$100 between yourself and {{ agent.recipient }}, who has no action. Choose "

```

```

        "the dollars transferred to the recipient."
    ),
    min_value=0,
    max_value=100,
)
schedule = InterviewSchedule.rounds(
    count=1,
    group_by="pair_id",
    reveal="after_round",
    finalize_when=GroupStopCondition("game", "complete"),
)
Survey([transfer, state.game.allocate(transfer)]).by(dictators).by(model).run(
    interview_schedule=schedule,
    disable_remote_inference=True,
    disable_remote_cache=True,
    cache=False,
    stop_on_exceptions=True,
)
return state

def trust_players():
    specs = [
        ("S1", "pair-1", 0, "sender", "highly trusting and reciprocal"),
        ("R1", "pair-1", 1, "receiver", "highly trusting and reciprocal"),
        ("S2", "pair-2", 0, "sender", "cautiously prosocial"),
        ("R2", "pair-2", 1, "receiver", "cautiously prosocial"),
        ("S3", "pair-3", 0, "sender", "strictly self-interested"),
        ("R3", "pair-3", 1, "receiver", "strictly self-interested"),
    ]
    return AgentList(
        [
            Agent(name=n, traits={"pair_id": p, "turn": t, "role": r, "norm": norm})
            for n, p, t, r, norm in specs
        ]
    )

def run_trust(path, model):
    state = SharedState(
        "{{ agent.pair_id }}", FileStateStore(path), game=SharedTrustGame(100, 3)
    )
    sent = QuestionNumerical(
        question_name="sent",
        question_text=(
            "You are {{ agent.name }}, the sender, and are {{ agent.norm }}. Choose "
            "$0-$100 to send. The amount is tripled for the receiver, who may return any amount."
        ),
        min_value=0,

```

```

        max_value=100,
    )
    returned = QuestionNumerical(
        question_name="returned",
        question_text=(
            "You are {{ agent.name }}, the receiver, and are {{ agent.norm }}. Current "
            "game: {{ shared_state.game }}. Return between $0 and the displayed "
            "receiver_available amount to the sender."
        ),
        min_value=0,
        max_value=300,
    )
    survey = Survey(
        [sent, state.game.send(sent), returned, state.game.return_funds(returned)]
    )
    survey.add_skip_rule("sent", "'{{ agent.role }}" != 'sender')
    survey.add_skip_rule("returned", "'{{ agent.role }}" != 'receiver')
    schedule = InterviewSchedule.grouped_round_robin(
        "pair_id", "turn", finalize_when=GroupStopCondition("game", "complete")
    )
    survey.by(trust_players()).by(model).run(
        interview_schedule=schedule,
        disable_remote_inference=True,
        disable_remote_cache=True,
        cache=False,
        stop_on_exceptions=True,
    )
    return state

def run_simulations(root: str | Path = ".", model_name="gemini-2.5-flash"):
    root, model = Path(root), Model(model_name)
    return {
        "dictator": run_dictator(root / "economic-game-dictator.jsonl", model),
        "trust": run_trust(root / "economic-game-trust.jsonl", model),
    }

if __name__ == "__main__":
    for game_name, state in run_simulations().items():
        print(f"# {game_name.title()}")
        for pair in ("pair-1", "pair-2", "pair-3"):
            print(state.render_markdown(scope=pair), "\n")

```

16.20 shared_state_adversarial_hiring_committee.py

Focus: Hiring committees. Inspect private assessments, shared evidence, aggregation, and final authority.

Compare this complete current program with the preferred construction in the corresponding chapter. Identify which lines express the research design and which exist only to serialize, schedule, authorize state reads, recover execution, or reshape results.

```
"""Hiring deliberation with conflicting candidates, recusal, and measured persuasion."""
```

```
from pathlib import Path
```

```
from edsl import Agent, AgentList, Model, QuestionFreeText, QuestionRank, Survey
from edsl.sharedstate import FileStateStore, SharedLog, SharedState
```

```
CANDIDATES = {
    "Atlas": "Exceptional technical architect; weak customer exposure and uneven cross-functional",
    "Rowan": "Exceptional enterprise seller; limited technical depth and an aggressive short-term",
    "Morgan": "Strong, broadly acceptable general manager; few standout achievements but no major",
    "Quinn": "Exceptional operator and cost manager; cautious product instincts and limited growth"
}
```

```
SPECS = [
    ("Maya", "CEO", "company-wide leadership and balanced judgment", None),
    ("Eli", "Engineering VP", "technical credibility and durable architecture", None),
    ("Sofia", "Sales VP", "commercial impact and customer trust", "Rowan"),
    ("Priya", "Product VP", "user-centered product judgment and collaboration", None),
    ("Noah", "Finance VP", "operating discipline and scalable economics", None),
]
```

```
def committee(*, voting_only: bool = False) -> AgentList:
    return AgentList(
        [
            Agent(
                name=name,
                traits={
                    "role": role,
                    "priority": priority,
                    "conflict": conflict or "none",
                },
            )
            for name, role, priority, conflict in SPECS
            if not voting_only or conflict is None
        ]
    )
```

```
def ranking_survey(state: SharedState, phase: str) -> Survey:
    dossiers = "\n".join(f"{name}: {summary}" for name, summary in CANDIDATES.items())
    name = f"{phase}_ranking"
    ranking = QuestionRank(
```



```

        question_name=name,
        question_text=(
            "You are {{ agent.name }}, the {{ agent.role }}. Your priority is "
            "{{ agent.priority }}. Your disclosed conflict is {{ agent.conflict }}. "
            f"Privately rank all candidates from strongest to weakest.\n\n{dossiers}"
        ),
        question_options=list(CANDIDATES),
    )
    log = state.initial_rankings if phase == "initial" else state.final_rankings
    return Survey(
        [
            ranking,
            log.append(reviewer="{{ agent.name }}", phase=phase, ranking=ranking),
        ]
    )

def deliberation_survey(state: SharedState) -> Survey:
    dossiers = "\n".join(f"{name}: {summary}" for name, summary in CANDIDATES.items())
    statement = QuestionFreeText(
        question_name="public_statement",
        question_text=(
            "You are {{ agent.name }}, the {{ agent.role }}. Your priority is "
            "{{ agent.priority }} and your disclosed conflict is {{ agent.conflict }}.\n\n"
            f"Candidate dossiers:\n{dossiers}\n\n"
            "Prior public statements:\n{{ shared_state.discussion.entries }}\n\n"
            "Make one concise, decision-relevant statement. If prior statements exist, "
            "explicitly challenge one claim or introduce material evidence not already "
            "raised. Do not merely agree. If you have a conflict, disclose it and avoid "
            "advocating for that candidate."
        ),
    )
    return Survey(
        [
            statement,
            state.discussion.append(speaker="{{ agent.name }}", statement=statement),
        ]
    )

def analyze(state: SharedState) -> tuple[str, dict[str, int], list[str]]:
    initial = {
        entry["reviewer"]: entry["ranking"]
        for entry in state.read().state["initial_rankings"]["entries"]
    }
    final = {
        entry["reviewer"]: entry["ranking"]
        for entry in state.read().state["final_rankings"]["entries"]
    }

```

```

voters = set(final)
scores = {candidate: 0 for candidate in CANDIDATES}
for ranking in final.values():
    for index, candidate in enumerate(ranking):
        scores[candidate] += len(CANDIDATES) - index - 1
changed = [name for name in sorted(voters) if initial[name] != final[name]]
winner = sorted(scores, key=lambda candidate: (-scores[candidate], candidate))[0]
return winner, scores, changed

def run_adversarial_hiring(
    log_path: str | Path = "hiring-committee-adversarial-live.jsonl",
    model_name: str = "gemini-2.5-flash",
) -> tuple[SharedState, str, dict[str, int], list[str]]:
    state = SharedState(
        "adversarial-hiring-committee",
        FileStateStore(log_path),
        initial_rankings=SharedLog(),
        discussion=SharedLog(),
        final_rankings=SharedLog(),
    )
    model = Model(model_name)
    options = {
        "disable_remote_inference": True,
        "disable_remote_cache": True,
        "cache": False,
        "stop_on_exceptions": True,
    }
    ranking_survey(state, "initial").by(committee()).by(model).run(**options)
    deliberation_survey(state).by(committee()).by(model).run(
        interview_schedule="serial", **options
    )
    ranking_survey(state, "final").by(committee(voting_only=True)).by(model).run(
        **options
    )
    winner, scores, changed = analyze(state)
    state.close()
    return state, winner, scores, changed

if __name__ == "__main__":
    shared_state, selected, tally, changed_voters = run_adversarial_hiring()
    print(shared_state.render_markdown())
    print(
        f"\nSelected: {selected}\nBorda tally: {tally}"
        f"\nVoters changing rankings: {changed_voters}"
    )

```

16.21 shared_state_bilateral_negotiation.py

Focus: Negotiation. Inspect tagged actions, alternating turns, visibility, and terminal outcomes.

Compare this complete current program with the preferred construction in the corresponding chapter. Identify which lines express the research design and which exist only to serialize, schedule, authorize state reads, recover execution, or reshape results.

```
"""Five rounds of parallel bilateral negotiations, serial within each pair."""
```

```
from pathlib import Path
```

```
from edsl import (
```

```
    Agent,  
    AgentList,  
    InterviewSchedule,  
    Model,  
    QuestionFreeText,  
    QuestionMultipleChoice,  
    QuestionNumerical,  
    Survey,  
)
```

```
from edsl.sharedstate import FileStateStore, SharedNegotiation, SharedState
```

```
def negotiation_agents() -> AgentList:
```

```
"""Three independent pairs; reservation values remain private traits."""
```

```
    pairs = [
```

```
        ("p1", 115, 72),  
        ("p2", 95, 60),  
        ("p3", 80, 88), # Deliberately has no mutually beneficial price.  
    ]
```

```
    agents = []
```

```
    for pair_id, buyer_value, seller_value in pairs:
```

```
        agents.extend(
```

```
            [
```

```
                Agent(
```

```
                    name=f"Buyer {pair_id}",
```

```
                    traits={
```

```
                        "pair_id": pair_id,
```

```
                        "turn_order": 0,
```

```
                        "role": "buyer",
```

```
                        "private_value": buyer_value,
```

```
                        "objective": (
```

```
                            "Buy only at or below your private maximum value. "
```

```
                            "Seek the lowest credible price."
```

```
                        ),
```

```
                    },
```

```
                ),
```

```
                Agent(
```

```

        name=f"Seller {pair_id}",
        traits={
            "pair_id": pair_id,
            "turn_order": 1,
            "role": "seller",
            "private_value": seller_value,
            "objective": (
                "Sell only at or above your private minimum value. "
                "Seek the highest credible price."
            ),
        },
    },
]
)
return AgentList(agents)

```

```

def build_negotiation(log_path: str | Path) -> tuple[Survey, SharedState]:
    state = SharedState(
        scope="{{ agent.pair_id }}",
        store=FileStateStore(log_path),
        negotiation=SharedNegotiation("used sailboat"),
    )
    context = (
        "You are the {{ agent.role }} negotiating a used sailboat. "
        "Your private reservation value is ${{{ agent.private_value }}}. "
        "{{ agent.objective }} Never reveal your reservation value.\n\n"
        "Your pair's transcript so far:\n{{ shared_state.negotiation.turns }}"
    )
    action = QuestionMultipleChoice(
        question_name="action",
        question_text=context + "\n\nChoose your next action.",
        question_options=["offer", "accept", "reject", "walk away"],
    )
    amount = QuestionNumerical(
        question_name="amount",
        question_text=(
            context
            + "\n\nYou chose {{ action.answer }}. Give the price you offer or accept; "
            "use 0 for reject or walk away."
        ),
        min_value=0,
        max_value=1000,
    )
    message = QuestionFreeText(
        question_name="message",
        question_text=(
            context
            + "\n\nYou chose {{ action.answer }} at ${{{ amount.answer }}}. Write one "

```

```

        "concise message to the other party."
    ),
)
return (
    Survey(
        [
            action,
            amount,
            message,
            state.negotiation.record(action, amount, message),
        ]
    ),
    state,
)

def run_negotiations(
    log_path: str | Path = "bilateral-negotiations.jsonl",
    model_name: str = "gemini-2.5-flash",
) -> SharedState:
    survey, state = build_negotiation(log_path)
    schedule = InterviewSchedule.grouped_round_robin(
        group_by="pair_id",
        order_by="turn_order",
        stop_when=state.negotiation.is_terminal,
    )
    (
        survey.by(negotiation_agents())
        .by(Model(model_name))
        .run(
            n=5,
            interview_schedule=schedule,
            disable_remote_inference=True,
            disable_remote_cache=True,
            cache=False,
            stop_on_exceptions=True,
        )
    )
    return state

if __name__ == "__main__":
    result = run_negotiations()
    for pair_id in ("p1", "p2", "p3"):
        print(result.render_markdown(scope=pair_id))
        print()

```

16.22 shared_state_binary_prediction_market.py

Focus: Prediction markets. Inspect quote formation, belief revision, ordering, and settlement.

Compare this complete current program with the preferred construction in the corresponding chapter. Identify which lines express the research design and which exist only to serialize, schedule, authorize state reads, recover execution, or reshape results.

```
"""A live binary-contract prediction market driven by private-belief agents."""

from pathlib import Path

from edsl import (
    Agent,
    AgentList,
    InterviewSchedule,
    Model,
    QuestionMultipleChoice,
    QuestionNumerical,
    Survey,
)
from edsl.sharedstate import FileStateStore, SharedBinaryMarket, SharedState
from edsl.sharedstate.steps import StepContext

CONTRACT = "Project Atlas reaches 100,000 weekly active users within six months"

def traders() -> AgentList:
    specs = [
        (
            "Aria",
            0,
            0.80,
            "bullish telemetry analyst",
            "trusts strong early usage data",
        ),
        ("Basil", 1, 0.66, "sales forecaster", "sees meaningful customer commitments"),
        (
            "Emi",
            2,
            0.54,
            "balanced market researcher",
            "weighs mixed qualitative evidence",
        ),
        ("Chen", 3, 0.38, "skeptical reliability engineer", "expects scaling failures"),
        ("Dara", 4, 0.22, "conservative risk analyst", "uses a pessimistic base rate"),
    ]
    return AgentList(
        [
```

```

        Agent(
            name=name,
            traits={
                "trader_order": trader_order,
                "private_belief": belief,
                "trader_type": trader_type,
                "evidence": evidence,
            },
        )
    for name, trader_order, belief, trader_type, evidence in specs
]
)

def build_survey(state: SharedState) -> Survey:
    action = QuestionMultipleChoice(
        question_name="market_action",
        question_text=(
            "Round {{ run.round }} of 3. You are {{ agent.name }}, a "
            "{{ agent.trader_type }} who {{ agent.evidence }}. Your private probability "
            "for YES is {{ agent.private_belief }}. Do not reveal it.\n\n"
            "Contract: {{ shared_state.market.contract }}\n"
            "Current YES price: {{ shared_state.market.yes_price }}\n"
            "Current NO price: {{ shared_state.market.no_price }}\n"
            "Your portfolio: {{ shared_state.market.your_portfolio }}\n"
            "Recent trades: {{ shared_state.market.recent_trades }}\n\n"
            "Choose buy_yes when YES is underpriced relative to your belief, buy_no "
            "when NO is underpriced, or hold when neither trade has positive value."
        ),
        question_options=["buy_yes", "buy_no", "hold"],
    )
    quantity = QuestionNumerical(
        question_name="trade_quantity",
        question_text=(
            "You chose {{ market_action.answer }}. Choose a quantity from 0 to 15 "
            "shares. Use 0 when holding. Size the trade according to your perceived "
            "edge while preserving cash for later rounds."
        ),
        min_value=0,
        max_value=15,
    )
    return Survey([action, quantity, state.market.trade(action, quantity)])

def run_market(
    log_path: str | Path = "binary-prediction-market.jsonl",
    model_name: str = "gemini-2.5-flash",
) -> SharedState:
    state = SharedState(

```

```

        "atlas-binary-market",
        FileStateStore(log_path),
        market=SharedBinaryMarket(CONTRACT, liquidity=40, initial_cash=100),
    )
    schedule = InterviewSchedule.rounds(
        count=3,
        within_round="serial",
        state_visibility="live",
        order_by="trader_order",
        round_order="rotate",
    )
    (
        build_survey(state)
        .by(traders())
        .by(Model(model_name))
        .run(
            interview_schedule=schedule,
            disable_remote_inference=True,
            disable_remote_cache=True,
            cache=False,
            stop_on_exceptions=True,
        )
    )
    # Resolve to YES after trading; agents never receive this outcome in their prompts.
    state.market.settle(True).execute(StepContext({}, "market-resolution"))
    state.close()
    return state

if __name__ == "__main__":
    print(run_market().render_markdown())

```

16.23 shared_state_budget_allocation.py

Focus: Resource allocation. Inspect conservation, conflicting requests, priority, and auditable allocation.

Compare this complete current program with the preferred construction in the corresponding chapter. Identify which lines express the research design and which exist only to serialize, schedule, authorize state reads, recover execution, or reshape results.

```

"""Live allocation from a finite shared civic budget."""

from pathlib import Path

from edsl import Agent, AgentList, InterviewSchedule, Model, QuestionMultipleChoice
from edsl import QuestionNumerical, Survey
from edsl.jobs.interview_schedule import GroupStopCondition
from edsl.sharedstate import FileStateStore, SharedBudgetPool, SharedState

```



```

PROJECTS = {
    "Cooling centers": "heat-wave shelters in underserved neighborhoods",
    "Library hours": "evening and weekend access at branch libraries",
    "Bike safety": "protected intersections on high-injury corridors",
    "Youth arts": "free after-school music and theater programs",
}

def delegates() -> AgentList:
    specs = [
        ("Inez", 0, "public health", "Cooling centers"),
        ("Jamal", 1, "education access", "Library hours"),
        ("Keiko", 2, "street safety", "Bike safety"),
        ("Luis", 3, "youth development", "Youth arts"),
        ("Mara", 4, "climate resilience", "Cooling centers"),
        ("Noah", 5, "neighborhood equity", "Library hours"),
    ]
    return AgentList(
        [
            Agent(
                name=name,
                traits={"seat": seat, "priority": priority, "favorite": favorite},
            )
            for name, seat, priority, favorite in specs
        ]
    )

def build_survey(state: SharedState) -> Survey:
    project = QuestionMultipleChoice(
        question_name="project",
        question_text=(
            "Round {{ run.round }}. You are {{ agent.name }}, representing "
            "{{ agent.priority }}; your preferred project is {{ agent.favorite }}.\n\n"
            "Shared budget: {{ shared_state.budget }}\n\n"
            "Choose one project for your next funding request. React to allocations "
            "already made and diversify if another need is now more urgent."
        ),
        question_options=list(PROJECTS),
    )
    amount = QuestionNumerical(
        question_name="amount",
        question_text=(
            "Request between $0 and $20 for {{ project.answer }}. Only "
            "{{ shared_state.budget.remaining }} remains. The grant will be partially "
            "filled if your request exceeds the remaining shared budget."
        ),
    ),

```

```

        min_value=0,
        max_value=20,
    )
    return Survey([project, amount, state.budget.fund(project, amount)])

def run_simulation(
    log_path: str | Path = "budget-allocation.jsonl",
    model_name: str = "gemini-2.5-flash",
) -> SharedState:
    state = SharedState(
        "city-mini-budget",
        FileStateStore(log_path),
        budget=SharedBudgetPool(75, PROJECTS),
    )
    schedule = InterviewSchedule.rounds(
        count=2,
        within_round="serial",
        state_visibility="live",
        order_by="seat",
        round_order="rotate",
        stop_when=GroupStopCondition("budget", "exhausted"),
    )
    (
        build_survey(state)
        .by(delegates())
        .by(Model(model_name))
        .run(
            interview_schedule=schedule,
            disable_remote_inference=True,
            disable_remote_cache=True,
            cache=False,
            stop_on_exceptions=True,
        )
    )
    state.close()
    return state

if __name__ == "__main__":
    print(run_simulation().render_markdown())

```

16.24 shared_state_coalition_formation.py

Focus: Coalition formation. Inspect membership, consent, exclusivity, and overlapping proposals.

Compare this complete current program with the preferred construction in the corresponding chapter. Identify which lines express the research design and which exist only to serialize, schedule, authorize state reads, recover execution, or reshape results.

```
"""Live capacity-constrained coalition formation with private preferences."""
```

```
from pathlib import Path
```

```
from edsl import (
```

```
    Agent,
```

```
    AgentList,
```

```
    InterviewSchedule,
```

```
    Model,
```

```
    QuestionMultipleChoice,
```

```
    Survey,
```

```
)
```

```
from edsl.sharedstate import FileStateStore, SharedCoalitionPool, SharedState
```

```
COALITIONS = {
```

```
    "Growth": {
```

```
        "platform": "maximize adoption through an ambitious public launch",
```

```
        "capacity": 2,
```

```
    },
```

```
    "Safety": {
```

```
        "platform": "delay launch until reliability and safeguards improve",
```

```
        "capacity": 2,
```

```
    },
```

```
    "Bridge": {
```

```
        "platform": "run a limited pilot with staged safety checkpoints",
```

```
        "capacity": 2,
```

```
    },
```

```
}
```

```
def participants() -> AgentList:
```

```
    specs = [
```

```
        ("Amina", 0, "Growth > Bridge > Safety", "rapid adoption"),
```

```
        ("Ben", 1, "Growth > Bridge > Safety", "commercial momentum"),
```

```
        ("Clara", 2, "Growth > Bridge > Safety", "market leadership"),
```

```
        ("Diego", 3, "Safety > Bridge > Growth", "system reliability"),
```

```
        ("Elena", 4, "Safety > Bridge > Growth", "public accountability"),
```

```
        ("Farah", 5, "Safety > Bridge > Growth", "risk reduction"),
```

```
        ("Gus", 6, "Bridge > Growth > Safety", "pragmatic compromise"),
```

```
    ]
```

```
    return AgentList(
```

```
        [
```

```
            Agent(
```

```
                name=name,
```

```
                traits={
```

```
                    "seat": seat,
```

```
                    "private_ranking": ranking,
```

```
                    "motivation": motivation,
```

```

        },
    )
    for name, seat, ranking, motivation in specs
]
)

def build_survey(state: SharedState) -> Survey:
    choice = QuestionMultipleChoice(
        question_name="coalition_choice",
        question_text=(
            "Round {{ run.round }} of 2. You are {{ agent.name }} and prioritize "
            "{{ agent.motivation }}. Your private ranking is "
            "{{ agent.private_ranking }}.\n\n"
            "Current coalition state: {{ shared_state.coalitions.coalitions }}\n"
            "Your membership: {{ shared_state.coalitions.your_membership }}\n"
            "Your previous request: {{ shared_state.coalitions.your_last_request }}\n"
            "Recent requests: {{ shared_state.coalitions.recent_requests }}\n\n"
            "Choose the coalition you now want to join. A full coalition rejects "
            "the request atomically and leaves your existing membership unchanged. "
            "Respond strategically to remaining capacity and prior rejections."
        ),
        question_options=list(COALITIONS),
    )
    return Survey([choice, state.coalitions.request(choice)])

def run_simulation(
    log_path: str | Path = "coalition-formation.jsonl",
    model_name: str = "gemini-2.5-flash",
) -> SharedState:
    state = SharedState(
        "launch-coalitions",
        FileStateStore(log_path),
        coalitions=SharedCoalitionPool(COALITIONS),
    )
    schedule = InterviewSchedule.rounds(
        count=2,
        within_round="serial",
        state_visibility="live",
        order_by="seat",
        round_order="rotate",
    )
    (
        build_survey(state)
        .by(participants())
        .by(Model(model_name))
        .run(
            interview_schedule=schedule,

```

```

        disable_remote_inference=True,
        disable_remote_cache=True,
        cache=False,
        stop_on_exceptions=True,
    )
)
state.close()
return state

if __name__ == "__main__":
    print(run_simulation().render_markdown())

```

16.25 shared_state_congested_matching_market.py

Focus: Matching markets. Inspect preferences, capacity, conflicts, and algorithmic tie-breaking.

Compare this complete current program with the preferred construction in the corresponding chapter. Identify which lines express the research design and which exist only to serialize, schedule, authorize state reads, recover execution, or reshape results.

```

"""A congested matching market that exercises deferred-acceptance rejection chains."""

import json

from pathlib import Path

from edsl import Agent, AgentList, Model, QuestionRank, Survey
from edsl.sharedstate import FileStateStore, SharedDeferredAcceptance, SharedState

PROGRAMS = {
    "Northstar": "Traditional quantitative public policy with a theoretical core.",
    "Lakeside": "Environmental policy with fieldwork and a small rural cohort.",
    "CivicLab": "Technology governance with applied AI projects and city internships.",
}

CAPACITIES = {"Northstar": 2, "Lakeside": 2, "CivicLab": 2}
PRIORITIES = {
    "Northstar": ["Diego", "Amina", "Evan", "Chloe", "Ben", "Farah"],
    "Lakeside": ["Chloe", "Evan", "Amina", "Farah", "Diego", "Ben"],
    "CivicLab": ["Farah", "Ben", "Amina", "Diego", "Chloe", "Evan"],
}

def applicants() -> AgentList:
    specs = [
        ("Amina", "AI regulation", "CivicLab", "Northstar", "Lakeside"),
        ("Ben", "civic technology", "CivicLab", "Northstar", "Lakeside"),
        (

```

```

        "Chloe",
        "digital environmental governance",
        "CivicLab",
        "Lakeside",
        "Northstar",
    ),
    ("Diego", "public-sector data science", "CivicLab", "Northstar", "Lakeside"),
    ("Evan", "environmental justice", "Lakeside", "Northstar", "CivicLab"),
    ("Farah", "algorithmic accountability", "CivicLab", "Northstar", "Lakeside"),
]
return AgentList(
    [
        Agent(
            name=name,
            traits={
                "interest": interest,
                "ideal": first,
                "fallback": second,
                "last_choice": third,
            },
        )
        for name, interest, first, second, third in specs
    ]
)

def preference_survey(state: SharedState) -> Survey:
    descriptions = "\n".join(
        f"{program}: {description}" for program, description in PROGRAMS.items()
    )
    ranking = QuestionRank(
        question_name="program_ranking",
        question_text=(
            "You are {{ agent.name }}, focused on {{ agent.interest }}. After careful "
            "research, your genuine preference is {{ agent.ideal }} first, "
            "{{ agent.fallback }} second, and {{ agent.last_choice }} third. Submit "
            "that complete private ranking; do not strategize about admissions.\n\n"
            f"Program descriptions:\n{descriptions}"
        ),
        question_options=list(PROGRAMS),
    )
    return Survey([ranking, state.market.collect(ranking)])

def blocking_pairs(state: SharedState, log_path: str | Path) -> list[tuple[str, str]]:
    requests = {}
    for line in Path(log_path).read_text().splitlines():
        event = json.loads(line)
        if event.get("op") == "collect":

```

```

        requests[event["args"]["student"]] = event["args"]["ranking"]
market = state.read().state["market"]
matches = market["matches"]
institution_matches = market["institution_matches"]
priority_rank = {
    institution: {student: rank for rank, student in enumerate(order)}
    for institution, order in PRIORITIES.items()
}
blocks = []
for student, ranking in requests.items():
    assigned = matches.get(student)
    for institution in ranking[: ranking.index(assigned)]:
        incumbents = institution_matches[institution]
        if len(incumbents) < CAPACITIES[institution] or any(
            priority_rank[institution][student]
            < priority_rank[institution][incumbent]
            for incumbent in incumbents
        ):
            blocks.append((student, institution))
return blocks

def run_congested_market(
    log_path: str | Path = "matching-market-congested.jsonl",
    model_name: str = "gemini-2.5-flash",
) -> tuple[SharedState, list[tuple[str, str]]]:
    state = SharedState(
        "congested-program-match",
        FileStateStore(log_path),
        market=SharedDeferredAcceptance(CAPACITIES, PRIORITIES),
    )
    (
        preference_survey(state)
        .by(applicants())
        .by(Model(model_name))
        .run(
            disable_remote_inference=True,
            disable_remote_cache=True,
            cache=False,
            stop_on_exceptions=True,
        )
    )
    state.close()
    return state, blocking_pairs(state, log_path)

if __name__ == "__main__":
    shared_state, blocks = run_congested_market()
    print(shared_state.render_markdown())

```

```
print(f"\nBlocking pairs: {blocks}")
```

16.26 shared_state_customer_feedback_synthesis.py

Focus: Multi-phase synthesis. Inspect phase boundaries, evidence provenance, and completed-input reads.

Compare this complete current program with the preferred construction in the corresponding chapter. Identify which lines express the research design and which exist only to serialize, schedule, authorize state reads, recover execution, or reshape results.

```
"""Evidence-backed, multi-role synthesis of a customer-feedback CSV."""

import csv
from pathlib import Path

from edsl import (
    Agent,
    AgentList,
    InterviewSchedule,
    Model,
    QuestionDict,
    QuestionList,
    QuestionMatrix,
    Survey,
)
from edsl.sharedstate import FileStateStore, SharedLog, SharedState

HERE = Path(__file__).resolve().parent
FEEDBACK_PATH = HERE / "customer_feedback_sample.csv"

def load_feedback(path: str | Path = FEEDBACK_PATH) -> list[dict[str, str]]:
    with Path(path).open(newline="") as handle:
        return list(csv.DictReader(handle))

def reviewers() -> AgentList:
    specs = [
        ("Priya", "product manager", "product gaps, frequency, and roadmap impact"),
        (
            "Marcus",
            "customer support lead",
            "customer pain, urgency, and support burden",
        ),
        ("Elena", "UX researcher", "usability, accessibility, and user context"),
        (
            "Noah",
```



```

        "business analyst",
        "segment patterns, retention risk, and evidence strength",
    ),
]
return AgentList(
    [
        Agent(name=name, traits={"role": role, "lens": lens})
        for name, role, lens in specs
    ]
)

def formatted_feedback(feedback: list[dict[str, str]]) -> str:
    return "\n".join(
        f"{row['comment_id']} [{row['segment']}] : {row['comment']}" for row in feedback
    )

def discovery_survey(state: SharedState, feedback: list[dict[str, str]]) -> Survey:
    proposal = QuestionDict(
        question_name="theme_proposal",
        question_text=(
            "Round {{ run.round }}. You are {{ agent.name }}, the {{ agent.role }}, "
            "focused on {{ agent.lens }}.\n\nCustomer comments:\n"
            f"{formatted_feedback(feedback)}\n\nThemes already proposed:\n"
            "{{ shared_state.proposals.entries }}\n\nPropose one important theme not "
            "already adequately represented. Every claim must cite exact comment IDs."
        ),
        answer_keys=["theme", "evidence_ids", "interpretation", "recommended_action"],
        value_types=["str", "list", "str", "str"],
        value_descriptions=[
            "Short, neutral theme name",
            "List of supporting comment IDs such as C03",
            "What the cited comments collectively show",
            "A concrete company action",
        ],
    ),
)
return Survey(
    [
        proposal,
        state.proposals.append(
            analyst="{{ agent.name }}", round="{{ run.round }}", proposal=proposal
        ),
    ]
)

def synthesis_survey(state: SharedState, feedback: list[dict[str, str]]) -> Survey:
    synthesis = QuestionList(

```

```

question_name="canonical_theme_names",
question_text=(
    "You are an impartial research lead. Consolidate overlapping proposals "
    "into exactly five decision-useful themes. Preserve minority observations, "
    "do not invent prevalence, and use only valid evidence IDs.\n\nComments:\n"
    f"{formatted_feedback(feedback)}\n\nAnalyst proposals:\n"
    f"{{ shared_state.proposals.entries }}\n\nReturn exactly five short, distinct "
    "canonical theme names."
),
min_list_items=5,
max_list_items=5,
)
return Survey([synthesis, state.synthesis.append(names=synthesis)])

def theme_editors(names: list[str]) -> AgentList:
    return AgentList(
        [
            Agent(
                name=f"Theme editor {index}",
                traits={"theme_order": index, "theme_name": name},
            )
            for index, name in enumerate(names, 1)
        ]
    )

def theme_detail_survey(state: SharedState, feedback: list[dict[str, str]]) -> Survey:
    detail = QuestionDict(
        question_name="theme_detail",
        question_text=(
            "Develop only the canonical theme '{{ agent.theme_name }}'. Use the "
            "original comments and analyst proposals. Every finding must cite exact "
            "comment IDs; do not invent frequency.\n\nComments:\n"
            f"{formatted_feedback(feedback)}\n\nProposals:\n"
            f"{{ shared_state.proposals.entries }}"
        ),
        answer_keys=["evidence_ids", "finding", "action"],
        value_types=["list", "str", "str"],
        value_descriptions=[
            "Supporting comment IDs",
            "Concise evidence-grounded finding",
            "Concrete recommended company action",
        ],
    )
    return Survey(
        [
            detail,
            state.theme_details.append(

```

```

        editor_order="{{ agent.theme_order }}",
        theme_name="{{ agent.theme_name }}",
        detail=detail,
    ),
]
)

def canonical_themes(state: SharedState) -> list[dict]:
    entries = state.read().state["theme_details"]["entries"]
    if len(entries) < 5:
        raise ValueError("synthesis requires five persisted theme details")
    latest = {int(entry["editor_order"]): entry for entry in entries}
    return [
        {
            "id": f"T{order}",
            "name": latest[order]["theme_name"],
            **latest[order]["detail"],
        }
        for order in range(1, 6)
    ]

def prioritization_survey(state: SharedState, themes: list[dict]) -> Survey:
    theme_ids = [theme["id"] for theme in themes]
    slate = "\n".join(
        f"{theme['id']}: {theme['name']} - {theme['finding']} Evidence: {theme['evidence_ids']}"
        for theme in themes
    )
    vote = QuestionMatrix(
        question_name="priority_votes",
        question_text=(
            "You are {{ agent.name }}, the {{ agent.role }}, focused on "
            f"{{{ agent.lens }}}. Rate every consolidated theme.\n\n{slate}\n\n"
            "Use high for an urgent next-quarter priority, medium for planned work, "
            "and low for monitoring or later action."
        ),
        question_items=theme_ids,
        question_options=["high", "medium", "low"],
        randomize_items=True,
    )
    return Survey([vote, state.priorities.append(voter="{{ agent.name }}", votes=vote)])

def score_priorities(state: SharedState, themes: list[dict]) -> list[dict]:
    scores = {theme["id"]: 0 for theme in themes}
    weights = {"high": 2, "medium": 1, "low": 0}
    for entry in state.read().state["priorities"]["entries"]:
        for theme_id, vote in entry["votes"].items():

```

```

        scores[theme_id] += weights[vote]
    return sorted(
        (dict(theme) | {"priority_score": scores[theme["id"]]}) for theme in themes),
        key=lambda theme: (-theme["priority_score"], theme["id"]),
    )

def run_customer_feedback_synthesis(
    log_path: str | Path = "customer-feedback-synthesis-live.jsonl",
    model_name: str = "gemini-2.5-flash",
) -> tuple[SharedState, list[dict]]:
    feedback = load_feedback()
    state = SharedState(
        "customer-feedback-synthesis",
        FileStateStore(log_path),
        proposals=SharedLog(),
        synthesis=SharedLog(),
        theme_details=SharedLog(),
        priorities=SharedLog(),
    )
    model = Model(model_name)
    options = {
        "disable_remote_inference": True,
        "disable_remote_cache": True,
        "cache": False,
        "stop_on_exceptions": True,
    }
    proposal_count = state.read().state["proposals"]["count"]
    if proposal_count < 4:
        discovery_survey(state, feedback).by(reviewers()).by(model).run(
            interview_schedule=InterviewSchedule.rounds(
                count=2,
                within_round="concurrent",
                state_visibility="snapshot",
                round_order="rotate",
            ),
            **options,
        )
    proposal_count = state.read().state["proposals"]["count"]
    if proposal_count < 4:
        raise RuntimeError(
            f"discovery incomplete: expected at least 4 persisted proposals, got {proposal_count}"
        )
    facilitator = Agent(
        name="Ruth",
        traits={"role": "research synthesis lead"},
    )
    if state.read().state["synthesis"]["count"] == 0:
        synthesis_survey(state, feedback).by(facilitator).by(model).run(**options)

```

```

if state.read().state["synthesis"]["count"] == 0:
    raise RuntimeError("consolidation incomplete: no synthesis was persisted")
names = state.read().state["synthesis"]["entries"][-1]["names"]
if state.read().state["theme_details"]["count"] < 5:
    theme_detail_survey(state, feedback).by(theme_editors(names)).by(model).run(
        **options
    )
detail_count = state.read().state["theme_details"]["count"]
if detail_count < 5:
    raise RuntimeError(
        f"theme detailing incomplete: expected 5 persisted details, got {detail_count}"
    )
themes = canonical_themes(state)
if state.read().state["priorities"]["count"] < len(reviewers()):
    prioritization_survey(state, themes).by(reviewers()).by(model).run(**options)
ballot_count = state.read().state["priorities"]["count"]
if ballot_count < len(reviewers()):
    raise RuntimeError(
        f"prioritization incomplete: expected 4 persisted ballots, got {ballot_count}"
    )
ranked = score_priorities(state, themes)
state.close()
return state, ranked

if __name__ == "__main__":
    shared_state, ranked_themes = run_customer_feedback_synthesis()
    print(shared_state.render_markdown())
    print("\nPrioritized themes:")
    for theme in ranked_themes:
        print(
            f"- {theme['name']} ({theme['priority_score']}/8): "
            f"{theme['finding']} → {theme['action']}"
        )

```

16.27 shared_state_delphi_forecast.py

Focus: Delphi panels. Inspect anonymity, revision, convergence, and forecast history.

Compare this complete current program with the preferred construction in the corresponding chapter. Identify which lines express the research design and which exist only to serialize, schedule, authorize state reads, recover execution, or reshape results.

```

"""Anonymous Delphi forecasting with facilitator feedback and convergence stopping."""

from pathlib import Path

from edsl import (
    Agent,

```

```

    AgentList,
    InterviewSchedule,
    Model,
    QuestionFreeText,
    QuestionNumerical,
    Survey,
)
from edsl.sharedstate import FileStateStore, SharedDelphiPanel, SharedLog, SharedState

QUESTION = (
    "What is the probability that the enterprise permissions product will launch "
    "by September 30 with all committed security and compliance requirements?"
)
MAX_ROUNDS = 5

def experts() -> AgentList:
    specs = [
        ("Amara", "program director", 68, "cross-team milestones and dependency risk"),
        (
            "Ben",
            "staff engineer",
            46,
            "technical scope, integration work, and reliability",
        ),
        (
            "Carmen",
            "enterprise sales lead",
            80,
            "customer commitments and commercial urgency",
        ),
        (
            "Dev",
            "security lead",
            37,
            "threat modeling, audit findings, and approval gates",
        ),
        (
            "Elise",
            "finance partner",
            56,
            "staffing capacity, historical delivery rates, and cost risk",
        ),
        (
            "Farid",
            "customer-success lead",
            64,
            "implementation readiness and customer acceptance",
        ),
    ]

```

```

    ),
]
return AgentList(
    [
        Agent(
            name=name,
            traits={
                "role": role,
                "private_estimate": estimate,
                "evidence_lens": evidence_lens,
            },
        )
        for name, role, estimate, evidence_lens in specs
    ]
)

def expert_survey(state: SharedState, round_number: int) -> Survey:
    estimate = QuestionNumerical(
        question_name="delphi_estimate",
        question_text=(
            f"Delphi round {round_number} of at most {MAX_ROUNDS}. You are an anonymous "
            f"{{ agent.role }} focused on {{ agent.evidence_lens }}.\n\n"
            f"Forecast question: {QUESTION}\n\nYour private starting evidence implies "
            f"{{ agent.private_estimate }}%. Prior-round anonymous panel statistics and "
            f"rationales:\n{{ shared_state.panel }}\n\nFacilitator summaries from "
            f"completed rounds:\n{{ shared_state.feedback.entries }}\n\nGive your independent "
            f"best estimate from 0 to 100. Revise only when the anonymous evidence is "
            f"persuasive; do not move merely to agree with the group."
        ),
        min_value=0,
        max_value=100,
    )
    confidence = QuestionNumerical(
        question_name="delphi_confidence",
        question_text=(
            "You estimated {{ delphi_estimate.answer }}%. Rate confidence from 0 to "
            "100 based on evidence quality, not closeness to consensus."
        ),
        min_value=0,
        max_value=100,
    )
    rationale = QuestionFreeText(
        question_name="delphi_rationale",
        question_text=(
            "Explain the two most decision-relevant reasons for your estimate and one "
            "fact that would materially change it. Do not identify yourself. At most "
            "85 words."
        ),
    )

```

```

    )
    return Survey(
        [
            estimate,
            confidence,
            rationale,
            state.panel.submit(
                estimate,
                confidence,
                rationale,
                round_number=round_number,
            ),
        ]
    )

def facilitator_survey(state: SharedState, round_number: int) -> Survey:
    feedback = QuestionFreeText(
        question_name="anonymous_feedback",
        question_text=(
            f"You are the neutral facilitator after Delphi round {round_number}. The "
            "panel view contains no expert names. Synthesize rather than advocate.\n\n"
            f"Forecast question: {QUESTION}\n\nAnonymous panel:\n"
            f"{{ shared_state.panel }}\n\nReturn at most 140 words with four labeled "
            "sections: CONSENSUS, HIGHER, LOWER, and UNRESOLVED. Preserve minority "
            "arguments and never identify or infer an expert."
        ),
    )
    return Survey(
        [feedback, state.feedback.append(round=round_number, feedback=feedback)]
    )

def round_complete(state: SharedState, round_number: int) -> bool:
    return any(
        summary["round"] == round_number and summary["complete"]
        for summary in state.read().state["panel"]["rounds"]
    )

def feedback_complete(state: SharedState, round_number: int) -> bool:
    return any(
        entry["round"] == round_number
        for entry in state.read().state["feedback"]["entries"]
    )

def final_report(state: SharedState) -> dict:
    panel = state.read().state["panel"]

```



```

rounds = [item for item in panel["rounds"] if item["complete"]]
first, final = rounds[0], rounds[-1]
return {
    "rounds_completed": len(rounds),
    "converged": panel["converged"],
    "initial_median": first["median"],
    "final_median": final["median"],
    "initial_range": first["range"],
    "final_range": final["range"],
    "final_weighted_mean": final["confidence_weighted_mean"],
    "final_anonymous_estimates": sorted(
        item["estimate"] for item in final["anonymous_rationales"]
    ),
}

def run_delphi(
    log_path: str | Path = "delphi-enterprise-launch.jsonl",
    model_name: str = "gemini-2.5-flash",
) -> tuple[SharedState, dict]:
    panelists = experts()
    state = SharedState(
        "enterprise-launch-delphi",
        FileStateStore(log_path),
        panel=SharedDelphiPanel(
            panel_size=len(panelists),
            range_threshold=18,
            median_shift_threshold=4,
            min_rounds=2,
        ),
        feedback=SharedLog(),
    )
    model = Model(model_name)
    options = {
        "disable_remote_inference": True,
        "disable_remote_cache": True,
        "cache": False,
        "stop_on_exceptions": True,
    }
    facilitator = Agent(name="Anonymous Delphi facilitator")
    for round_number in range(1, MAX_ROUNDS + 1):
        if not round_complete(state, round_number):
            expert_survey(state, round_number).by(panelists).by(model).run(
                interview_schedule=InterviewSchedule.rounds(
                    count=1,
                    within_round="concurrent",
                    state_visibility="snapshot",
                ),
                **options,

```

```

    )
    if not round_complete(state, round_number):
        raise RuntimeError(
            f"Delphi round {round_number} did not persist all responses"
        )
    if not feedback_complete(state, round_number):
        facilitator_survey(state, round_number).by(facilitator).by(model).run(
            **options
        )
    if not feedback_complete(state, round_number):
        raise RuntimeError(
            f"Delphi round {round_number} facilitator feedback was not persisted"
        )
    if state.read().state["panel"]["converged"]:
        break
    report = final_report(state)
    state.close()
    return state, report

if __name__ == "__main__":
    shared_state, report = run_delphi()
    print(shared_state.render_markdown())
    print(f"\nFinal Delphi report: {report}")

```

16.28 shared_state_disaster_response.py

Focus: Resource allocation. Inspect conservation, conflicting requests, priority, and auditable allocation.

Compare this complete current program with the preferred construction in the corresponding chapter. Identify which lines express the research design and which exist only to serialize, schedule, authorize state reads, recover execution, or reshape results.

```

"""Two-wave disaster response with atomic, capability-constrained resources."""

from pathlib import Path

from edsl import Agent, AgentList, Model, QuestionMultipleChoice, Survey
from edsl.sharedstate import FileStateStore, SharedResourceBoard, SharedState

INCIDENTS = [
    {
        "id": "I1",
        "round": 1,
        "severity": 5,
        "capability": "fire",
        "description": "warehouse fire",
    }

```

```

    },
    {
      "id": "I2",
      "round": 1,
      "severity": 4,
      "capability": "medical",
      "description": "multi-vehicle bus crash",
    },
    {
      "id": "I3",
      "round": 1,
      "severity": 3,
      "capability": "utility",
      "description": "downed distribution line",
    },
    {
      "id": "I4",
      "round": 2,
      "severity": 5,
      "capability": "utility",
      "description": "hospital backup-generator failure",
    },
    {
      "id": "I5",
      "round": 2,
      "severity": 4,
      "capability": "security",
      "description": "urgent neighborhood evacuation",
    },
    {
      "id": "I6",
      "round": 2,
      "severity": 3,
      "capability": "fire",
      "description": "brush fire near homes",
    },
  ],
  RESOURCES = {
    "Engine-7": "fire",
    "Ambulance-3": "medical",
    "Grid-Crew-2": "utility",
    "Patrol-5": "security",
  }
}

def responders() -> AgentList:
  specs = [
    ("Chief Rivera", "fire incident commander", "Engine-7", "fire"),
    ("Dr. Chen", "EMS medical director", "Ambulance-3", "medical"),
  ]

```

```

        ("Sam Okafor", "utility dispatcher", "Grid-Crew-2", "utility"),
        ("Captain Lewis", "police watch commander", "Patrol-5", "security"),
    ]
    return AgentList(
        [
            Agent(
                name=name,
                traits={
                    "role": role,
                    "resource": resource,
                    "capability": capability,
                },
            )
            for name, role, resource, capability in specs
        ]
    )

def wave_survey(state: SharedState, wave: int) -> Survey:
    wave_incidents = [item for item in INCIDENTS if item["round"] == wave]
    incident_text = "\n".join(
        f"{item['id']}: severity {item['severity']}, requires {item['capability']} - {item['desc']}"
        for item in wave_incidents
    )
    incident = QuestionMultipleChoice(
        question_name=f"wave_{wave}_incident",
        question_text=(
            f"Disaster-response wave {wave}. You are {{ agent.name }}, the "
            f"{{ agent.role }}, controlling {{ agent.resource }} with capability "
            f"{{ agent.capability }}.\n\nNew incidents:\n"
            f"{incident_text}\n\nCurrent shared resource board:\n"
            f"{{ shared_state.board }}\n\nSelect the highest-severity unassigned new "
            f"incident your available resource can serve, or none."
        ),
        question_options=[item["id"] for item in wave_incidents] + ["none"],
    )
    resource = QuestionMultipleChoice(
        question_name=f"wave_{wave}_resource",
        question_text=(
            "You selected {{ "
            f"wave_{wave}_incident.answer"
            f" }}. Select your resource {{ agent.resource }} if deploying it; otherwise "
            "select none. Never select another agency's resource."
        ),
        question_options=list(RESOURCES) + ["none"],
    )
    return Survey(
        [
            incident,

```

```

        resource,
        state.board.allocate(
            incident,
            resource,
            responder="{{ agent.name }}",
            round_number=wave,
        ),
    ]
)

def run_disaster_response(
    log_path: str | Path = "disaster-response.jsonl",
    model_name: str = "gemini-2.5-flash",
) -> SharedState:
    state = SharedState(
        "coastal-storm-response",
        FileStateStore(log_path),
        board=SharedResourceBoard(INCIDENTS, RESOURCES),
    )
    model = Model(model_name)
    options = {
        "disable_remote_inference": True,
        "disable_remote_cache": True,
        "cache": False,
        "stop_on_exceptions": True,
    }
    for wave in (1, 2):
        wave_survey(state, wave).by(responders()).by(model).run(**options)
    state.close()
    return state

if __name__ == "__main__":
    print(run_disaster_response().render_markdown())

```

16.29 shared_state_family_message_board.py

Focus: Message boards. Inspect append-only identity, ordering, audiences, and read watermarks.

Compare this complete current program with the preferred construction in the corresponding chapter. Identify which lines express the research design and which exist only to serialize, schedule, authorize state reads, recover execution, or reshape results.

```

"""A multi-round family discussion generated by real persona-driven LLM agents."""

from pathlib import Path

from edsl import Agent, AgentList, Model, QuestionFreeText, Survey

```

```

from edsl.sharedstate import FileStateStore, SharedMessageBoard, SharedState

def family_agents() -> AgentList:
    return AgentList(
        [
            Agent(
                name="John",
                traits={
                    "family_role": "parent",
                    "spouse": "Robin",
                    "children": ["Ada", "Paul"],
                    "relationships": "Robin is your spouse; Ada and Paul are your children.",
                    "preference": "Go sailing together",
                    "persona": "Practical and safety-conscious; wants the family together and se
                },
            ),
            Agent(
                name="Robin",
                traits={
                    "family_role": "parent",
                    "spouse": "John",
                    "children": ["Ada", "Paul"],
                    "relationships": "John is your spouse; Ada and Paul are your children.",
                    "preference": "Go kayaking",
                    "persona": "Loves quiet coves and active exploration; advocates for kayaking
                },
            ),
            Agent(
                name="Ada",
                traits={
                    "family_role": "child",
                    "parents": ["John", "Robin"],
                    "sibling": "Paul",
                    "relationships": "John and Robin are your parents; Paul is your brother.",
                    "preference": "Go sailing",
                    "persona": "Enthusiastic about sailing and helping with the boat; thoughtful
                },
            ),
            Agent(
                name="Paul",
                traits={
                    "family_role": "child",
                    "parents": ["John", "Robin"],
                    "sibling": "Ada",
                    "relationships": "John and Robin are your parents; Ada is your sister.",
                    "preference": "Stay home and play video games",
                    "persona": "Would rather play games, but wants inclusion and will negotiate
                },
            ),
        ]
    )

```

```

    ),
]
)

def build_survey(log_path: str | Path) -> tuple[Survey, SharedState]:
    state = SharedState(
        scope="family-weekend",
        store=FileStateStore(log_path),
        board=SharedMessageBoard(),
    )
    author = QuestionFreeText(
        question_name="author",
        question_text="Return only your name. Your name is {{ agent.name }}.",
    )
    reply_to = QuestionFreeText(
        question_name="reply_to",
        question_text=(
            "Current family message board:\n{{ shared_state.board.messages }}\n\n"
            "Return only the exact author name you want to reply to, or NONE to "
            "start a new thread."
        ),
    )
    message = QuestionFreeText(
        question_name="message",
        question_text=(
            "Current family message board:\n{{ shared_state.board.messages }}\n\n"
            "Your preference: {{ agent.preference }}.\n"
            "Your persona: {{ agent.persona }}.\n"
            "Your family role: {{ agent.family_role }}.\n"
            "Your family relationships: {{ agent.relationships }}.\n"
            "You chose to reply to: {{ reply_to.answer }}.\n\n"
            "Write one concise, constructive message in your own voice. React to "
            "the discussion, advocate for your view, and seek compromise when "
            "appropriate. Do not prefix the message with your name."
        ),
    )
    survey = Survey(
        [author, reply_to, message, state.board.add(author, message, reply_to)]
    )
    return survey, state

def run_discussion(
    log_path: str | Path = "family-board.jsonl",
    rounds: int = 1,
    model_name: str = "gemini-2.5-flash",
) -> SharedState:
    """Run real LLM agents sequentially so each post sees every earlier post."""

```

```

survey, state = build_survey(log_path)
agents = family_agents()
model = Model(model_name)

for round_number in range(1, rounds + 1):
    round_agents = agents.duplicate()
    for agent in round_agents:
        agent.traits["discussion_round"] = round_number
    prior_count = state.read().state["board"]["message_count"]
    results = (
        survey.by(round_agents)
        .by(model)
        .run(
            interview_schedule="serial",
            disable_remote_inference=True,
            disable_remote_cache=True,
            cache=False,
            stop_on_exceptions=True,
        )
    )

    # The entire AgentList was submitted once. Serial scheduling preserves
    # its order, and each writing prompt contains every preceding board post.
    rows_by_name = {row["agent"]["name"]: row for row in results.to_dict()["data"]}
    messages = state.read().state["board"]["messages"]
    for offset, agent in enumerate(round_agents):
        prompt = rows_by_name[agent.name]["prompt"]["message_user_prompt"]["text"]
        visible = messages[: prior_count + offset]
        assert all(entry["message"] in prompt for entry in visible)

    return state

if __name__ == "__main__":
    board_state = run_discussion()
    print(board_state.render_markdown())

```

16.30 shared_state_forecast_revision.py

Focus: Forecast revision. Inspect initial beliefs, revealed evidence, and revised beliefs.

Compare this complete current program with the preferred construction in the corresponding chapter. Identify which lines express the research design and which exist only to serialize, schedule, authorize state reads, recover execution, or reshape results.

```

"""Forecasters update private-signal estimates after seeing a live consensus."""

from pathlib import Path

```



```

from edsl import Agent, AgentList, InterviewSchedule, Model, QuestionNumerical, Survey
from edsl.sharedstate import FileStateStore, SharedForecast, SharedState

def forecasters() -> AgentList:
    signals = [
        ("Aria", 78, "product telemetry specialist", "trusts behavioral usage data"),
        ("Basil", 64, "sales-operations analyst", "trusts customer commitments"),
        ("Chen", 42, "reliability engineer", "focuses on technical failure modes"),
        ("Dara", 28, "financial risk analyst", "uses conservative base rates"),
        (
            "Emi",
            55,
            "market researcher",
            "balances qualitative and quantitative evidence",
        ),
    ]
    return AgentList(
        [
            Agent(
                name=name,
                traits={
                    "private_signal": signal,
                    "expertise": expertise,
                    "method": method,
                },
            )
            for name, signal, expertise, method in signals
        ]
    )

def build_survey(state: SharedState) -> Survey:
    forecast = QuestionNumerical(
        question_name="probability",
        question_text=(
            "You are {{ agent.name }}, a {{ agent.expertise }} who {{ agent.method }}. "
            "Estimate the probability that Project Atlas will reach 100,000 weekly "
            "active users within six months. Your private evidence implies "
            "{{ agent.private_signal }}%.\n\n"
            "Forecasts visible at this moment:\n{{ shared_state.forecasts.latest }}\n"
            "Current confidence-weighted consensus: "
            "{{ shared_state.forecasts.confidence_weighted_probability }}\n\n"
            "Give your best probability from 0 to 100. Use others' forecasts as "
            "evidence, but do not discard your private signal without reason."
        ),
        min_value=0,
        max_value=100,
    )

```

```

confidence = QuestionNumerical(
    question_name="confidence",
    question_text=(
        "You forecast {{ probability.answer }}%. Rate your confidence in that "
        "estimate from 0 to 100, considering your evidence and the visible "
        "disagreement among other forecasters."
    ),
    min_value=0,
    max_value=100,
)
return Survey([forecast, confidence, state.forecasts.submit(forecast, confidence)])

def run_forecasts(
    log_path: str | Path = "forecast-revision.jsonl",
    model_name: str = "gemini-2.5-flash",
) -> SharedState:
    state = SharedState(
        "atlas-forecast",
        FileStateStore(log_path),
        forecasts=SharedForecast(),
    )
    agents = forecasters()
    model = Model(model_name)
    schedule = InterviewSchedule.rounds(
        count=3, within_round="concurrent", state_visibility="snapshot"
    )
    (
        build_survey(state)
        .by(agents)
        .by(model)
        .run(
            interview_schedule=schedule,
            disable_remote_inference=True,
            disable_remote_cache=True,
            cache=False,
            stop_on_exceptions=True,
        )
    )
    return state

if __name__ == "__main__":
    print(run_forecasts().render_markdown())

```

16.31 shared_state_hiring_committee.py

Focus: Hiring committees. Inspect private assessments, shared evidence, aggregation, and final authority.

Compare this complete current program with the preferred construction in the corresponding chapter. Identify which lines express the research design and which exist only to serialize, schedule, authorize state reads, recover execution, or reshape results.

```
"""A multi-stage hiring committee with private reviews and a secret ballot."""

from pathlib import Path

from edsl import Agent, AgentList, Model, QuestionFreeText, QuestionRank, Survey
from edsl.sharedstate import FileStateStore, SharedLog, SharedState

CANDIDATES = {
    "Avery": "Product leader; strong customer discovery and cross-functional delivery.",
    "Blake": "Engineering leader; strong systems design and organizational scaling.",
    "Casey": "Commercial leader; strong enterprise sales and partnerships.",
    "Devon": "Operations leader; strong process design and financial discipline.",
}

def committee() -> AgentList:
    specs = [
        ("Maya", "CEO", "balanced leadership and company-wide judgment"),
        (
            "Eli",
            "Engineering VP",
            "technical depth and effective engineering leadership",
        ),
        ("Sofia", "Sales VP", "customer credibility and commercial impact"),
        ("Priya", "Product VP", "product judgment and user-centered execution"),
        ("Noah", "Finance VP", "operating discipline and scalable decision-making"),
    ]
    return AgentList(
        [
            Agent(name=name, traits={"role": role, "priority": priority})
            for name, role, priority in specs
        ]
    )

def private_review_survey(state: SharedState) -> Survey:
    dossiers = "\n".join(f"{name}: {summary}" for name, summary in CANDIDATES.items())
    ranking = QuestionRank(
        question_name="private_ranking",
        question_text=(
```

```

        "You are {{ agent.name }}, the {{ agent.role }}. Your hiring priority is "
        "{{ agent.priority }}. Independently rank every candidate from strongest "
        f"to weakest overall.\n\nCandidate dossiers:\n{dossiers}"
    ),
    question_options=list(CANDIDATES),
)
rationale = QuestionFreeText(
    question_name="private_rationale",
    question_text=(
        "Briefly explain your private ranking {{ private_ranking.answer }}. "
        "Identify the most important strength and concern in at most 70 words."
    ),
)
return Survey(
    [
        ranking,
        rationale,
        state.private_reviews.append(
            reviewer="{{ agent.name }}", ranking=ranking, rationale=rationale
        ),
    ]
)

def shortlist_from_private_reviews(state: SharedState, size: int = 3) -> list[str]:
    entries = state.read().state["private_reviews"]["entries"]
    scores = {candidate: 0 for candidate in CANDIDATES}
    for entry in entries:
        for index, candidate in enumerate(entry["ranking"]):
            scores[candidate] += len(CANDIDATES) - index - 1
    return sorted(scores, key=lambda candidate: (-scores[candidate], candidate))[:size]

def deliberation_survey(state: SharedState, shortlist: list[str]) -> Survey:
    comment = QuestionFreeText(
        question_name="committee_statement",
        question_text=(
            "You are {{ agent.name }}, the {{ agent.role }}. The anonymized scoring "
            f"shortlisted {' '.join(shortlist)}.\n\nPublic comments so far:\n"
            "{{ shared_state.public_discussion.entries }}\n\nAdd one concise public "
            "comment that responds to the discussion and identifies a decisive "
            "comparison. Do not reveal your private ranking."
        ),
    ),
)
return Survey(
    [
        comment,
        state.public_discussion.append(speaker="{{ agent.name }}", comment=comment),
    ]
)

```

```

)

def ballot_survey(state: SharedState, shortlist: list[str]) -> Survey:
    ballot = QuestionRank(
        question_name="secret_ballot",
        question_text=(
            "After the committee discussion, privately rank the shortlisted candidates "
            "from your preferred hire to least preferred. Vote using your own judgment."
        ),
        question_options=shortlist,
    )
    return Survey(
        [
            ballot,
            state.secret_ballots.append(voter="{{ agent.name }}", ranking=ballot),
        ]
    )

def final_tally(state: SharedState, shortlist: list[str]) -> tuple[str, dict[str, int]]:
    entries = state.read().state["secret_ballots"]["entries"]
    scores = {candidate: 0 for candidate in shortlist}
    for entry in entries:
        for index, candidate in enumerate(entry["ranking"]):
            scores[candidate] += len(shortlist) - index - 1
    winner = sorted(scores, key=lambda candidate: (-scores[candidate], candidate))[0]
    return winner, scores

def run_hiring_committee(
    log_path: str | Path = "hiring-committee-live.jsonl",
    model_name: str = "gemini-2.5-flash",
) -> tuple[SharedState, list[str], str, dict[str, int]]:
    state = SharedState(
        "vp-hiring-committee",
        FileStateStore(log_path),
        private_reviews=SharedLog(),
        public_discussion=SharedLog(),
        secret_ballots=SharedLog(),
    )
    agents = committee()
    model = Model(model_name)
    run_options = {
        "disable_remote_inference": True,
        "disable_remote_cache": True,
        "cache": False,
        "stop_on_exceptions": True,
    }
}

```

```

private_review_survey(state).by(agents).by(model).run(**run_options)
shortlist = shortlist_from_private_reviews(state)
deliberation_survey(state, shortlist).by(agents).by(model).run(
    interview_schedule="serial", **run_options
)
ballot_survey(state, shortlist).by(agents).by(model).run(**run_options)
winner, scores = final_tally(state, shortlist)
state.close()
return state, shortlist, winner, scores

if __name__ == "__main__":
    shared_state, finalists, selected, tally = run_hiring_committee()
    print(shared_state.render_markdown())
    print(f"\nShortlist: {finalists}\nSelected: {selected}\nBorda tally: {tally}")

```

16.32 shared_state_incident_response.py

Focus: Work queues and command synthesis. Inspect atomic claims, resumability, and synthesis from task output.

Compare this complete current program with the preferred construction in the corresponding chapter. Identify which lines express the research design and which exist only to serialize, schedule, authorize state reads, recover execution, or reshape results.

```

"""Distributed incident investigation followed by commander synthesis."""

from pathlib import Path

from edsl import Agent, AgentList, Model, QuestionFreeText, Survey
from edsl.sharedstate import FileStateStore, SharedLog, SharedState, SharedWorkPool

TASKS = [
    {
        "id": "metrics",
        "area": "service metrics",
        "evidence": "Latency rose immediately after API release 4.8; CPU stayed normal.",
    },
    {
        "id": "deploy",
        "area": "deployment history",
        "evidence": "Release 4.8 changed retry defaults from 2 attempts to 8.",
    },
    {
        "id": "database",
        "area": "database behavior",
        "evidence": "Write locks spiked, but slow-query volume did not change.",
    },
]

```

```

{
    "id": "traffic",
    "area": "traffic and dependencies",
    "evidence": "A payment dependency began returning transient 503s at 09:02.",
},
]

def responders():
    specs = [
        ("Ari", "site reliability engineer"),
        ("Bo", "backend engineer"),
        ("Cy", "database specialist"),
        ("Dee", "dependency operations lead"),
    ]
    return AgentList([Agent(name=name, traits={"role": role}) for name, role in specs])

def investigation_survey(state):
    report = QuestionFreeText(
        question_name="investigation",
        question_text=(
            "You are {{ agent.name }}, the {{ agent.role }}, responding to an outage. "
            "Users see intermittent checkout timeouts.\n\n"
            "Your atomically assigned investigation: {{ shared_state.work.claimed }}\n"
            "Evidence posted so far: {{ shared_state.evidence.entries }}\n\n"
            "Analyze only your assigned evidence. Report observation, likely causal "
            "implication, confidence, and one recommended action. Clearly separate "
            "observed facts from inference."
        ),
    )
    return Survey(
        [
            state.work.claim_before(report),
            report,
            state.work.complete(report),
            state.evidence.append(
                sender="{{ agent.name }}",
                kind="investigation",
                report=report,
            ),
        ]
    )

def commander_survey(state):
    resolution = QuestionFreeText(
        question_name="resolution",
        question_text=(

```

```

        "You are incident commander Morgan. Checkout has intermittent timeouts.\n\n"
        "All investigation reports: {{ shared_state.evidence.entries }}\n\n"
        "Synthesize a root-cause hypothesis, immediate mitigation, verification "
        "step, and confidence. Call out any unresolved uncertainty."
    ),
)
return Survey(
    [
        resolution,
        state.evidence.append(
            sender="Morgan", kind="commander_resolution", report=resolution
        ),
    ]
)

def run_simulation(
    log_path: str | Path = "incident-response.jsonl",
    model_name="gemini-2.5-flash",
):
    state = SharedState(
        "checkout-incident",
        FileStateStore(log_path),
        work=SharedWorkPool(TASKS),
        evidence=SharedLog(),
    )
    model = Model(model_name)
    (
        investigation_survey(state)
        .by(responders())
        .by(model)
        .run(
            interview_schedule="concurrent",
            disable_remote_inference=True,
            disable_remote_cache=True,
            cache=False,
            stop_on_exceptions=True,
        )
    )
    (
        commander_survey(state)
        .by(Agent(name="Morgan", traits={"role": "incident commander"}))
        .by(model)
        .run(
            disable_remote_inference=True,
            disable_remote_cache=True,
            cache=False,
            stop_on_exceptions=True,
        )
    )

```



```

    )
    state.close()
    return state

if __name__ == "__main__":
    print(run_simulation().render_markdown())

```

16.33 shared_state_launch_readiness_review.py

Focus: Launch review. Inspect typed findings, ownership, gates, and final decision records.

Compare this complete current program with the preferred construction in the corresponding chapter. Identify which lines express the research design and which exist only to serialize, schedule, authorize state reads, recover execution, or reshape results.

```

"""Evidence-backed product launch review with mitigations, vetoes, and dissent."""

```

```

from pathlib import Path
from statistics import median

from edsl import (
    Agent,
    AgentList,
    Model,
    QuestionDict,
    QuestionFreeText,
    QuestionMultipleChoice,
    QuestionNumerical,
    Survey,
)
from edsl.sharedstate import FileStateStore, SharedLog, SharedState

```

```

LAUNCH = "Enterprise Permissions launch on September 30, 2026"

```

```

REVIEW_DATE = "August 28, 2026"

```

```

COMMON_STATUS = ""

```

```

- Core implementation is feature-complete; automated acceptance tests pass at 93%.
- Two medium-severity security findings remain open; no critical findings are known.
- Updated data-processing terms are drafted but not yet signed off by Legal.
- Administrator documentation is approximately 80% complete.
- Three lighthouse customers have committed to the date.
- Launch-week support coverage exists, but the escalation runbook is incomplete.
""".strip()

```

```

def reviewers() -> AgentList:
    specs = [
        (

```

```

    "Maya",
    "Product",
    "B1",
    "customer value, scope coherence, and adoption",
    False,
    "Beta users completed key workflows, but bulk role editing remains confusing.",
),
(
    "Eli",
    "Engineering",
    "B2",
    "technical quality, reliability, and delivery capacity",
    False,
    "Load tests pass at expected volume; rollback automation has not had a full rehearsal",
),
(
    "Dev",
    "Security",
    "B3",
    "security exposure and control effectiveness",
    True,
    "One open finding concerns privileged-session timeout; exploitability is moderate and",
),
(
    "Lena",
    "Legal",
    "B4",
    "contractual obligations, privacy, and regulatory exposure",
    True,
    "The revised DPA language is acceptable in principle but needs final outside-counsel",
),
(
    "Sofia",
    "Sales",
    "B5",
    "revenue commitments and market credibility",
    False,
    "Three enterprise buyers tie Q4 expansions to launch, but one expects a bulk-adminis",
),
(
    "Farid",
    "Customer Success",
    "B6",
    "implementation readiness and customer outcomes",
    False,
    "Implementation teams can onboard lighthouse accounts; administrator training materi",
),
(
    "Omar",

```

```

        "Operations",
        "B7",
        "supportability, observability, and incident response",
        False,
        "Dashboards and alerts exist; escalation ownership after US business hours is still
    ),
]
return AgentList(
    [
        Agent(
            name=name,
            traits={
                "function": function,
                "blocker_id": blocker_id,
                "lens": lens,
                "veto_authority": veto,
                "private_evidence": evidence,
            },
        )
        for name, function, blocker_id, lens, veto, evidence in specs
    ]
)

def initial_assessment_survey(state: SharedState) -> Survey:
    score = QuestionNumerical(
        question_name="initial_readiness",
        question_text=(
            f"Private initial review of {LAUNCH}. You represent {{{ agent.function }}} "
            "and focus on {{ agent.lens }}.\n\nCommon status:\n"
            f"{COMMON_STATUS}\n\nPrivate evidence available to your function:\n"
            f"{{ agent.private_evidence }}\n\nScore readiness from 0 to 100 before "
            "seeing anyone else's assessment. Use these anchors: 0 means impossible "
            "or prohibited; 50 means material unresolved blockers; 75 means ready only "
            "with explicit conditions; 100 means fully verified and ready."
        ),
        min_value=0,
        max_value=100,
    )
    recommendation = QuestionMultipleChoice(
        question_name="initial_recommendation",
        question_text="Give your private initial recommendation.",
        question_options=["launch", "limited_launch", "delay"],
    )
    blocker = QuestionDict(
        question_name="primary_blocker",
        question_text=(
            "Document the single most decision-relevant blocker or concern from your "
            "function. If none is launch-blocking, document the most important residual "

```

```

        "risk. Keep each field under 40 words."
    ),
    answer_keys=["title", "severity", "evidence", "owner_function"],
    value_types=["str", "str", "str", "str"],
    value_descriptions=[
        "Short blocker title",
        "One of critical, high, medium, or low",
        "Concrete evidence rather than general concern",
        "Function accountable for mitigation",
    ],
)
rationale = QuestionFreeText(
    question_name="initial_rationale",
    question_text=(
        "Explain your score and recommendation in at most 60 words, citing the "
        "common status or your private evidence."
    ),
)
return Survey(
    [
        score,
        recommendation,
        blocker,
        rationale,
        state.assessments.append(
            reviewer="{{ agent.name }}",
            function="{{ agent.function }}",
            stage="initial",
            score=score,
            recommendation=recommendation,
            rationale=rationale,
        ),
        state.blockers.append(
            blocker_id="{{ agent.blocker_id }}",
            reporter_function="{{ agent.function }}",
            blocker=blocker,
        ),
    ]
)

def mitigation_survey(state: SharedState) -> Survey:
    context = (
        "You represent {{ agent.function }}. All independently submitted blockers "
        "are now revealed simultaneously:\n{{ shared_state.blockers.entries }}\n\n"
    )
    addressed = QuestionFreeText(
        question_name="addressed_blockers",
        question_text=(

```

```

        context
        + "List the comma-separated blocker IDs your function can materially address. "
        f"The review date is {REVIEW_DATE} and launch is September 30, 2026."
    ),
)
mitigation = QuestionFreeText(
    question_name="mitigation_plan",
    question_text="State the concrete mitigation or scope restriction in at most 55 words.",
)
evidence = QuestionFreeText(
    question_name="verification_evidence",
    question_text=(
        "State the evidence already available or required to verify the mitigation. "
        "Do not claim closure without evidence. At most 55 words."
    ),
)
residual = QuestionFreeText(
    question_name="residual_risk",
    question_text="State the risk remaining after mitigation in at most 45 words.",
)
deadline = QuestionFreeText(
    question_name="mitigation_deadline",
    question_text=(
        f"Give a specific deadline between {REVIEW_DATE} and September 30, 2026."
    ),
)
)
return Survey(
    [
        addressed,
        mitigation,
        evidence,
        residual,
        deadline,
        state.mitigations.append(
            owner="{{ agent.name }}",
            owner_function="{{ agent.function }}",
            addressed_blocker_ids=addressed,
            mitigation=mitigation,
            verification_evidence=evidence,
            residual_risk=residual,
            deadline=deadline,
        ),
    ]
)

def final_assessment_survey(state: SharedState) -> Survey:
    score = QuestionNumerical(
        question_name="final_readiness",

```

```

question_text=(
    "Reassess launch readiness independently after reviewing all blockers and "
    "mitigations.\n\nBlockers:\n{{ shared_state.blockers.entries }}\n\n"
    "Mitigations:\n{{ shared_state.mitigations.entries }}\n\nScore 0 to 100. "
    "Use these anchors: 0 means impossible or prohibited; 50 means material "
    "unresolved blockers; 75 means ready only with explicit conditions; 100 "
    "means fully verified and ready."
),
min_value=0,
max_value=100,
)
recommendation = QuestionMultipleChoice(
    question_name="final_recommendation",
    question_text=(
        "Give your final recommendation. A limited launch means only the three "
        "lighthouse customers with explicit controls."
    ),
    question_options=["launch", "limited_launch", "delay"],
)
approval = QuestionMultipleChoice(
    question_name="approval_status",
    question_text=(
        "State whether your function approves unconditionally, approves subject "
        "to a stated condition, or does not approve."
    ),
    question_options=["approved", "conditional", "not_approved"],
)
condition = QuestionFreeText(
    question_name="approval_condition",
    question_text=(
        "State the exact approval condition, or 'none' if unconditional. Include "
        "what evidence closes it and a deadline no later than September 30, 2026. "
        "At most 55 words."
    ),
)
rationale = QuestionFreeText(
    question_name="final_rationale",
    question_text=(
        "Briefly explain what changed or did not change your assessment. Preserve "
        "any dissent from the apparent group direction. At most 60 words."
    ),
)
return Survey(
    [
        score,
        recommendation,
        approval,
        condition,
        rationale,
    ]
)

```

```

        state.final_reviews.append(
            reviewer="{ agent.name }",
            function="{ agent.function }",
            veto_authority="{ agent.veto_authority }",
            score=score,
            recommendation=recommendation,
            approval=approval,
            condition=condition,
            rationale=rationale,
        ),
    ]
)

def require_count(state: SharedState, primitive: str, expected: int) -> None:
    count = state.read().state[primitive]["count"]
    if count < expected:
        raise RuntimeError(
            f"{primitive} incomplete: expected {expected} persisted records, got {count}"
        )

def missing_reviewers(
    state: SharedState,
    agents: AgentList,
    primitive: str,
    identity_field: str,
) -> AgentList:
    completed = {
        entry[identity_field] for entry in state.read().state[primitive]["entries"]
    }
    return AgentList([agent for agent in agents if agent.name not in completed])

def decide(state: SharedState) -> dict:
    initial = state.read().state["assessments"]["entries"]
    final = state.read().state["final_reviews"]["entries"]
    mitigations = state.read().state["mitigations"]["entries"]
    vetoes = [
        review
        for review in final
        if review["veto_authority"]
        and (
            review["recommendation"] == "delay" or review["approval"] == "not_approved"
        )
    ]
    final_scores = [review["score"] for review in final]
    recommendations = [review["recommendation"] for review in final]
    if vetoes:

```

```

        decision = "delay"
    elif median(final_scores) >= 75 and recommendations.count("launch") >= 4:
        decision = "launch"
    elif median(final_scores) >= 60 and recommendations.count("delay") <= 2:
        decision = "limited_launch"
    else:
        decision = "delay"
    conditions = [
        {
            "function": review["function"],
            "condition": review["condition"],
        }
        for review in final
        if review["approval"] != "approved"
        or review["condition"].strip().lower() != "none"
    ]
    dissent = [
        {
            "function": review["function"],
            "recommendation": review["recommendation"],
            "rationale": review["rationale"],
        }
        for review in final
        if review["recommendation"] != decision
    ]
    initial_by_function = {item["function"]: item for item in initial}
    movement = {
        review["function"]: review["score"]
        - initial_by_function[review["function"]]["score"]
        for review in final
    }
    return {
        "decision": decision,
        "initial_median": median(item["score"] for item in initial),
        "final_median": median(final_scores),
        "score_movement": movement,
        "vetoes": vetoes,
        "conditions": conditions,
        "dissent": dissent,
        "owners_and_deadlines": [
            {
                "owner": item["owner"],
                "function": item["owner_function"],
                "blockers": item.get("response", item)["addressed_blocker_ids"],
                "deadline": item.get("response", item)["deadline"],
            }
            for item in mitigations
        ],
    }
}

```



```

def run_launch_review(
    log_path: str | Path = "launch-readiness-review-live.jsonl",
    model_name: str = "gemini-2.5-flash",
) -> tuple[SharedState, dict]:
    state = SharedState(
        "enterprise-permissions-launch",
        FileStateStore(log_path),
        assessments=SharedLog(),
        blockers=SharedLog(),
        mitigations=SharedLog(),
        final_reviews=SharedLog(),
    )
    agents = reviewers()
    model = Model(model_name)
    options = {
        "disable_remote_inference": True,
        "disable_remote_cache": True,
        "cache": False,
        "stop_on_exceptions": True,
    }
    initial_missing = AgentList(
        [
            agent
            for agent in agents
            if agent.name
            not in {
                entry["reviewer"]
                for entry in state.read().state["assessments"]["entries"]
            }
            or agent.traits["blocker_id"]
            not in {
                entry["blocker_id"]
                for entry in state.read().state["blockers"]["entries"]
            }
        ]
    )
    if initial_missing:
        initial_assessment_survey(state).by(initial_missing).by(model).run(**options)
    require_count(state, "assessments", 7)
    require_count(state, "blockers", 7)
    mitigation_missing = missing_reviewers(state, agents, "mitigations", "owner")
    if mitigation_missing:
        mitigation_survey(state).by(mitigation_missing).by(model).run(**options)
    require_count(state, "mitigations", 7)
    final_missing = missing_reviewers(state, agents, "final_reviews", "reviewer")
    if final_missing:
        final_assessment_survey(state).by(final_missing).by(model).run(**options)

```

```

    require_count(state, "final_reviews", 7)
    decision = decide(state)
    state.close()
    return state, decision

if __name__ == "__main__":
    shared_state, launch_decision = run_launch_review()
    print(shared_state.render_markdown())
    print(f"\nDecision record: {launch_decision}")

```

16.34 shared_state_legislative_amendments.py

Focus: Versioned document revision. Inspect amendment identity, lineage, voting, and stale-version protection.

Compare this complete current program with the preferred construction in the corresponding chapter. Identify which lines express the research design and which exist only to serialize, schedule, authorize state reads, recover execution, or reshape results.

```

"""Live serial revision of a small bill by competing legislators."""

from pathlib import Path

from edsl import Agent, AgentList, InterviewSchedule, Model, QuestionFreeText, Survey
from edsl.sharedstate import FileStateStore, SharedDocument, SharedState

INITIAL_BILL = """1. Automated decisions must be disclosed to affected residents.
2. Agencies must publish an annual summary of automated systems.
3. This act takes effect 30 days after passage."""

def legislators():
    specs = [
        ("Rosa", 0, "civil-liberties advocate", "appeals and individual due process"),
        ("Sam", 1, "city operations chair", "feasible implementation and cost"),
        ("Talía", 2, "labor representative", "worker consultation and job protections"),
        (
            "Uma",
            3,
            "technology reformer",
            "audits, transparency, and measurable enforcement",
        ),
    ]
    return AgentList(
        [
            Agent(name=name, traits={"seat": seat, "role": role, "priority": priority})
            for name, seat, role, priority in specs

```

```

    ]
)

def build_survey(state):
    draft = QuestionFreeText(
        question_name="revised_bill",
        question_text=(
            "Round {{ run.round }} of 2. You are {{ agent.name }}, a {{ agent.role }} "
            "focused on {{ agent.priority }}.\n\nCurrent bill:\n"
            "{{ shared_state.bill.text }}\n\nRecent revisions: "
            "{{ shared_state.bill.recent_revisions }}\n\nReturn the complete bill with one "
            "careful amendment. Preserve provisions you do not intend to change."
        ),
    )
    rationale = QuestionFreeText(
        question_name="rationale",
        question_text="Briefly explain the single amendment you made and its tradeoff.",
    )
    return Survey([draft, rationale, state.bill.revise(draft, rationale)])

def run_simulation(
    log_path: str | Path = "legislative-amendments.jsonl",
    model_name="gemini-2.5-flash",
):
    state = SharedState(
        "automated-decisions-bill",
        FileStateStore(log_path),
        bill=SharedDocument("Automated Decisions Accountability Act", INITIAL_BILL),
    )
    schedule = InterviewSchedule.rounds(
        count=2,
        within_round="serial",
        state_visibility="live",
        order_by="seat",
        round_order="rotate",
    )
    (
        build_survey(state)
        .by(legislators())
        .by(Model(model_name))
        .run(
            interview_schedule=schedule,
            disable_remote_inference=True,
            disable_remote_cache=True,
            cache=False,
            stop_on_exceptions=True,
        )
    )

```

```

    )
    state.close()
    return state

if __name__ == "__main__":
    print(run_simulation().render_markdown())

```

16.35 shared_state_live_review_queue.py

Focus: Work queues. Inspect exclusive claims, failure recovery, and exactly-once completion.

Compare this complete current program with the preferred construction in the corresponding chapter. Identify which lines express the research design and which exist only to serialize, schedule, authorize state reads, recover execution, or reshape results.

```

"""Concurrent reviewers atomically claim distinct papers before prompt rendering."""

from pathlib import Path

from edsl import Agent, AgentList, Model, QuestionDict, Survey
from edsl.sharedstate import FileStateStore, SharedState, SharedWorkPool

PAPERS = [
    {
        "id": "paper-causal",
        "title": "Adaptive stopping in online field experiments",
        "abstract": "A sequential testing method for marketplace experiments with delayed outcomes",
    },
    {
        "id": "paper-privacy",
        "title": "Private evaluation of language models",
        "abstract": "A framework for releasing benchmark aggregates under differential privacy.",
    },
    {
        "id": "paper-games",
        "title": "Cooperation under imperfect monitoring",
        "abstract": "Laboratory evidence from repeated public-goods games with noisy signals.",
    },
    {
        "id": "paper-agents",
        "title": "Auditing tool-using agents",
        "abstract": "Methods for reconstructing state reads, tool calls, and causal action traces",
    },
]

def reviewers() -> AgentList:

```

```

return AgentList(
    [
        Agent(name="Rina", traits={"expertise": "causal inference"}),
        Agent(name="Omar", traits={"expertise": "privacy and security"}),
        Agent(name="Lin", traits={"expertise": "behavioral economics"}),
        Agent(name="Grace", traits={"expertise": "AI evaluation"}),
    ]
)

def build_survey(state: SharedState) -> Survey:
    review = QuestionDict(
        question_name="review",
        question_text=(
            "You are {{ agent.name }}, an expert in {{ agent.expertise }}. You have "
            "atomically claimed this paper:\n{{ shared_state.work.claimed }}\n\n"
            "Review only that paper. Give a recommendation and a concise rationale "
            "that identifies one strength and one concern."
        ),
        answer_keys=["recommendation", "rationale"],
        value_types=["str", "str"],
        value_descriptions=[
            "One of accept, revise, or reject",
            "A concise review with one strength and one concern",
        ],
    )
    claim = state.work.claim_before(review)
    return Survey([claim, review, state.work.complete(review)])

def run_queue(
    log_path: str | Path = "live-review-queue.jsonl",
    model_name: str = "gemini-2.5-flash",
) -> SharedState:
    state = SharedState(
        "live-review-queue",
        FileStateStore(log_path),
        work=SharedWorkPool(PAPERS),
    )
    (
        build_survey(state)
        .by(reviewers())
        .by(Model(model_name))
        .run(
            disable_remote_inference=True,
            disable_remote_cache=True,
            cache=False,
            stop_on_exceptions=True,
        )
    )

```

```

    )
    return state

if __name__ == "__main__":
    print(run_queue().render_markdown())

```

16.36 shared_state_matching_market.py

Focus: Matching markets. Inspect preferences, capacity, conflicts, and algorithmic tie-breaking.

Compare this complete current program with the preferred construction in the corresponding chapter. Identify which lines express the research design and which exist only to serialize, schedule, authorize state reads, recover execution, or reshape results.

```

"""Student-proposing deferred acceptance with LLM-generated preferences."""

from pathlib import Path

from edsl import Agent, AgentList, Model, QuestionRank, Survey
from edsl.sharedstate import FileStateStore, SharedDeferredAcceptance, SharedState

PROGRAMS = {
    "Northstar": "Quantitative public policy; urban campus; intense mathematical core.",
    "Lakeside": "Environmental policy; small collaborative cohort; fieldwork emphasis.",
    "CivicLab": "Technology and governance; project-based curriculum; strong internships.",
}

CAPACITIES = {"Northstar": 2, "Lakeside": 2, "CivicLab": 2}
PRIORITIES = {
    "Northstar": ["Amina", "Diego", "Farah", "Ben", "Chloe", "Evan"],
    "Lakeside": ["Chloe", "Evan", "Ben", "Farah", "Amina", "Diego"],
    "CivicLab": ["Farah", "Ben", "Diego", "Amina", "Evan", "Chloe"],
}

def students() -> AgentList:
    specs = [
        ("Amina", "econometrics and housing policy", "rigorous quantitative training"),
        (
            "Ben",
            "civic technology and product design",
            "hands-on projects and internships",
        ),
        (
            "Chloe",
            "climate adaptation and conservation",
            "fieldwork and a close cohort",
        ),
    ]

```

```

        ("Diego", "data science and transportation", "technical depth and city access"),
        (
            "Evan",
            "environmental justice and community organizing",
            "collaboration and applied work",
        ),
        (
            "Farah",
            "AI governance and public institutions",
            "technology-policy integration",
        ),
    ]
    return AgentList(
        [
            Agent(
                name=name,
                traits={"interests": interests, "priority": priority},
            )
            for name, interests, priority in specs
        ]
    )

def preference_survey(state: SharedState) -> Survey:
    descriptions = "\n".join(
        f"{program}: {description}" for program, description in PROGRAMS.items()
    )
    ranking = QuestionRank(
        question_name="program_ranking",
        question_text=(
            "You are {{ agent.name }}. Your interests are {{ agent.interests }}, and "
            "you especially value {{ agent.priority }}. Rank every program from your "
            f"most to least preferred.\n\nPrograms:\n{descriptions}\n\n"
            "These preferences are private. Do not try to predict other applicants."
        ),
        question_options=list(PROGRAMS),
    )
    return Survey([ranking, state.market.collect(ranking, student="{{ agent.name }}")])

def run_matching_market(
    log_path: str | Path = "matching-market.jsonl",
    model_name: str = "gemini-2.5-flash",
) -> SharedState:
    state = SharedState(
        "graduate-program-match",
        FileStateStore(log_path),
        market=SharedDeferredAcceptance(CAPACITIES, PRIORITIES),
    )

```

```

(
    preference_survey(state)
    .by(students())
    .by(Model(model_name))
    .run(
        disable_remote_inference=True,
        disable_remote_cache=True,
        cache=False,
        stop_on_exceptions=True,
    )
)
state.close()
return state

if __name__ == "__main__":
    print(run_matching_market().render_markdown())

```

16.37 shared_state_meeting_agenda.py

Focus: Agenda setting. Inspect proposal, prioritization, and immutable meeting order.

Compare this complete current program with the preferred construction in the corresponding chapter. Identify which lines express the research design and which exist only to serialize, schedule, authorize state reads, recover execution, or reshape results.

```

"""LLM agents propose meeting agenda items, then vote on the shared slate."""

from pathlib import Path

from edsl import (
    Agent,
    AgentList,
    Model,
    QuestionFreeText,
    QuestionMatrix,
    Survey,
)
from edsl.sharedstate import FileStateStore, SharedAgenda, SharedState

def meeting_agents() -> AgentList:
    return AgentList(
        [
            Agent(
                name="Maya",
                traits={
                    "role": "CEO and meeting chair",
                    "priority": "alignment on the most consequential company decision",

```



```

        "persona": "strategic, concise, and focused on decisions rather than updates
    },
),
Agent(
    name="Eli",
    traits={
        "role": "engineering lead",
        "priority": "technical reliability, delivery risks, and sustainable execution",
        "persona": "pragmatic and specific about tradeoffs",
    },
),
Agent(
    name="Sofia",
    traits={
        "role": "sales lead",
        "priority": "customer commitments, pipeline, and near-term revenue",
        "persona": "customer-oriented and commercially urgent",
    },
),
Agent(
    name="Noah",
    traits={
        "role": "finance lead",
        "priority": "runway, resource allocation, and measurable returns",
        "persona": "analytical and disciplined about opportunity cost",
    },
),
Agent(
    name="Priya",
    traits={
        "role": "product and design lead",
        "priority": "user needs, product quality, and a coherent roadmap",
        "persona": "empathetic, evidence-driven, and attentive to product clarity",
    },
),
]
)

```

```

def build_proposal_survey(state: SharedState) -> Survey:
    proposal = QuestionFreeText(
        question_name="agenda_proposal",
        question_text=(
            "You are {{ agent.name }}, the {{ agent.role }}. Your priority is "
            "{{ agent.priority }} and you are {{ agent.persona }}.\n\n"
            "Agenda items already proposed:\n{{ shared_state.agenda.proposals }}\n\n"
            "Propose one distinct agenda item for a 60-minute leadership meeting. "
            "Write a concrete decision-oriented title in at most 16 words. Do not "
            "repeat an existing item and do not include your name."
        )
    )

```

```

    ),
)
return Survey([proposal, state.agenda.propose(proposal)])

def build_voting_survey(state: SharedState) -> Survey:
    proposals = state.read().state["agenda"]["proposals"]
    item_ids = [item["id"] for item in proposals]
    slate = "\n".join(
        f"{item['id']}: {item['title']} (proposed by {item['proposer']})"
        for item in proposals
    )
    vote = QuestionMatrix(
        question_name="agenda_votes",
        question_text=(
            "You are {{ agent.name }}, the {{ agent.role }}. Your priority is "
            "{{ agent.priority }}.\n\n"
            f"Proposed agenda:\n{slate}\n\n"
            "Vote on every item. Vote up when it deserves scarce meeting time, "
            "neutral when useful but not essential, and down when it should be "
            "handled asynchronously. Judge all proposals, including your own."
        ),
        question_items=item_ids,
        question_options=["up", "neutral", "down"],
        randomize_items=True,
    )
    return Survey([vote, state.agenda.vote(vote)])

def run_agenda_simulation(
    log_path: str | Path = "meeting-agenda.jsonl",
    model_name: str = "gemini-2.5-flash",
) -> SharedState:
    state = SharedState(
        "leadership-meeting",
        FileStateStore(log_path),
        agenda=SharedAgenda(),
    )
    agents = meeting_agents()
    model = Model(model_name)

    # Proposals are serial so later participants can avoid duplicating earlier ideas.
    (
        build_proposal_survey(state)
        .by(agents)
        .by(model)
        .run(
            interview_schedule="serial",
            disable_remote_inference=True,

```

```

        disable_remote_cache=True,
        cache=False,
        stop_on_exceptions=True,
    )
)

# The resulting slate fixes the QuestionMatrix rows. Ballots are independent.
(
    build_voting_survey(state)
    .by(agents)
    .by(model)
    .run(
        disable_remote_inference=True,
        disable_remote_cache=True,
        cache=False,
        stop_on_exceptions=True,
    )
)
return state

if __name__ == "__main__":
    print(run_agenda_simulation().render_markdown())

```

16.38 shared_state_peer_review_matching.py

Focus: Matching with conflicts. Inspect conflict validation, capacity, and deterministic assignment.

Compare this complete current program with the preferred construction in the corresponding chapter. Identify which lines express the research design and which exist only to serialize, schedule, authorize state reads, recover execution, or reshape results.

```

"""Assign reviewers to papers using private rankings and deterministic priority."""

from pathlib import Path

from edsl import Agent, AgentList, Model, QuestionRank, Survey
from edsl.sharedstate import FileStateStore, SharedMatchPool, SharedState

PAPERS = {
    "P1": "Causal inference for marketplace experiments",
    "P2": "Privacy-preserving language-model evaluation",
    "P3": "Behavioral dynamics in repeated public-goods games",
}

def reviewers() -> AgentList:
    specs = [

```

```

        ("Rina", 1, "causal inference and field experiments", "P1"),
        ("Omar", 2, "privacy, security, and model evaluation", "P2"),
        ("Lin", 3, "behavioral economics and repeated games", "P3"),
        ("Mateo", 4, "experimental design and applied statistics", "P2"),
        ("Grace", 5, "machine learning evaluation and governance", "P1"),
        ("Tariq", 6, "game theory and computational social science", "P3"),
    ]
    return AgentList(
        [
            Agent(
                name=name,
                traits={
                    "review_order": order,
                    "expertise": expertise,
                    "conflict": conflict,
                },
            )
            for name, order, expertise, conflict in specs
        ]
    )

def build_survey(state: SharedState) -> Survey:
    paper_list = "\n".join(f"{key}: {title}" for key, title in PAPERS.items())
    ranking = QuestionRank(
        question_name="paper_ranking",
        question_text=(
            "You are reviewer {{ agent.name }} with expertise in "
            "{{ agent.expertise }}. You have a conflict with {{ agent.conflict }}"
            "and must rank it last.\n\n"
            f"Papers:\n{paper_list}\n\n"
            "Rank all paper IDs from best to worst review fit. Prefer papers where "
            "your expertise adds the most value, subject to the conflict rule."
        ),
        question_options=list(PAPERS),
    )
    return Survey(
        [
            ranking,
            state.assignments.collect(
                ranking,
                claimant="{{ agent.name }}",
                priority="{{ agent.review_order }}",
            ),
        ]
    )

def run_matching(

```

```

    log_path: str | Path = "peer-review-matching.jsonl",
    model_name: str = "gemini-2.5-flash",
) -> SharedState:
    state = SharedState(
        "peer-review-panel",
        FileStateStore(log_path),
        assignments=SharedMatchPool(list(PAPERS), capacity=2),
    )
    (
        build_survey(state)
        .by(reviewers())
        .by(Model(model_name))
        .run(
            disable_remote_inference=True,
            disable_remote_cache=True,
            cache=False,
            stop_on_exceptions=True,
        )
    )
    state.close()
    return state

if __name__ == "__main__":
    print(run_matching().render_markdown())

```

16.39 shared_state_prediction_market_private_news.py

Focus: Private-news markets. Inspect private news, public prices, sequencing, and exposure auditing.

Compare this complete current program with the preferred construction in the corresponding chapter. Identify which lines express the research design and which exist only to serialize, schedule, authorize state reads, recover execution, or reshape results.

```

"""Binary prediction market with private signals released just in time."""

from pathlib import Path

from edsl import (
    Agent,
    AgentList,
    InterviewSchedule,
    Model,
    QuestionMultipleChoice,
    QuestionNumerical,
    Survey,
)
from edsl.sharedstate import (
    FileStateStore,

```

```

    SharedBinaryMarket,
    SharedSignalSchedule,
    SharedState,
)
from edsl.sharedstate.steps import StepContext

CONTRACT = "Project Lyra ships its public release before December 1"

SIGNALS = {
    "Nova": [
        "Your prior probability is 0.62 based on the published roadmap.",
        "A private beta report says all critical workflows now pass.",
        "A trusted engineer says release automation is complete.",
    ],
    "Oren": [
        "Your prior probability is 0.46 because past launches slipped.",
        "An internal incident caused a one-week reliability delay.",
        "The incident review closed with no remaining launch blockers.",
    ],
    "Priya": [
        "Your prior probability is 0.55 based on customer readiness.",
        "Two design partners privately committed to launch-day adoption.",
        "A major integration partner moved certification back two weeks.",
    ],
    "Quinn": [
        "Your prior probability is 0.36 due to regulatory uncertainty.",
        "Counsel privately identified an unresolved compliance question.",
        "The regulator granted the required expedited approval.",
    ],
}

def traders() -> AgentList:
    roles = [
        ("Nova", 0, "product telemetry analyst"),
        ("Oren", 1, "reliability forecaster"),
        ("Priya", 2, "customer research lead"),
        ("Quinn", 3, "regulatory risk analyst"),
    ]
    return AgentList(
        [
            Agent(name=name, traits={"seat": seat, "role": role})
            for name, seat, role in roles
        ]
    )

def build_survey(state: SharedState) -> Survey:

```

```

action = QuestionMultipleChoice(
    question_name="market_action",
    question_text=(
        "Round {{ run.round }} of 3. You are {{ agent.name }}, the "
        "{{ agent.role }}.\n\n"
        "New private signal: {{ shared_state.news.your_signal }}\n"
        "Your revealed signal history: {{ shared_state.news.your_signal_history }}\n"
        "Do not reveal the private signals directly.\n\n"
        "Contract: {{ shared_state.market.contract }}\n"
        "Current YES price: {{ shared_state.market.yes_price }}\n"
        "Current NO price: {{ shared_state.market.no_price }}\n"
        "Your portfolio: {{ shared_state.market.your_portfolio }}\n"
        "Recent trades: {{ shared_state.market.recent_trades }}\n\n"
        "Update your belief using only signals revealed so far, then choose a trade."
    ),
    question_options=["buy_yes", "buy_no", "hold"],
)
quantity = QuestionNumerical(
    question_name="trade_quantity",
    question_text=(
        "You chose {{ market_action.answer }}. Choose 0-12 shares, preserving "
        "cash for future news. Use 0 for hold."
    ),
    min_value=0,
    max_value=12,
)
return Survey(
    [
        state.news.reveal_before(action),
        action,
        quantity,
        state.market.trade(action, quantity),
    ]
)

def run_simulation(
    log_path: str | Path = "prediction-market-private-news.jsonl",
    model_name: str = "gemini-2.5-flash",
) -> SharedState:
    state = SharedState(
        "lyra-private-news-market",
        FileStateStore(log_path),
        news=SharedSignalSchedule(SIGNALS),
        market=SharedBinaryMarket(CONTRACT, liquidity=35, initial_cash=100),
    )
    schedule = InterviewSchedule.rounds(
        count=3,
        within_round="serial",
    )

```

```

        state_visibility="live",
        order_by="seat",
        round_order="rotate",
    )
    (
        build_survey(state)
        .by(traders())
        .by(Model(model_name))
        .run(
            interview_schedule=schedule,
            disable_remote_inference=True,
            disable_remote_cache=True,
            cache=False,
            stop_on_exceptions=True,
        )
    )
    state.market.settle(True).execute(StepContext({}, "market-resolution"))
    state.close()
    return state

if __name__ == "__main__":
    print(run_simulation().render_markdown())

```

16.40 shared_state_public_goods.py

Focus: Repeated public goods. Inspect round-local contributions, history visibility, and stable identity.

Compare this complete current program with the preferred construction in the corresponding chapter. Identify which lines express the research design and which exist only to serialize, schedule, authorize state reads, recover execution, or reshape results.

```

"""Repeated public-goods game using a generic shared append-only log."""

from pathlib import Path

from edsl import (
    Agent,
    AgentList,
    InterviewSchedule,
    Model,
    QuestionFreeText,
    QuestionNumerical,
    Survey,
)
from edsl.sharedstate import FileStateStore, SharedLog, SharedState

ENDOWMENT = 20

```



```
MULTIPLIER = 1.6
```

```
def public_goods_agents() -> AgentList:
    return AgentList(
        [
            Agent(
                name="Avery",
                traits={
                    "strategy": "Conditional cooperator",
                    "persona": "You begin cooperatively but respond strongly to evidence of free
                },
            ),
            Agent(
                name="Blake",
                traits={
                    "strategy": "Self-interested optimizer",
                    "persona": "You maximize your own total payoff and reason strategically about
                },
            ),
            Agent(
                name="Casey",
                traits={
                    "strategy": "Group-oriented contributor",
                    "persona": "You value group welfare, fairness, and establishing cooperative
                },
            ),
            Agent(
                name="Devon",
                traits={
                    "strategy": "Reciprocal pragmatist",
                    "persona": "You match demonstrated cooperation but avoid being exploited.",
                },
            ),
        ]
    )
```

```
def build_round_survey(state: SharedState) -> Survey:
    contribution = QuestionNumerical(
        question_name="contribution",
        question_text=(
            "Round {{ run.round }} of 4 in a repeated public-goods game. "
            f"You receive {ENDOWMENT} tokens this round. Every contributed token "
            f"is multiplied by {MULTIPLIER} and divided equally among four players. "
            "Uncontributed tokens remain yours.\n\n"
            "Your strategy: {{ agent.strategy }}. {{ agent.persona }}\n\n"
            "Contribution history visible at this moment:\n"
            f"{{ shared_state.contributions.entries }}\n\n"
        )
    )
```

```

        "Choose an integer contribution from 0 through 20."
    ),
    min_value=0,
    max_value=20,
)
rationale = QuestionFreeText(
    question_name="rationale",
    question_text=(
        "You chose to contribute {{ contribution.answer }} tokens in round "
        "{{ run.round }}. In one sentence, explain the strategic reason."
    ),
)
return Survey(
    [
        contribution,
        rationale,
        state.contributions.append(
            player="{{ agent.name }}",
            strategy="{{ agent.strategy }}",
            round="{{ run.round }}",
            amount=contribution,
            rationale=rationale,
        ),
    ]
)

def summarize(state: SharedState) -> str:
    entries = state.read().state["contributions"]["entries"]
    by_round = {}
    payoffs = {agent.name: 0.0 for agent in public_goods_agents()}
    for entry in entries:
        by_round.setdefault(int(entry["round"]), []).append(entry)
    lines = ["# Repeated public-goods game", ""]
    for round_number, contributions in sorted(by_round.items()):
        pot = sum(entry["amount"] for entry in contributions)
        share = pot * MULTIPLIER / 4
        lines.extend(
            [
                f"## Round {round_number}",
                "",
                "| Player | Contribution | Round payoff | Rationale |",
                "|---|---:|---:|---|",
            ]
        )
    for entry in contributions:
        payoff = ENDOWMENT - entry["amount"] + share
        payoffs[entry["player"]] += payoff
        lines.append(

```

```

        f"| {entry['player']} | {entry['amount']:g} | {payoff:.1f} | "
        f"{entry['rationale']} |"
    )
    lines.extend(["", f"**Group contribution:** {pot:g}/80", ""])
lines.extend(
    [
        "## Total payoffs",
        "",
        "| Player | Payoff |",
        "|---|---:|",
        *[
            f"| {player} | {payoff:.1f} |"
            for player, payoff in sorted(payoffs.items(), key=lambda item: -item[1])
        ],
    ]
)
return "\n".join(lines)

def run_public_goods(
    log_path: str | Path = "public-goods.jsonl",
    model_name: str = "gemini-2.5-flash",
) -> SharedState:
    state = SharedState(
        "public-goods-game",
        FileStateStore(log_path),
        contributions=SharedLog(),
    )
    agents = public_goods_agents()
    model = Model(model_name)
    schedule = InterviewSchedule.rounds(
        count=4, within_round="concurrent", state_visibility="snapshot"
    )
    (
        build_round_survey(state)
        .by(agents)
        .by(model)
        .run(
            interview_schedule=schedule,
            disable_remote_inference=True,
            disable_remote_cache=True,
            cache=False,
            stop_on_exceptions=True,
        )
    )
    return state

if __name__ == "__main__":

```

```
print(summarize(run_public_goods()))
```

16.41 shared_state_rumor_diffusion.py

Focus: Information diffusion. Inspect claim provenance, exposure paths, and beliefs versus messages.

Compare this complete current program with the preferred construction in the corresponding chapter. Identify which lines express the research design and which exist only to serialize, schedule, authorize state reads, recover execution, or reshape results.

```
"""Rumor diffusion over a network using a viewer-filtered SharedLog."""

from pathlib import Path

from edsl import Agent, AgentList, Model, QuestionFreeText, Survey
from edsl.sharedstate import FileStateStore, SharedLog, SharedState
from edsl.sharedstate.steps import StepContext

def network_agents() -> AgentList:
    return AgentList(
        [
            Agent(
                name="Alice",
                traits={
                    "neighbors": ["Ben", "Cara"],
                    "disposition": "excitable early adopter who readily shares interesting workp
                },
            ),
            Agent(
                name="Ben",
                traits={
                    "neighbors": ["Alice", "Dina"],
                    "disposition": "careful skeptic who flags uncertainty and dislikes overclaim
                },
            ),
            Agent(
                name="Cara",
                traits={
                    "neighbors": ["Alice", "Eli"],
                    "disposition": "social connector who retells stories vividly and confidently
                },
            ),
            Agent(
                name="Dina",
                traits={
                    "neighbors": ["Ben", "Eli"],
                    "disposition": "detail-oriented analyst who distinguishes evidence from hear
```

```

    ),
    Agent(
        name="Eli",
        traits={
            "neighbors": ["Cara", "Dina"],
            "disposition": "optimistic colleague inclined to interpret ambiguous news po
        },
    ),
]
)

def build_survey(state: SharedState) -> Survey:
    message = QuestionFreeText(
        question_name="network_message",
        question_text=(
            "Round {{ agent.diffusion_round }} of a workplace information-sharing "
            "simulation. You are {{ agent.name }}, a {{ agent.disposition }}. Your "
            "network neighbors are {{ agent.neighbors }}.\n\n"
            "Messages visible to you:\n{{ shared_state.messages.entries }}\n\n"
            "Send one concise message to your neighbors describing what you currently "
            "believe is happening. Preserve caveats you consider important, but retell "
            "the information naturally in your own voice. If you have no credible new "
            "information, say so. Do not mention this is a simulation."
        ),
    )
    return Survey(
        [
            message,
            state.messages.append(
                sender="{{ agent.name }}",
                recipients="{{ agent.neighbors }}",
                round="{{ agent.diffusion_round }}",
                message=message,
            ),
        ]
    )

def seed_rumor(state: SharedState) -> None:
    state.messages.append(
        sender="System",
        recipients=["Alice"],
        round=0,
        message=(
            "A friend in HR says leadership may announce a four-day workweek next "
            "month, but they did not see an official memo."
        ),
    )
).execute(StepContext({}, "seed"))

```

```

def run_diffusion(
    log_path: str | Path = "rumor-diffusion.jsonl",
    model_name: str = "gemini-2.5-flash",
) -> SharedState:
    state = SharedState(
        "workplace-network",
        FileStateStore(log_path),
        messages=SharedLog(visible_to="recipients"),
    )
    seed_rumor(state)
    agents = network_agents()
    model = Model(model_name)
    for round_number in range(1, 4):
        round_agents = agents.duplicate()
        for agent in round_agents:
            agent.traits["diffusion_round"] = round_number
            (
                build_survey(state)
                .by(round_agents)
                .by(model)
                .run(
                    interview_schedule="serial",
                    disable_remote_inference=True,
                    disable_remote_cache=True,
                    cache=False,
                    stop_on_exceptions=True,
                )
            )
    return state

if __name__ == "__main__":
    result = run_diffusion()
    print(result.render_markdown())

```

16.42 shared_state_strategic_planning_tiered.py

Focus: Tiered strategic planning. Inspect cross-tier visibility and provenance-preserving summaries.

Compare this complete current program with the preferred construction in the corresponding chapter. Identify which lines express the research design and which exist only to serialize, schedule, authorize state reads, recover execution, or reshape results.

```

"""Tiered strategic planning with feasible-package deliberation and selection."""

from itertools import product
from pathlib import Path

```

```

from edsl import (
    Model,
    QuestionDict,
    QuestionFreeText,
    QuestionMatrix,
    QuestionNumerical,
    QuestionRank,
    Survey,
)
from edsl.sharedstate import FileStateStore, SharedLog, SharedState

from shared_state_strategic_planning_workshop import (
    BUDGET,
    challenge_survey,
    executives,
    proposal_entries,
    proposal_survey,
    require_count,
)

TIERS = ("minimum", "target", "expanded")

def tier_survey(state: SharedState) -> Survey:
    minimum_cost = QuestionNumerical(
        question_name="minimum_cost",
        question_text=(
            "You are {{ agent.name }}. Review your proposal and all challenges:\n"
            "{{ shared_state.proposals.entries }}\n{{ shared_state.challenges.entries }}\n\n"
            "Set a minimum viable cost from 10 to 25 units."
        ),
        min_value=10,
        max_value=25,
    )
    minimum_outcome = QuestionFreeText(
        question_name="minimum_outcome",
        question_text="State the concrete deliverable at minimum funding in at most 30 words.",
    )
    target_cost = QuestionNumerical(
        question_name="target_cost",
        question_text="Set a target cost from 26 to 40 units.",
        min_value=26,
        max_value=40,
    )
    target_outcome = QuestionFreeText(
        question_name="target_outcome",
        question_text=(
            "State the additional measurable outcome delivered at target funding, "

```

```

        "beyond the minimum tier, in at most 30 words."
    ),
)
expanded_cost = QuestionNumerical(
    question_name="expanded_cost",
    question_text="Set an expanded cost from 41 to 60 units.",
    min_value=41,
    max_value=60,
)
expanded_outcome = QuestionFreeText(
    question_name="expanded_outcome",
    question_text=(
        "State the additional outcome delivered at expanded funding, beyond the "
        "target tier, in at most 30 words."
    ),
)
controls = QuestionDict(
    question_name="tier_controls",
    question_text=(
        "Provide one metric shared across tiers, the largest risk, its mitigation, "
        "and any hard dependency. Keep each field under 30 words."
    ),
    answer_keys=["metric", "risk", "mitigation", "dependency"],
    value_types=["str", "str", "str", "str"],
    value_descriptions=[
        "Measurable success metric",
        "Largest risk",
        "Concrete mitigation",
        "Hard dependency or none",
    ],
)
)
return Survey(
    [
        minimum_cost,
        minimum_outcome,
        target_cost,
        target_outcome,
        expanded_cost,
        expanded_outcome,
        controls,
        state.tiers.append(
            proposal_id="{{ agent.proposal_id }}",
            sponsor="{{ agent.name }}",
            minimum_cost=minimum_cost,
            minimum_outcome=minimum_outcome,
            target_cost=target_cost,
            target_outcome=target_outcome,
            expanded_cost=expanded_cost,
            expanded_outcome=expanded_outcome,

```



```

        controls=controls,
    ),
]
)

def tier_entries(state: SharedState) -> dict[str, dict]:
    return {
        entry["proposal_id"]: entry for entry in state.read().state["tiers"]["entries"]
    }

def tier_options(state: SharedState) -> list[dict]:
    proposals = {entry["proposal_id"]: entry for entry in proposal_entries(state)}
    options = []
    for proposal_id, tier_plan in sorted(tier_entries(state).items()):
        for tier in TIERS:
            options.append(
                {
                    "id": f"{proposal_id}-{tier}",
                    "proposal_id": proposal_id,
                    "tier": tier,
                    "title": proposals[proposal_id]["title"],
                    "sponsor": proposals[proposal_id]["sponsor"],
                    "cost": tier_plan[f"{tier}_cost"],
                    "outcome": tier_plan[f"{tier}_outcome"],
                }
            )
    return options

def tier_voting_survey(state: SharedState, options: list[dict]) -> Survey:
    proposal_ids = sorted({option["proposal_id"] for option in options})
    slate = "\n".join(
        f"{option['id']}: {option['title']} - {option['cost']} units; {option['outcome']}"
        for option in options
    )
    vote = QuestionMatrix(
        question_name="tier_votes",
        question_text=(
            "You are {{ agent.name }}, the {{ agent.role }}. Rate the value-for-cost "
            "of each initiative after comparing its three funding tiers. The total "
            f"portfolio budget is {BUDGET}.\n\n{slate}"
        ),
        question_items=proposal_ids,
        question_options=["high", "medium", "low"],
        randomize_items=True,
    )
    return Survey([vote, state.tier_votes.append(voter="{{ agent.name }}", votes=vote)])

```

```

def candidate_portfolios(
    state: SharedState, options: list[dict], limit=3
) -> list[dict]:
    by_proposal = {}
    for option in options:
        by_proposal.setdefault(option["proposal_id"], []).append(option)
    weights = {"high": 2, "medium": 1, "low": 0}
    initiative_support = {proposal_id: 0 for proposal_id in by_proposal}
    for entry in state.read().state["tier_votes"]["entries"]:
        for proposal_id, vote in entry["votes"].items():
            initiative_support[proposal_id] += weights[vote]
    delivery_multiplier = {"minimum": 0.65, "target": 0.85, "expanded": 1.0}
    support = {
        option["id"]: initiative_support[option["proposal_id"]]
        * delivery_multiplier[option["tier"]]
        for option in options
    }
    candidates = []
    proposal_ids = sorted(by_proposal)
    choices = [[None, *by_proposal[proposal_id]] for proposal_id in proposal_ids]
    for selected in product(*choices):
        selected = [option for option in selected if option is not None]
        cost = sum(option["cost"] for option in selected)
        if not selected or cost > BUDGET:
            continue
        score = sum(support[option["id"]] for option in selected)
        candidates.append(
            {
                "options": selected,
                "cost": cost,
                "support": score,
                "initiative_count": len(selected),
            }
        )
    candidates.sort(
        key=lambda item: (
            -item["support"],
            -item["initiative_count"],
            item["cost"],
            [option["id"] for option in item["options"]],
        )
    )
    finalists = []
    seen = set()
    for candidate in candidates:
        signature = tuple(option["id"] for option in candidate["options"])
        if signature not in seen:

```

```

        finalists.append(candidate | {"id": f"Portfolio {len(finalists) + 1}"})
        seen.add(signature)
    if len(finalists) == limit:
        break
return finalists

def format_portfolios(portfolios: list[dict]) -> str:
    return "\n".join(
        f"{{portfolio['id']}} ({{portfolio['cost']}}/{BUDGET}, initial support "
        f"{{portfolio['support']}}): "
        + "; ".join(
            f"{{option['id']}} {{option['title']}}" for option in portfolio["options"]
        )
        for portfolio in portfolios
    )

def package_discussion_survey(state: SharedState, portfolios: list[dict]) -> Survey:
    statement = QuestionFreeText(
        question_name="package_statement",
        question_text=(
            "You are {{ agent.name }}, the {{ agent.role }}. Debate the feasible "
            f"packages below, not isolated pet projects.\n\n{{format_portfolios(portfolios)}}\n\n"
            "Prior package discussion:\n\n{{ shared_state.package_discussion.entries }}\n\n"
            "Identify one cross-initiative complement, conflict, dependency, or omitted "
            "opportunity that should determine the final package. At most 60 words."
        ),
    )
    return Survey(
        [
            statement,
            state.package_discussion.append(
                speaker="{{ agent.name }}", statement=statement
            ),
        ]
    )

def package_ballot_survey(state: SharedState, portfolios: list[dict]) -> Survey:
    ballot = QuestionRank(
        question_name="package_ranking",
        question_text=(
            "Privately rank all feasible packages after reviewing the package-level "
            f"discussion.\n\n{{format_portfolios(portfolios)}}\n\nDiscussion:\n"
            "{{ shared_state.package_discussion.entries }}"
        ),
        question_options=[portfolio["id"] for portfolio in portfolios],
    )

```

```

return Survey(
    [
        ballot,
        state.package_votes.append(voter="{{ agent.name }}", ranking=ballot),
    ]
)

def select_package(state: SharedState, portfolios: list[dict]) -> tuple[dict, dict]:
    scores = {portfolio["id"]: 0 for portfolio in portfolios}
    for entry in state.read().state["package_votes"]["entries"]:
        for index, package_id in enumerate(entry["ranking"]):
            scores[package_id] += len(portfolios) - index - 1
    winner_id = sorted(scores, key=lambda item: (-scores[item], item))[0]
    return next(item for item in portfolios if item["id"] == winner_id), scores

def run_tiered_workshop(
    log_path: str | Path = "strategic-planning-tiered.jsonl",
    model_name: str = "gemini-2.5-flash",
) -> tuple[SharedState, dict, dict]:
    state = SharedState(
        "tiered-strategic-planning",
        FileStateStore(log_path),
        proposals=SharedLog(),
        challenges=SharedLog(),
        tiers=SharedLog(),
        tier_votes=SharedLog(),
        package_discussion=SharedLog(),
        package_votes=SharedLog(),
    )
    agents = executives()
    model = Model(model_name)
    run_options = {
        "disable_remote_inference": True,
        "disable_remote_cache": True,
        "cache": False,
        "stop_on_exceptions": True,
    }
    if state.read().state["proposals"]["count"] < 5:
        proposal_survey(state).by(agents).by(model).run(**run_options)
    require_count(state, "proposals", 5)
    if state.read().state["challenges"]["count"] < 5:
        challenge_survey(state).by(agents).by(model).run(
            interview_schedule="serial", **run_options
        )
    require_count(state, "challenges", 5)
    if state.read().state["tiers"]["count"] < 5:
        tier_survey(state).by(agents).by(model).run(**run_options)

```

```

require_count(state, "tiers", 5)
options = tier_options(state)
if state.read().state["tier_votes"]["count"] < 5:
    tier_voting_survey(state, options).by(agents).by(model).run(**run_options)
require_count(state, "tier_votes", 5)
portfolios = candidate_portfolios(state, options)
if state.read().state["package_discussion"]["count"] < 5:
    package_discussion_survey(state, portfolios).by(agents).by(model).run(
        interview_schedule="serial", **run_options
    )
require_count(state, "package_discussion", 5)
if state.read().state["package_votes"]["count"] < 5:
    package_ballot_survey(state, portfolios).by(agents).by(model).run(**run_options)
require_count(state, "package_votes", 5)
winner, scores = select_package(state, portfolios)
state.close()
return state, winner, scores

if __name__ == "__main__":
    shared_state, selected, tally = run_tiered_workshop()
    print(shared_state.render_markdown())
    print(f"\nSelected {selected['id']} ({selected['cost']}/{BUDGET})")
    for option in selected["options"]:
        print(f"- {option['id']}: {option['title']} - {option['outcome']}")
    print(f"Package Borda tally: {tally}")

```

16.43 shared_state_strategic_planning_workshop.py

Focus: Propose–challenge–revise–vote. Inspect typed phases and stable proposal identifiers.

Compare this complete current program with the preferred construction in the corresponding chapter. Identify which lines express the research design and which exist only to serialize, schedule, authorize state reads, recover execution, or reshape results.

```

"""A strategic-planning workshop that funds a portfolio under a fixed budget."""

from itertools import combinations
from pathlib import Path

from edsl import (
    Agent,
    AgentList,
    Model,
    QuestionDict,
    QuestionFreeText,
    QuestionMatrix,
    QuestionMultipleChoice,
    QuestionNumerical,

```

```

    Survey,
)
from eds1.sharedstate import FileStateStore, SharedLog, SharedState

BUDGET = 100

def executives() -> AgentList:
    specs = [
        ("Maya", "CEO", "P1", "company strategy and durable differentiation"),
        (
            "Eli",
            "Engineering VP",
            "P2",
            "reliability, platform leverage, and execution capacity",
        ),
        (
            "Sofia",
            "Sales VP",
            "P3",
            "revenue growth, customer commitments, and market access",
        ),
        (
            "Priya",
            "Product VP",
            "P4",
            "customer value, adoption, and coherent product direction",
        ),
        (
            "Noah",
            "Finance VP",
            "P5",
            "capital efficiency, risk, and measurable returns",
        ),
    ]
    return AgentList(
        [
            Agent(
                name=name,
                traits={
                    "role": role,
                    "proposal_id": proposal_id,
                    "priority": priority,
                },
            )
            for name, role, proposal_id, priority in specs
        ]
    )

```

```

def proposal_survey(state: SharedState) -> Survey:
    title = QuestionFreeText(
        question_name="initiative_title",
        question_text=(
            "You are {{ agent.name }}, the {{ agent.role }}, focused on "
            "{{ agent.priority }}. Propose one distinctive strategic initiative for "
            "the next year. Give only a concrete title of at most 10 words."
        ),
    )
    cost = QuestionNumerical(
        question_name="initial_cost",
        question_text=(
            "Estimate the initiative's required budget units from 20 to 60. The "
            f"company has {BUDGET} units total for all initiatives."
        ),
        min_value=20,
        max_value=60,
    )
    case = QuestionDict(
        question_name="initiative_case",
        question_text=(
            "Build a concise business case for {{ initiative_title.answer }} costing "
            "{{ initial_cost.answer }} units. Use at most 35 words per field."
        ),
        answer_keys=["outcome", "metric", "largest_risk", "dependency"],
        value_types=["str", "str", "str", "str"],
        value_descriptions=[
            "Expected strategic outcome",
            "One measurable success metric",
            "Largest execution or market risk",
            "Important dependency, or none",
        ],
    )
    return Survey(
        [
            title,
            cost,
            case,
            state.proposals.append(
                proposal_id="{{ agent.proposal_id }}",
                sponsor="{{ agent.name }}",
                title=title,
                cost=cost,
                business_case=case,
            ),
        ]
    )

```

```

def challenge_survey(state: SharedState) -> Survey:
    proposal_ids = [entry["proposal_id"] for entry in proposal_entries(state)]
    target = QuestionMultipleChoice(
        question_name="challenge_target",
        question_text=(
            "You are {{ agent.name }}, the {{ agent.role }}. Review the proposals and "
            "public challenges so far:\n{{ shared_state.proposals.entries }}\n"
            "{{ shared_state.challenges.entries }}\n\nSelect another sponsor's proposal "
            "whose assumptions most need scrutiny."
        ),
        question_options=proposal_ids,
    )
    challenge = QuestionFreeText(
        question_name="strategic_challenge",
        question_text=(
            "Challenge one specific assumption in {{ challenge_target.answer }}. "
            "Explain the decision-relevant risk or opportunity cost in at most 55 words."
        ),
    )
    return Survey(
        [
            target,
            challenge,
            state.challenges.append(
                challenger="{{ agent.name }}", target=target, challenge=challenge
            ),
        ]
    )

def revision_survey(state: SharedState) -> Survey:
    cost = QuestionNumerical(
        question_name="revised_cost",
        question_text=(
            "You are {{ agent.name }} revising {{ agent.proposal_id }}. Review all "
            "proposals and challenges:\n{{ shared_state.proposals.entries }}\n"
            "{{ shared_state.challenges.entries }}\n\nSubmit a defensible revised cost "
            "from 20 to 60 units."
        ),
        min_value=20,
        max_value=60,
    )
    revision = QuestionDict(
        question_name="revised_case",
        question_text=(
            "Revise your initiative after the challenges. In at most 40 words per "
            "field, state the refined outcome, metric, and concrete risk mitigation."
        )
    )

```



```

    ),
    answer_keys=["outcome", "metric", "risk_mitigation"],
    value_types=["str", "str", "str"],
    value_descriptions=[
        "Refined outcome",
        "Measurable success metric",
        "Response to the most important challenge",
    ],
)
return Survey(
    [
        cost,
        revision,
        state.revisions.append(
            proposal_id="{{ agent.proposal_id }}",
            sponsor="{{ agent.name }}",
            cost=cost,
            revised_case=revision,
        ),
    ]
)

def voting_survey(state: SharedState) -> Survey:
    proposals = {entry["proposal_id"]: entry for entry in proposal_entries(state)}
    revisions = {entry["proposal_id"]: entry for entry in revision_entries(state)}
    slate = "\n".join(
        f"{proposal_id}: {proposal['title']} ({revisions[proposal_id]['cost']} units) - "
        f"{revisions[proposal_id]['revised_case']}"
        for proposal_id, proposal in sorted(proposals.items())
    )
    vote = QuestionMatrix(
        question_name="portfolio_votes",
        question_text=(
            "You are {{ agent.name }}, the {{ agent.role }}. Rate each revised "
            f"initiative for inclusion in a {BUDGET}-unit strategic portfolio. Judge "
            f"all proposals, including your own.\n\n{slate}"
        ),
        question_items=sorted(proposals),
        question_options=["high", "medium", "low"],
        randomize_items=True,
    )
    return Survey([vote, state.votes.append(voter="{{ agent.name }}", votes=vote)])

def proposal_entries(state: SharedState) -> list[dict]:
    return state.read().state["proposals"]["entries"]

```

```

def revision_entries(state: SharedState) -> list[dict]:
    return state.read().state["revisions"]["entries"]

def select_portfolio(state: SharedState) -> tuple[list[dict], int, int]:
    proposals = {entry["proposal_id"]: entry for entry in proposal_entries(state)}
    revisions = {entry["proposal_id"]: entry for entry in revision_entries(state)}
    weights = {"high": 2, "medium": 1, "low": 0}
    scores = {proposal_id: 0 for proposal_id in proposals}
    for entry in state.read().state["votes"]["entries"]:
        for proposal_id, vote in entry["votes"].items():
            scores[proposal_id] += weights[vote]
    feasible = []
    ids = sorted(proposals)
    for size in range(len(ids) + 1):
        for subset in combinations(ids, size):
            cost = sum(revisions[item]["cost"] for item in subset)
            if cost <= BUDGET:
                score = sum(scores[item] for item in subset)
                feasible.append((score, len(subset), -cost, subset))
    score, _, negative_cost, selected_ids = max(feasible)
    selected = [
        {
            "proposal_id": proposal_id,
            "title": proposals[proposal_id]["title"],
            "sponsor": proposals[proposal_id]["sponsor"],
            "cost": revisions[proposal_id]["cost"],
            "support_score": scores[proposal_id],
            **revisions[proposal_id]["revised_case"],
        }
        for proposal_id in selected_ids
    ]
    return selected, -negative_cost, score

def require_count(state: SharedState, primitive: str, expected: int) -> None:
    count = state.read().state[primitive]["count"]
    if count < expected:
        raise RuntimeError(
            f"{primitive} phase incomplete: expected {expected} persisted records, got {count}"
        )

def run_strategic_workshop(
    log_path: str | Path = "strategic-planning-workshop.jsonl",
    model_name: str = "gemini-2.5-flash",
) -> tuple[SharedState, list[dict], int, int]:
    state = SharedState(
        "annual-strategic-planning",
    )

```

```

        FileStateStore(log_path),
        proposals=SharedLog(),
        challenges=SharedLog(),
        revisions=SharedLog(),
        votes=SharedLog(),
    )
    agents = executives()
    model = Model(model_name)
    options = {
        "disable_remote_inference": True,
        "disable_remote_cache": True,
        "cache": False,
        "stop_on_exceptions": True,
    }
    proposal_survey(state).by(agents).by(model).run(**options)
    require_count(state, "proposals", 5)
    challenge_survey(state).by(agents).by(model).run(
        interview_schedule="serial", **options
    )
    require_count(state, "challenges", 5)
    revision_survey(state).by(agents).by(model).run(**options)
    require_count(state, "revisions", 5)
    voting_survey(state).by(agents).by(model).run(**options)
    require_count(state, "votes", 5)
    selected, cost, score = select_portfolio(state)
    state.close()
    return state, selected, cost, score

if __name__ == "__main__":
    shared_state, portfolio, total_cost, support = run_strategic_workshop()
    print(shared_state.render_markdown())
    print(f"\nFunded portfolio ({total_cost}/{BUDGET}, support {support}):")
    for initiative in portfolio:
        print(
            f"- {initiative['proposal_id']} {initiative['title']} - "
            f"{initiative['cost']} units, support {initiative['support_score']}/10"
        )

```

Chapter 17

Closing perspective

The preferred interface should make the scientifically important decisions obvious: assignment, information, timing, legal action, settlement, stopping, and failure. It should make bookkeeping decisions disappear: parsing logs, reconstructing payoffs, matching role strings, counting completion rows, and copying run options.

The examples show that EDSL does not need a single grand “multi-agent” object. It needs a small number of composable, checked concepts with an inspectable expansion. Researchers should be able to begin with a named recipe, inspect its fields and stages, replace one component, validate the result without model calls, and save a complete run artifact for replication.